Hands-On System Design

Learn System Design, Scaling Applications, Software Development Design Patterns with Real Use-Cases



HARSH KUMAR RAMCHANDANI

bpb

Hands-On System Design

Learn System Design, Scaling Applications, Software Development Design Patterns with Real Use-Cases



HARSH KUMAR RAMCHANDANI

bpb

Hands-On System Design

Learn System Design, Scaling Applications, Software Development Design Patterns with Real Use-Cases

---

Harsh Kumar Ramchandani



www.bpbonline.com

FIRST EDITION 2022

Copyright © BPB Publications, India

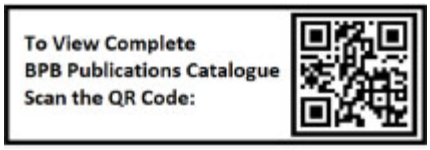
ISBN: 978-93-55512-369

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.



www.bpbonline.com

Dedicated to

My beloved Parents:

Shri Ashok Ramchandani
Pushplata Ramchandani

My Wife: ***Aishwarya***

&

My Siblings: ***Bharti*** *&* ***Kamal***

About the Author

Harsh Kumar Ramchandani is a Software Engineer with rich experience of more than six years. He has worked in numerous fields like Web Application Development, Compiler Development, Android Application Development, Performance Optimization, and RTL simulation. He enjoys writing code in his free time and loves reading about new tools and technologies in the industry. He earned his bachelor's degree in computer science from the Netaji Subhas University of Technology, formerly Netaji Subhas Institute of Technology (Delhi University), and has published a research paper as well. He is a coding enthusiast, keen to learn new things, and occasionally indulges in writing poems and playing badminton.

About the Reviewers

With over 13 years of experience in the software industry, **Shrey Khurana** is an Engineering Manager, an enthusiast in Cloud and Web Development. He currently works at R1RCM as a Manager - Engineering. His skill in Web Application Development, Cloud, and more has seen him become a Microsoft MVP in Developer Technologies. Shrey is also a problem solver on Angular, React, Azure, SQL, .Net, Core, Selenium, etc. He now focuses on channeling his knowledge into software architecture and sharing it with the community by mentoring, creating POCS, running workshops, writing blogs to help make the world a better and more developed place. His motto is to simplify the problems the developers face and hence make the software life cycle enjoyable for them.

Suraj Agarwal has 12+ years of industry experience working on scalable distributed systems. He is currently working with Google as Senior Engineering Manager and has previously been employed with Amazon, Inmobi, Housing.com, and NetApp.

He has extensive experience working on high throughput systems across adtech, e-commerce, fintech, and retail. He has built and scaled systems to 50k+ QPS and led large engineering teams.

Acknowledgement

I encountered many challenges while writing this book but I feel the learnings and knowledge I amassed while writing the book is the biggest reward I could earn. I can begin to thank many people in this journey but one thing that truly helped me is the love and support from my dear ones.

I am eternally grateful to my parents as their continuous encouragement and support pushed me to work hard and pursue new learning opportunities. All those learnings helped me a lot in writing this book.

I would like to thank my wife for her immense aid and valuable inputs while writing this book. I sought her help when I was stuck or when I needed some clarity about a particular topic. The healthy technical discussions with her helped me to write things in a clear and concise manner.

I am wholeheartedly thankful to the technical reviewers for their meticulous analysis of the book.

Our gratitude also extends to the team at BPB Publications for being supportive enough to provide us with time to finish this book and assist us wherever we had doubts in completing any aspect of this book.

Preface

The idea of writing this book originated in my mind when I was reading a tech blog that discussed building a social media website. I was not able to absorb all the content at once as there were many grey areas in my system designing mindset. Post that I appeared in some system designing discussions and I couldn't design a satisfactory scalable system. As curious as I was to learn something new, I got inquisitive and started reading various blogs and books to increase my knowledge. Slowly, reading tech blogs became a habit and I found it convenient to document this knowledge in a book.

The book is meant to serve programmers – beginners or mid-senior level and students who are willing to learn the key aspect of scalable system designing. I have learned that this knowledge will be helpful to you, first, professionally, and second, in your system design interviews.

The book is primarily divided into three parts. The first part of the book contains five chapters that explain the fundamentals of system designing. It discusses various technologies like Cache memory, Messaging queues, Databases, etc. which are used to build modern-day scalable distributed systems. It also touches on some core networking concepts like the OSI model, important protocols like HTTP and REST architecture. Furthermore, different strategies to scale databases horizontally and vertically are also covered. In addition, an emphasis on different architectural styles – monolithic, service-oriented, and microservices is put to help a user understand the pros and cons of each one. Also, Object-oriented principles are discussed and explained with C++-style code snippets.

The second part of the book takes the responsibility of explaining – how to start the system design in a practical way. It contains three chapters where the first chapter explains the various design patterns used in the industry with C++ style code snippets. It explains different behavioral, creational, and structural design patterns. The next chapter describes the basic tools and techniques to design a system in an object-oriented way. It describes how to define system requirements and how to use tools like class diagrams to depict important objects of the system. Finally, the tools and techniques

discussed are used to design a system so that readers can understand how to put in use everything they just learned.

The final part of the book contains five chapters and each chapter contains a dedicated system design problem. In each chapter, a feasible solution to the system design problem is discussed using the techniques and technologies learned in the previous two sections.

Readers must understand that there is no single solution to any system design problem. Also, the more you read, the better you become in system designing. The book is just a needle in the haystack and is designed to build a solid foundation for system designers.

Coloured Images

Please follow the link to download the
Coloured Images of the book:

https://rebrand.ly/e97690

We have code bundles from our rich catalogue of books and videos available at **https://github.com/bpbpublications**. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at **www.bpbonline.com** and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at: **business@bpbonline.com** for more details.

At **www.bpbonline.com**, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive

exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at **business@bpbonline.com** with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit **www.bpbonline.com**. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit **www.bpbonline.com**.

Table of Contents

1. Fundamentals of System Design Principles

- Introduction
- Structure
- Objectives
- Introduction to distributed scalable systems
- Bottlenecks in designing scalable systems
- Performance and scalability
- Principles of scaling data access
  - *Cache*
    - *Local cache*
    - *Global cache*
    - *Distributed cache*
  - *Proxy servers*
- Data indexing
  - *Load balancers*
    - *Types of load balancers*
    - *Load balancer algorithms*
  - *Asynchronism*
- Content Delivery Network (CDN)
- CAP theorem
- Databases
  - *Relational and non-relational databases*
  - *Some other important databases*
    - *Spatial database*
    - *Graph database*
- Redundancy
- Conclusion
- Points to remember
- Multiple choice questions
  - *Answers*
- Questions
- Key terms

2. Networking Basics
- Introduction
- Structure
- Objectives
- Introduction to network
- OSI model
  - *Physical layer*
  - *Data-link layer*
  - *Network layer*
  - *Transport layer*
  - *Session layer*
  - *Presentation layer*
  - *Application layer*
- Hypertext Transfer Protocol (HTTP)
- Transmission Control Protocol
- User Datagram Protocol
- Introduction to REST architecture
- Conclusion
- Points to remember
- Multiple choice questions
  - *Answers*
- Questions
- Key terms

3. Scaling Databases
- Introduction
- Structure
- Objectives
- Horizontal scaling
- Vertical scaling
- Database scaling methods
  - *Read replicas*
  - *Multi-master replica*
  - *Connection pooling*
  - *Connection load balancing*
- Conclusion
- Points to remember

Multiple choice questions
Answers
Questions
Key terms

4. System Architecture
Introduction
Structure
Objectives
Monolithic systems
Service-oriented architecture
SOA implementation approaches
Microservices
Microservices versus SOA
Microservices versus monolithic systems
CI/CD for microservices
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

5. Introduction to OOPs
Introduction
Structure
Objectives
Introduction to OOPs
OOPs features
Class
Object
Inheritance
Single inheritance
Multi-level inheritance
Multiple inheritance
Hierarchical inheritance
Hybrid inheritance

Polymorphism
Abstraction
Encapsulation
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

6. Design Patterns
Introduction
Structure
Objectives
Design patterns and their uses
Types of design patterns
Creational
Object pool
Prototype
Singleton
Behavioral
Chain of responsibility
Mediator
Observer
Structural
Bridge
Decorator
Flyweight
Proxy
Conclusion
Points to remember
Multiple choice questions
Answers
Questions

7. Object-Oriented Design Methodology
Introduction

Structure
Objectives
Insight into object-oriented design methodology
FURPS+ requirements
Performing object-oriented analysis and design
Defining system requirements
Class diagrams
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

8. Approaching System Design Problems
Introduction
Structure
Objectives
Getting started
Requirement gathering
Back of the envelope calculations
Scaling the design
SQL versus NoSQL
Identifying the algorithm
Partitioning the data
Cache
Database cleanup
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

9. Design a Key-Value Data Store
Introduction
Structure

Objectives
Defining the system requirements
Back-of-the-envelope calculations
Creating a scalable system
Consistency or availability
High-level design
Scaling the database
Durability and consistency
Re-defining the high-level design
Conclusion
Points to remember
Multiple choice question
Answer
Questions
Key term

10. Design a File Hosting Service
Introduction
Structure
Objectives
Defining system requirements
Creating scalable system
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

11. Design a Business Communication Tool
Introduction
Structure
Objectives
Defining system requirements
Back-of-the-envelope calculations
Designing a scalable system
Storing messages: SQL or NoSQL?

Scaling data access
Conclusion
Points to remember
Multiple choice question
Answer
Questions
Key terms

12. Design a Card Authentication System
Introduction
Structure
Objectives
Defining system requirements
Back-of-the-envelope calculations
Creating scalable system design
Scaling the design components
Load balancer
Cache
Security
Conclusion
Points to remember
Multiple choice questions
Answers
Questions
Key terms

13. Design an Image Hosting Application
Introduction
Structure
Objectives
Defining system requirements
Creating a scalable system
Conclusion
Points to remember
Multiple choice question
Answer
Questions

Key terms

Index

CHAPTER 1

Fundamentals of System Design Principles

Introduction

In today's world, technology usage is ubiquitous. There are a plethora of systems out there that solve our day-to-day problems, such as booking a cab, buying groceries online, and connecting to a distant friend. Designing such systems is the need of the hour, and it requires an understanding of various topics. A novice system designer may be able to design such a system; however, creating a system that works even well for a large number of users is the real challenge. This chapter discusses all the bottlenecks that a system designer may face while creating a system that supports a large number of users. Such systems can also be known as distributed scalable systems if they are distributed across different networks and scale well for a large number of users. Various technologies, methodologies, and principles that a system designer can use to design distributed scalable systems are explained in this chapter.

Structure

In this chapter, we will cover the following topics:

- Introduction to distributed scalable systems
- Bottlenecks in designing scalable system
- Performance and scalability
- Principles of scaling data access
- Content Delivery Network (CDN)
- CAP theorem
- Databases
- Redundancy

Objectives

After studying this unit, you will be able to understand the meaning of distributed scalable systems and the challenges in designing distributed scalable systems. You will also learn about different technologies one should use to overcome the hurdles of designing large-scale distributed systems.

Introduction to distributed scalable systems

A computer is typically used to perform various operations in today's world. An operation can be as simple as performing an addition operation or as complex as solving a convoluted mathematical problem. With the increase in computing power and the breadth of problems that can be solved by today's computer, there is a need to have a group of computers that can work together to solve a common problem. These groups of computers are typically called computer networks. A network may not necessarily have computers. We may have a network of physical servers, virtual machines, or even databases as well.

A system can be any application that solves a business problem or provides a specific service. For example, an e-commerce website like Amazon provides a platform to buy various things. A social media website like Facebook allows a user to connect with her friends—chat with them, do a video call and even share pictures, videos, and so on.

A distributed system is a system with its components located across different machines. These machines communicate with each other following some protocols and form a network. We can have various components of the system located across different machines or even the same components replicated on different machines.

A distributed system offers a plethora of advantages. First, as we keep various components on different machines, the workload is distributed. This makes the system more efficient. Second, if we replicate a component of the system on two or more machines, then in the case failure occurs on one machine, we have other machines to perform the task. So, there is no single point of failure. This will help the system to be more available. Third, assigning a specific job to each machine makes debugging easy as we know which component to look for if the system fails to work. Also, from the

management perspective, we may have a different set of teams working simultaneously on the different components of the system. Such breakdown is crucial from the point of accountability, and it also allows us to delegate a specific task to a single team. Therefore, many large organizations adopt this model as this helps in expeditious resolution of issues that may arise in the day-to-day working of the system. Additionally, this helps in the clean separation of different tasks.

However, distributed systems have certain tradeoffs as well. First, there will be some latency as different components of systems communicate with each other via some protocols. Second, there is an added cost with each machine.

A distributed system, in order to cater to the needs of several users, must be able to respond to the user with minimal delay. It might be the case that a distributed system works well with a small number of users, but if the user base increases, then the system may become unresponsive. Such a situation, though undesirable, often happens if system designing is not done by keeping in mind the scalability aspect. A distributed scalable system works efficiently even if the numbers of users are large, ensuring minimum turnaround time for each request. To sum up, a distributed scalable system is a system whose components are located across the network, and it works well for a large number of users. This book aims to cover all those aspects that one must keep in mind to design scalable distributed systems.

Bottlenecks in designing scalable systems

System designing is never limited to just developing the system. Generally, whenever a system is designed, the driving factor is to solve a problem or to provide a service to the user such as booking a cab or ordering food or groceries or clothes using the mobile phone that gets directly delivered to the user at their home. A system designer has to ensure that the system developed is capable enough to serve a large number of users because any business thrives on the user base only. Hence, it is imperative to design a scalable system that works well not only for a small number but also for a large number of users. However, creating such distributed scalable system requires a system designer to take care of several parameters, which are discussed as follow:

Availability: Availability denotes if the system is available to serve a user request. Consider a Simple distributed system that takes a mathematical

equation from the user, computes a result, and returns it back to the user. It might be the case that mathematical equation can be as simple as 4+4 or as complex as 4^99 + 3^20 + log89 + *n*th root of (893554656) (where ^ is power operation). The system can work well if traffic to the system is low because each user can be assigned a machine to do the operation and return back the result. However, when traffic is quite large with each user requesting complex computations, then each machine allocated for the task may take a significant amount of time to produce the result. For a case where there are M machines in the system and M user requests at a given time, then the system will ensure availability for all M users. But the moment there is a spike in the number of users (where user requests > M)—the user will have to wait for the system to get free. This will increase the turnaround time for the requests and may have a negative impact on the overall business. So, ensuring availability is an important point in any scalable distributed system.

Database access: Eventually, any distributed scalable system can grow in size. Consider another scenario, an e-commerce website providing a platform to the user to order products. Such systems usually store various attributes of a product such as a price, quantity, reviews, specifications, and so on in a database. If the website offers a large number of products, then the database storing all this information will be huge in size (storing TBs of data), and every database read/write operation will be very expensive in terms of time. Now consider a typical scenario, where a user visits the website, searches a few products, and selects a couple of them for purchase. If we observe carefully, for each product a user is viewing, we need to make a database read query to retrieve its attributes (price, images, specs, and so on). A database read operation may work well if our database is small in size and the number of active users is less. However, when we have a large number of active users, there would be several databases read queries, thus, degrading the overall application's performance. So, for such a system to be scalable, some other storage unit must be deployed between the application and the database.

Concurrent users: Concurrency denotes if a system is able to handle a large number of users at the same time. Considering the same example of an e-commerce website where a large number of users are trying to read the specifications of a particular product. If we deal with such requests in a naïve way, then we end up queuing a large number of read requests for a

single product, and it will boil down to a database read operation for the same input many times. A scalable system should employ some intelligent ways to handle such situations.

Manageability: Manageability denotes how efficiently a system can detect failures and how efficiently it can recover from them. In any large-scale distributed system, errors/failures/bugs are bound to occur. A bug-free system is a myth. No matter how much effort a system designer invests, the error will occur in a system. So, the focus should not only be to make the system error-free but also to make sure how to recover a system from an erroneous state. We know a distributed system consists of a large number of components that interact with each other. Debugging issues in such a system can be a daunting task. So, a system designer must keep all these points in mind while laying out the architecture of the system.

Reliability: Reliability implies that the system will perform the task assigned to it in a satisfactory manner and will produce consistent and correct results even with increased load. A distributed system generally consists of a distributed database, one that spans across multiple geographic locations, forming a database network. A write operation on such a distributed database cannot be done in a single go as doing it will be very expensive in terms of time. So, data consistency problems may arise in such scenarios, which will make the system unreliable.

Latency: Latency denotes how long a system would take to respond to a user's request. A distributed scalable system must ensure minimal latency as taking an unreasonable amount of time to respond to a user's request can give a huge blow to the reputation. For example, any two users chatting over a social media app like Facebook would want their messages to get delivered to one another as quickly as possible. Any delay can totally ruin their experience, which in turn may drive them away from using the application in the future. So, a distributed scalable system must be designed by keeping in mind that latency should be minimum.

Cost: We all know that by pooling more money, one can always buy more hardware. A naïve way to solve the problem of dealing with a large number of users is to add more hardware. Such a solution though it works for some use cases, might not be feasible for all businesses as there is an added cost of buying and maintaining each new hardware unit.

All these factors must be kept in mind while designing any distributed scalable systems. However, one must also note that at times it is not possible to incorporate all these, and you might have to let go of one or a few of them.

Performance and scalability

Performance and scalability are two important terms that one often heard a lot in their day-to-day technical life. Performance implies how fast or slow a system works for a single user. However, scalability signifies how a fast or slow system works for a large number of users. So, if a system is not scalable, then it means that system might work fast for one user but does not work fast under a heavy load.

A system is said to be scalable if adding more resources to it results in increased performance. The increase in performance signifies that the system would respond quickly even if incoming requests are very large.

A system designer must keep scalability in mind while architecting the system. Scalability must not be an afterthought because applications become very hard to scale if they were not designed initially, keeping this parameter in mind. Sometimes, a system works well with a small number of users; however, as the number of requests increases or data in the database grows, the system does not work well in terms of turnaround time.

Scalability is applicable to different parts of the system. We can scale access to the system scale access to the database queries (read/write). Scaling access to the system denotes that every user request is admitted and processed by the system in a reasonable time. Scaling database implies that request for database read/write must be efficient in terms of time for a large set of users. All these concepts will be explained in detail in subsequent chapters.

Principles of scaling data access

Most web applications can be overly simplified to the following form:

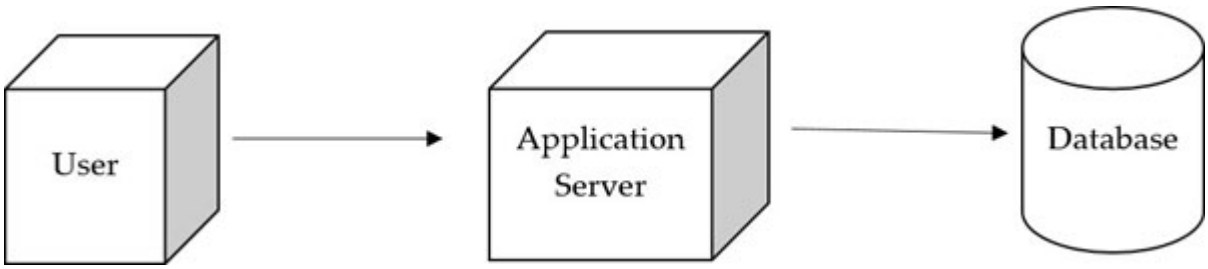


Figure 1.1: *A simple web application architecture*

A user sends a request to the system or application server. The application server then sends a request to the database, which returns a result, and the server finally sends the response back to the user.

This system design may work well for a few users. However, with growing data in the database and large user requests, access to the application server and database must be scaled as well. A system designed like this can be easily overwhelmed with a growing number of requests. For example, consider a scenario where we have tons of data in our database. Assuming each database access is taking more than 30 seconds, and if at any point of time the traffic to the system is of order 1,000, then each writes a request to a database will take 30×1,000, which is equal to 30,000 seconds; that is, close to 8.4 hours! No business can afford such a huge turnaround time. So, some kind of technologies must be deployed between client and server and between server and database to scale the system to handle such user requests.

Subsequent sections cover various concepts such as cache, indexes, queues, load balancers, and proxy servers for scaling access to data and application servers.

Cache

A cache is a high-speed memory that is used to store data. The crux of using cache is simple: in order to prevent access to a slower data store, it stores the result of recent requests in high-speed memory to respond to the user as quickly as possible. Cache takes advantage of the locality of reference principle, which says, recently accessed data is likely to be accessed again.

The cache is widely used. They are used in modern computers as well. Cache memory is a piece of hardware that sits between the processor and

main memory. The processor first checks the requested data in the cache memory. If it is present, data is returned from the cache, and it is called a cache hit; otherwise, it is a cache miss, and requested data is returned from the main memory.

Caches are widely used in the distributed system to scale access to the database. In the following sections, different forms in which cache is used to leverage database access are discussed.

Local cache

In a local caching scheme, cache memory is present in each node of the distributed system. So, whenever a user request lands on an application server, instead of querying the database directly, it will first check its cache. If it is a cache-hit, then the server returns the result to the user.

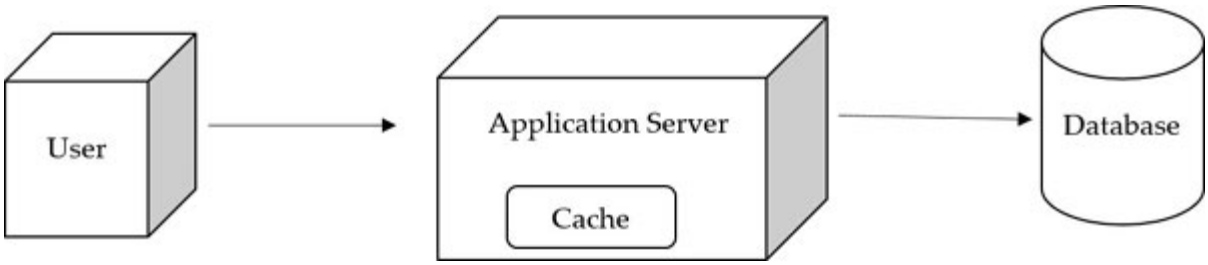


Figure 1.2: *Local cache*

The problem with this local caching scheme is that if a user's request is routed to a particular server, the result is not already cached, and then the server will cache the result for future requests. However, if next time the same request is routed to a different server, then again, a cache miss will occur. We may have a chain of events like this that may degrade the system's performance. Additionally, the problem with the local caching scheme is that keeping all local caches consistent is very hard. Proper cache eviction strategies must be deployed to make sure all local caches contain the most recent data only. In order to overcome this situation, a global cache can be used.

Global cache

In the global caching scheme, we have a global cache. Each requested data is first searched in the global cache, and if it is present in the cache, then it

is returned. However, if we have a cache miss, that is, if data is not present in the cache, then proper strategies must be deployed to handle it. This scheme can have two variations to deal with the situation when requested data is not present in the global cache.

In one variation, it is the responsibility of the global cache to retrieve the result from the database in case of a cache miss and return it to the server. Such an arrangement is shown as follows:

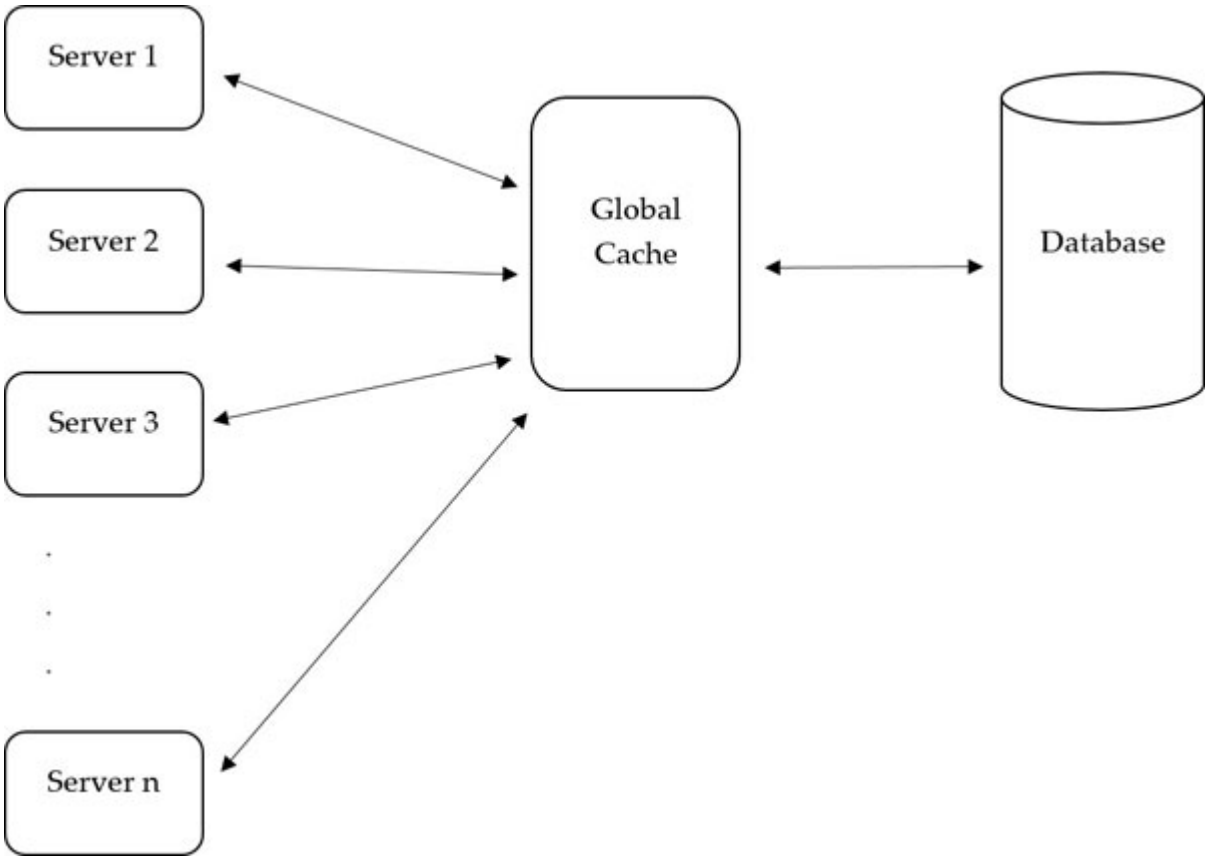


Figure 1.3: *Global cache with cache itself retrieving data from database in case of a cache miss*

In the other variation, the server will first query the global cache; in case of a cache miss, the server itself queries the database to get the result. Such an arrangement would be something like:

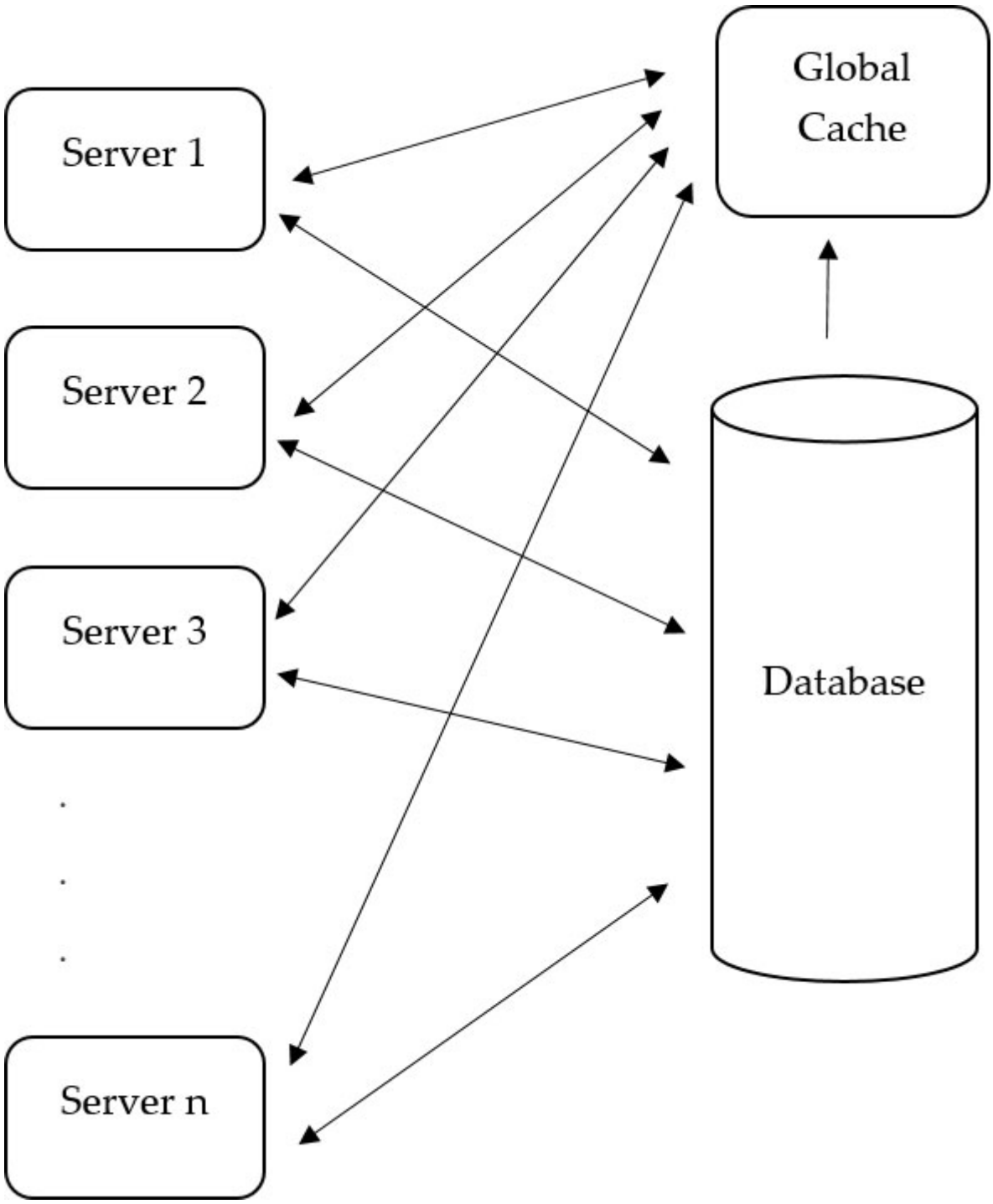


Figure 1.4: *Global cache with server retrieving data from database instead of cache in case of cache miss*

As one can see, in the second variation, there is some complexity added to the system as the system has to take care of fetching data from the cache, and if it is not present, then getting it from the main data store. Also, there

is an added responsibility to update the cache as well. With the first organization, the cache itself manages all this, and hence, the system is less complicated. Thus, many applications resort to using the first variation as cache itself manages everything.

Distributed cache

In the distributed caching scheme, data is cached on multiple servers or nodes which is known as a cache cluster. Here, some form of hashing scheme is used to distribute data across different nodes in a cluster. Hashing is used to locate the node in the cache cluster where the requested data is present. Also, data is replicated on some nodes within the cluster. Replication is essential so that even if one node goes down, we have another node with the same cached data.

One such popular open-source cache is Memcached which can be used as both local as well as a distributed cache.

Proxy servers

In a typical distributed system, a client makes a request, and one of the available servers is connected to fulfill the request. It is very easy to overwhelm the servers with this sort of arrangement as every application will have a limited set of servers, and generally, the number of user requests will be huge. To pass this hurdle and to make our system work efficiently at scale, we need to devise some other techniques.

A proxy server is an intermediate piece of hardware or software, and it sits between the client and the actual servers.

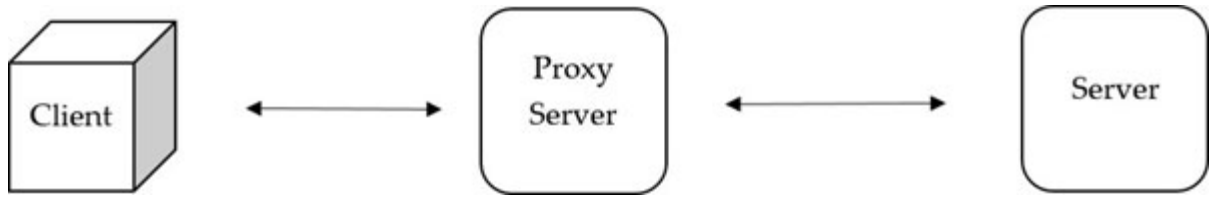


Figure 1.5: *Using proxy servers for scaling application access*

Now, every request is passing through from the client to the proxy server, and it is responsible for directing it to the actual backend server. We can add

some logic to our proxy servers and reduce the burden on our actual servers. There are various ways to achieve this.

One way to achieve this is request collapsing. Imagine we have an e-commerce website. The new mobile is just launched. Everyone is looking forward to buying it. Now, it is very likely that at the moment, several requests are being made just for that one particular product. Here, in such scenarios, we can leverage the proxy servers. Proxy servers, before actually routing our request to the backend servers, can collapse all the similar requests and create a single request. So, if at any point in time millions of requests are generated from the client-side, we have only one request being made to the server. This can be immensely helpful in reducing the load on our servers and making our system more available.

Another way is instead of collapsing the request that are requesting the same data; we can also collapse the requests, which are requesting the data that is spatially together in the main data store. The point is generally when we read data from the database we get an entire disk block in one read operation. Since one disk block is made up of small data parts such as data1, data2, … data*n* and if several requests demand data parts from one disk block only, it makes more sense to collapse such request as we can save several expensive databases read operations. Hence, by collapsing such requests, we need only one disk access operation. This will bring tremendous gain if we have several TBs of data stored in our database.

There are many popular proxy servers available in the industry. A few of them are Squid and Varnish, which are quite popular.

Note: Like Proxy Servers, the reverse proxy also exists. A proxy server acts as an intermediary server between the client and the actual server. It receives a request from the client, filters it, and then sends it to the specific backend server. In contrast, a reverse proxy sends the request to the servers and retrieves resources from them on behalf of the client. They are used very commonly to hide the data originating servers as they return the resources to the client, which appears to be coming from the actual server itself. Reverse proxies provide a lot of advantages as follows:

- **Hiding the original server.**

- **Helps in reducing the burden on the original server by caching commonly requested data.**
- **It can secure access by adding HTTP access authentication to a server that does not have it inherently.**

Data indexing

A trivial search algorithm operating on a data store will search every piece of data in the database to find the result. Such an algorithm, although simple, is highly inefficient if we have several TBs of data in our system. Indexing is one way to improve data retrieval operations. It does so at the cost of additional memory where it keeps the indexing data structure. Indexing is often used for databases. One or more columns of the database are used for the database indexing.

An index data structure typically has an indexing key (made using a combination of one or more table columns) and the reference or pointer which points to the actual data in the data store.

For example, in a database that has a Student's details such as their names, pin code, class, subjects, and so on, and supporting queries like finding students living in a particular region can have an index data structure with indexing key as pin code and data reference to a disk block which contain all the students in that particular pin code. So, a query like giving names of all students living in the pin code 11005 will first lead to a search in the index data structure; if entry is found in the index data structure, then data reference field is accessed, which will point to a disk block containing all the students in that pin code. As we can see, search overhead will be significantly reduced in this case as instead of searching linearly, we can reduce the search time to constant time operation by using some intelligent techniques like hashing while implementing our index data structure.

We must note while creating an index data structure; it is important to consider how data will be stored in the database and how the database will be queried. For example, in the previous discussion of students' databases with their names, addresses, and so on. It makes more sense to organize data by order of pin code if we know queries are more of the form like giving names of students who live in the pin code xyz. However, if one is

not storing data in the database by their pin code, then even indexing might not be very helpful. So, in order to leverage the indexing technique, one must consider: How the database will be queried and how we are storing data in one disk block.

Load balancers

Load Balancers are a piece of hardware or software that are used to distribute the application traffic across a number of servers. Load balancers play a crucial role by increasing the capacity and reliability of the system. It ensures that no single server is overburdened by evenly distributing the traffic on the network. Using a load balancer helps a system deal with a large number of concurrent users.

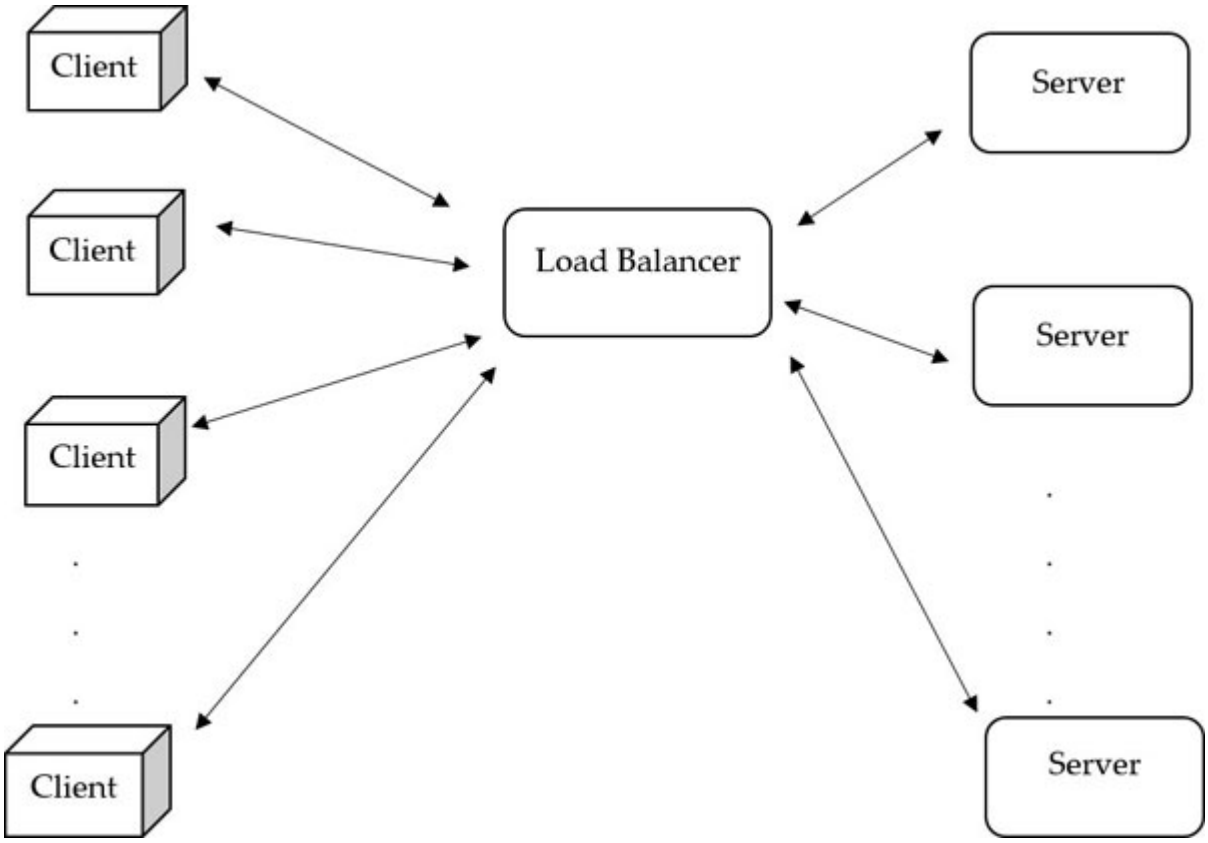


Figure 1.6: *Load balancers*

Types of load balancers

Load balancers are generally grouped into two categories: Layer 4 and Layer 7.

Application (Layer 7)
Presentation (Layer 6)
Session (Layer 5)
Transport (Layer 4)
Network (Layer 3)
Data link (Layer 2)
Physical (Layer 1)

Table 1.1: *OSI Model*

Layer 4 and Layer 7 denote the transport and application layer of the OSI model.

L4 (or Layer 4) load balancer operates at the transport (and network) layer. Whenever any client sends a request, it creates a **Transmission Connection Protocol** (**TCP**) connection with the load balancer. L4 load balancer inspects TCP stream to make a routing decision. It does not inspect the contents of the request to transfer it to some upstream servers.

L7 (or Layer 7) load balancer operates at the application layer. It makes the routing decision by inspecting the contents of the request. For example, we have a web application that hosts different videos; the user can read reviews of videos, watch videos or even purchase a video. Such a system can have various servers to serve different contents. So, in such a scenario, a user requesting content like images or videos can be directed to a server that is optimized to serve multimedia content. A user who wants to see order logs can be routed to a server that is responsible for transactional logs.

Load balancer algorithms

A load balancer can use different algorithms to route traffic across a number of servers. Some common algorithms are Round Robin, Weighted Round Robin, or even selecting a random node.

In the Round Robin technique, the load balancer rotates requests among servers in the order they are received. For example, a system with three servers, A, B, and C: The first request is routed to A, second to B, and third

to C, and Load Balancer continues to pass the request to the server in this order.

In Weighted Round Robin, each server has a weight associated with it. So, if A, B, and C have weights in the ratio of 4:2:1 then the first four requests are sent to A, the next 2 to B, and then one request to C. This order is again repeated for further requests.

In random selection, one node is selected at random and assigned to serve the client's request.

Further, load balancing algorithms can be categorized as static and dynamic.

A static algorithm does not take into account the state of the system. The state of the system implies the load level of the nodes. Instead, the number of processing units available with the node, their communication speed, and power are known beforehand, along with the arrival time and resource requirements of each request. Static algorithms work well if incoming requests are fairly similar in nature.

In a dynamic algorithm, routing decisions are made according to the current load level of a particular node. So, at any point in time, an incoming request can be dynamically routed from an overloaded node to a node that is less overloaded.

Asynchronism

Imagine a system where each client requests a task to be processed. The server takes the task, computes the result, and returns it back to the client. A simple model like this works well if the server receives a request that it can handle. However, as soon as the server's bandwidth is exhausted, it cannot take any more tasks. All further requests will be queued up. These requests, which are queued, will be allocated processing unit only when some in-process task is completed. Until then, the client who has made a request will be essentially blocked. Such systems are known as synchronous systems.

This kind of synchronous behavior degrades the application performance, which could be detrimental from the business perspective. The client is stuck, doing no work at all, and is simply waiting for a system to get free. Also, adding more servers in the system may result in the same scenario, if in the future, the user base increases. This can act as a temporary solution

but might not work in the long term. In order to solve this problem, we need some abstraction between the client and server so that no client is forced to wait. As soon as a client sends a request, it must be acknowledged that his request is admitted in the system, and the server should be notified when the request is completed so that it can send back the response to the client. This behavior is known as asynchronism and is very useful as it does not block the client unnecessarily.

In order to achieve asynchronism, messaging queues can be used in the system. With messaging queues in the system, the system behaves in the following way:

- The client sends a request.
- If no server is available to fulfill it immediately, then the request is stored in the queue.
- The client is notified that his request has been received. The client is free to continue his work.
- One of the worker threads picks up the task from the queue, performs the necessary operation, and returns the result.
- The system sends the response to the client.

Queues can be used in a wide range of situations. Consider a write-heavy system like an Image upload platform available to users in different countries. A user uploads the image, and other users can view the image, comment on it, like it, and share it further. A system like this will have a huge database of images and related content. Each image upload operation translates to updating that image in several databases, which are located in different regions. If the implementation of a system is done in a manner that each image upload operation must update all databases (which already have TBs of data), then this will give a blow to the system availability for further requests. Queues are quite useful in such use cases. Instead of blocking the user, it notifies the user that image upload is in progress, let him do other work, store the image upload operation in the queue. One of the worker threads picks it up and finishes the upload operation. Once the write is successful, the user will be notified.

Queues can be used in online ticket booking platforms as well. Once a user finishes his bookings, a task to notify the user via email or phone can be

stored in the queue, and the system can do this asynchronously once one of the nodes/servers is available.

Content Delivery Network (CDN)

A CDN is one more asset used to create scalable distributed systems. To understand the crux of CDN, consider a video-streaming system like YouTube. A system/website that serves different videos, movies, or TV Shows to the user across the globe. A system like this serving multimedia content should be highly optimized to serve videos to the users in different geographical locations. It would be awful if every user request to watch a video is routed to a distant central server because transferring multimedia content back to the client is expensive. CDN is best suited for this use case. When we use CDN in our system, instead of routing the user request to watch a video to some distant servers, we try to find a server that is nearest to the user. A server in CDN may have a copy of the video cached already, or it will get it from the originating server and then cache it so that the user would be able to watch the video with minimal latency.

The following picture explains the difference between a network with a single server and the network using the CDN scheme.

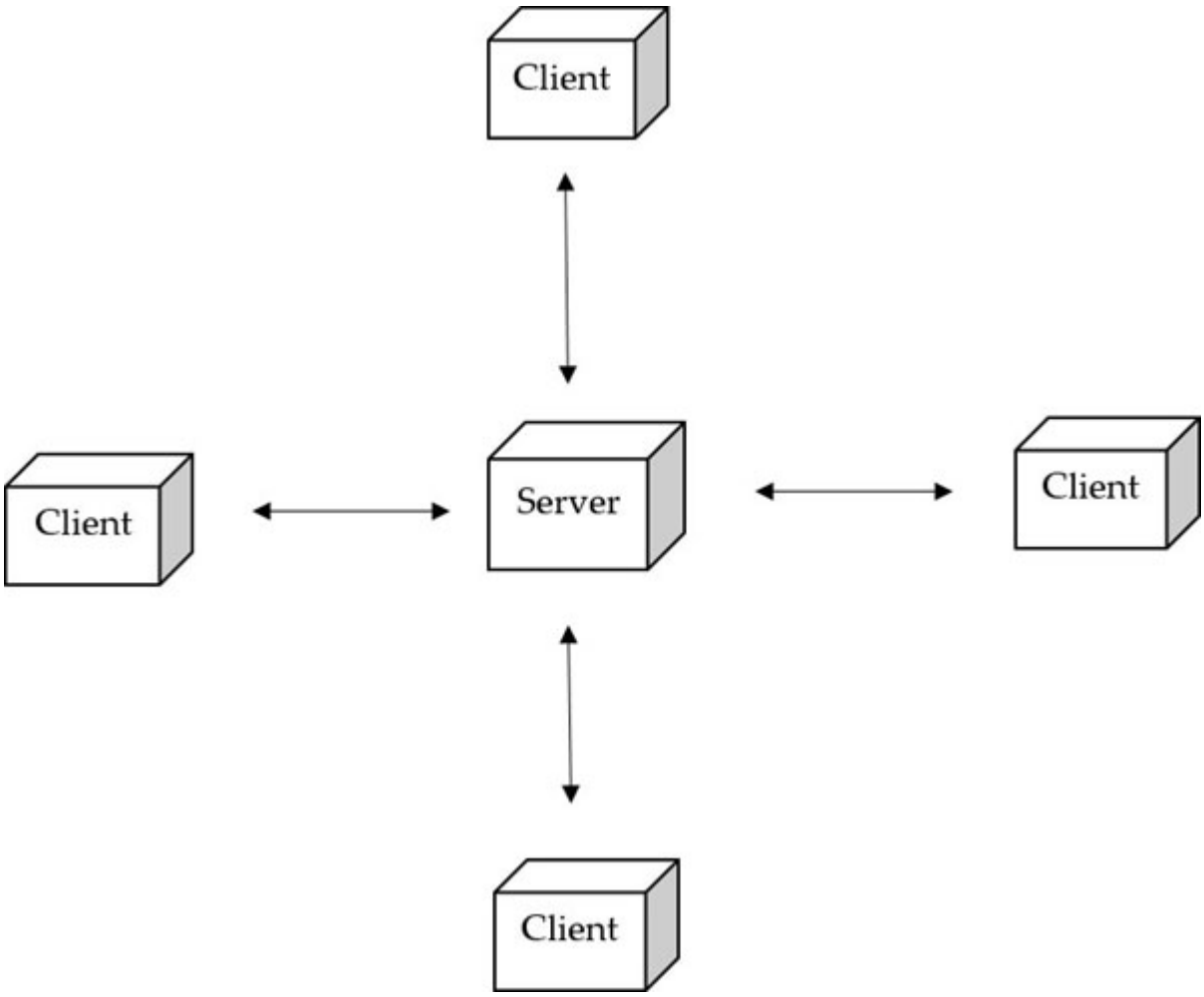


Figure 1.7: *A network with a single server and without CDN*

The abovementioned scheme represents a simple hypothetical view of a network where we have a single server, and it serves multiple clients. We can easily deduce that in such an arrangement, the server will be heavily overloaded if all clients request resources at the same time. The situation will get worse if the server responsible for serving the content is located in an entirely different geographical location. To deal with such use cases, CDN can be used. CDN ensures that servers responsible for serving content are located close to the user. Such an arrangement works really well if the system is responsible for serving multimedia content like videos and photographs, where bandwidth and latency play a crucial role from the cost as well as user's experience perspective.

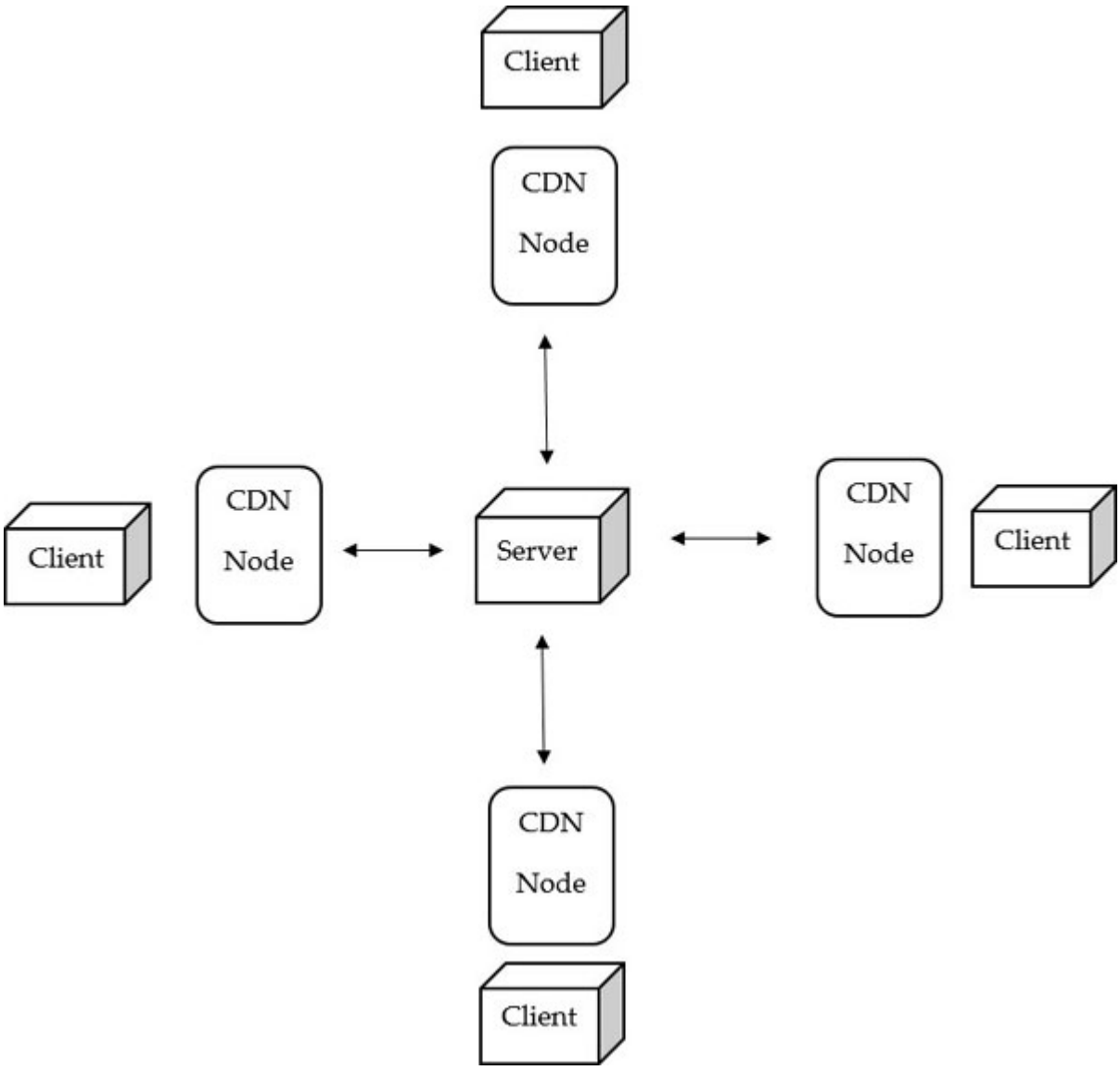


Figure 1.8: *CDN scheme in a network*

Formally, CDN is distributed network of servers. The goal of CDN is to provide high performance by distributing the service close to the end-user. Over a period of time, CDN is used extensively to solve several performance problems of the Internet. Today, CDN is used to serve text, graphics, media files, software, and documents as well. CDN helps to reduce site load time and latency.

In order to leverage the CDN, the content creators or business owners pay CDN to deliver their content to the end-user. CDN owners, in turn, pay the Internet Service Providers to host their servers in their data centers. When it boils down to serving content to the user, the CDN scheme algorithmically

finds a CDN node that is nearest and optimal to serve the user. Hence, the CDN node that is best for serving the request of a user is chosen. A variety of factors play a role in this, like the number of hops in the path between client and CDN node, the best available CDN node in terms of performance, and latency in transferring the content.

CAP theorem

Consider that you start a new cake shop that receives orders telephonically. Since you do not have much money at this point, you started all by yourself, advertising your phone number in posters posted at different places. Slowly, people start calling you, placing orders, and you are delivering them the cake. It is all going well, and slowly more and more people are calling every day, placing their cake orders. One fine day, a lot of customers called and lashed at you, saying that they tried to call you many times yesterday and still could not connect with you. You realized that since yesterday you received a lot of calls, so some calls could not connect. You realized that you were not available to some users, and being unavailable could be detrimental to your business. An idea strikes your mind, you called up your close friend and asked him to pitch in, and he agreed. So, you updated your system in such a way that whenever a customer calls, his call will be routed either to your phone or your friend's phone, depending upon which one is available. Now, you feel satisfied as even if you are busy, then your friend would be able to take incoming calls. You are happy that availability (A) is taken care of.

One day a customer calls, and you are the one who picks it up. The customer asks about the status of his order; you tell him to wait, you check your entry but could not find anything. You ask him to reconfirm, which makes him furious, and he cuts the call and cancels his order. You have no idea what happened. You rushed to your friend's desk ask him if any such customer had placed an order today, to which he says, yes! Suddenly, you understand where you went wrong. You forgot to take care of consistency (C). The entries lying with you are not recent. It might be the case that some customer whose call is picked by you for an order may cancel his order at a later point in time, but this call may be routed to your friend's phone. Also, as you saw today, the order was placed on your friend's phone, but a status query came to your phone, and you were unaware of the entry. You realized

that entries lying with you and your friends must be recent at all times. So, you decided that you will now make sure that all the entries will be noted in a shared document like Google Docs, and both of you will refer to it while answering customer queries.

Your business is going well now. You have ensured consistency and availability. You are making lots of profits, and you are sure that with Christmas coming, you will generate a lot of profit, and using these profits, you can buy your favorite car. However, your friend fell ill the same day and could not join you. You realize that today you have to give up availability; however, you will have consistency. This is partition tolerance (P).

With these series of events, you now realize that you can either have two out of three – that is, your system can be either CA, CP, or AP. However, you do know that things like sickness or falling ill are unavoidable (partition tolerance). These are bound to happen, so deep down your heart, you know that you will always have to take care of this parameter (P).

To sum it up, CAP Theorem says that a distributed system can have two out of these three properties:

- **Consistency**: Every read must always get a most recent write-only.
- **Availability**: It implies that every user's request should get a response.
- **Partition tolerance**: It means that even if there is a communication break between any two nodes in the network, then also the system should continue to operate.

Practically, an AC system does not exist (even if people claim it to be) as network failures, packet drops are bound to happen. So, most of the systems are either AP or CP. Choosing between availability or consistency purely depends on your business needs. For instance, if we are designing a banking system, then consistency is very important as we cannot allow a transaction of 500 USD when the user has 200 USD in his account. However, making the system unavailable for some transactions is still okay rather than allowing an inconsistent transaction. A scenario where we can sacrifice consistency over availability is a Messaging app. If we are creating a Messaging app, then availability is important as it is totally fine if messages are coming out of order as long as every message is received.

Databases

Databases are a crucial component of any scalable distributed application. They are used to store application data. We cannot dump every data pertaining to the application on some disk or storage device because storing is only one side of the story. There may arise a need to retrieve this data at some later stage in time. As such, databases are needed because it is a collection of inter-related or un-related data, which helps in fast and efficient retrieval as well as storage of data. **Database Management System (DBMS)** is the software used to manage these databases.

DBMS provides various functions to manage the database. They are:

- **Data definition**: Deals with creation, deletion, and modification of the schema of the database.
- **Data manipulation**: Deals with insertion, removal, or modification of the data or entry in a database.
- **Data retrieval**: It helps an application or user with the retrieval of data from the database in a form as requested.
- **Data administration**: It helps to enforce constraints on the database, register and monitor users and deal with situations such as data corruption and unexpected system failure.

Relational and non-relational databases

Databases can be classified as relational and non-relational. Relational databases have a mathematical basis; they are based on a branch of algebraic theory known as relational algebra. In relational databases, we use tables to store the data. This is very different from the non-relational ones, where data are stored in some different forms. For example, some non-relational database stores data as a collection of JSON.

Some examples of relational databases are MySQL, PostgreSQL and some popular non-relational databases are NoSQL and MongoDB.

A very pertinent question that comes in any system designing process is to choose between relational and non-relational databases. There are a few points which we can consider in order to choose one:

The relational database has a predefined schema, and they are structured. For example, a book that has the name, age, phone number, and address of a person. However, non-relational databases have dynamic schema, and they are unstructured. For example, a folder that contains various information about a person, such as his name, address, age, shopping preferences, food habits, work time, and so on.

In relational databases, data is stored in tables, and each table is constituent of rows and columns. Columns specify the attributes, and each row contains information about one entity. In contrast with this, the non-relational database could be of various forms like a key-value store, wide-column database, document database, and so on.

In a relational database, we can use Structured Query Language to query the data. SQL is a powerful tool as it can perform various operations such as joining tables, querying data based on various filters, and so on. Non-relational databases have a different mechanism for interacting with the data, and that depends on how they are storing the data.

Generally, relational databases are **ACID** (**Atomicity, Consistency, Isolation, and Durability**) compliant, whereas most of the non-relational databases sacrifice the ACID compliance to ensure good performance and scalability.

To summarize, it makes more sense to choose a relational database if the business has predictable data in terms of structure and size, and the relation between entities is important. There are strategies like normalization for a relational database that limits data duplication as well. Non-relational databases are more suited where data is dynamic and follows no fixed structure. Additionally, relational databases are generally vertically scalable, and non-relational databases are horizontally scalable. So, non-relational databases can handle massive traffic by adding more database servers, and thus, they become an ideal choice for massive data.

Some other important databases

There are a plethora of databases available that a system designer can use depending on the use case. Some of them are discussed in the next section.

Spatial database

Sometimes, a need arises when you need to work with the geometric coordinates (latitude, longitude) of the user. Such use cases are very common for cab booking platforms like Uber, Ola, and so on. There is a category of databases known as spatial databases, which are optimized for the storage and query of the data pertaining to the geometric space. The spatial database supports a wide range of object representations ranging from simple geometric objects such as points, lines, and polygons to the more complex ones such as 3D objects, topological coverage, and networks.

Graph database

Any social media website like Facebook deals with a large number of users. Such a system not only needs user data, but the relationship between each user is equally important to support native queries. For instance, a query like finding friends of person A or finding mutual friends of A and B will be heavily used in any social media website, and computing results for it demands a system to store relationships between each user. In such use cases, a graph database can be used efficiently to process the query as it takes advantage of association between different objects and stores them in a manner that relationship queries work faster.

Graph databases are based on graph theory. A graph consists of a set where each item can be either vertex or an edge. A vertex represents an entity like a user, product, or anything that a business needs to store, and the edge represents the association between two vertices or entities. For example, a social media website may have a vertex representing a user and an edge between two vertices/users if they are friends with each other.

Redundancy

System failures or data corruption can happen with any distributed application. No matter how hard one tries, failures are bound to happen. Apart from making a system fault-tolerant, a system designer should also work on what to do when unexpected events of failure occur. There are many ways to deal with it, and redundancy is one of them.

Consider you are the owner of an e-commerce website. Your application takes orders from users across the globe. Some people order expensive things like mobile phones, cameras, laptops, and some people order less

expensive stuff like fruits, vegetables, and so on, but it does not matter to you because every order brings a share of revenue and profit. Now, suppose whenever a user orders something, you are storing the order details in a database. One fine day, the storage device where your database (the one storing order details) was residing crashed, leading to the complete loss of data. You tried a lot for data recovery, but that did not work. Customers who have already made payment and are expecting the delivery of their order start overwhelming your phone and call center. Unsatisfied by the response, they got furious. Such a situation is highly undesirable for any business. Redundancy can help a lot in such situations.

In order to deal with such scenarios, one must keep data redundant across several different databases at several different places. Keeping databases at different locations helps to recover in case of any physical hazard. Now, if at all your database crashes, you have its copy that you can use to recover quickly. The only pain a system designer has to take is to ensure how and when to take the backup of the data, how to diagnose if a failure has occurred, and how to automate the recovery process.

Apart from keeping data redundant, we can create redundancy for the application instances as well. For example, our application has one instance running on one machine. Due to some unknown reasons, the machine crashes, and the whole application goes down. In order to deal with such a crisis, we can keep multiple copies of application instances on different machines. A central unit can oversee the health of each application instance, and if anyone goes down, we have another instance readily available of the same application.

To summarize, redundancy is very crucial for a distributed system because a large downtime for a system can damage the company’s reputation.

Conclusion

A distributed scalable system is a system that works efficiently for a large number of users. Designing a distributed scalable system involves meticulously analyzing the bottlenecks that may arise during the design and then carefully eliminating some or all. These bottlenecks could be how you will make sure that the system does not hang if suddenly millions of users are hitting your system or how to make sure that your database does not blow up with a huge amount of data. Apart from this, latency, concurrency,

scaling database access are some other factors that must be taken care of. A designer can choose various technologies such as cache, proxy servers, load balancers, and CDN to create distributed scalable systems. Since scaling data access is important for a distributed scalable system so, this chapter also touches on databases. It explains what a database is, some important types of databases, what is redundancy, and why it is needed. Also, the CAP theorem is discussed that explains that any distributed system can be either consistent-available or available-partition tolerant or consistent-partition tolerant.

Points to remember

- A distributed scalable system is a system that is distributed (components located on different networks) and works efficiently for a large number of users.
- In order to design a scalable system, a system designer has to take care of many aspects such as availability, manageability, latency, cost, and so on.
- Performance and scalability though they appear similar, are two different terms. Performance implies how fast or slow a system works for a single user. However, scalability signifies how a fast or slow system works for a large number of users.
- Scaling data access is important to create scalable systems. There are various technologies to achieve this, such as cache, proxy servers, and load balancers.
- CDN is distributed network of servers. The goal of CDN is to provide high performance by distributing the service close to the end-user.
- Databases are a crucial component of any scalable distributed application. They are used to store application data. Databases can be relational or non-relational. Some important databases are spatial and graph databases.
- Redundancy is a technique to deal with abnormal situations like data corruption or data loss. Redundancy can be used with application instances as well as with databases.

- CAP theorem says that it is impossible for any distributed system to have more than two out of consistency, availability, and partition tolerance.

Multiple choice questions

1. **Which of the following is true for load balancer?**
    a. It is used to block incoming requests.
    b. It is used to store frequently accessed data.
    c. It is used to evenly distribute application traffic across a number of servers.
    d. It is used to collapse multiple requests into one single request.
2. **Layer 4 load balancer works at which layer of the OSI model?**
    a. Application layer
    b. Session layer
    c. Physical layer
    d. Transport layer
3. **What is cache memory used for?**
    a. It is used to store frequently accessed data
    b. It is used to prevent unauthorized access to data
    c. It is used to provide a backup in case the main data store fails
    d. It is used to establish a secure connection between client and a server

Answers

1. **c**
2. **d**
3. **a**

Questions

1. What is a distributed scalable system?
2. How does creating a scalable system different from a normal system?
3. What is a Load Balancer, and how does it work?
4. What are the different ways to scale access to data and applications?

Key terms

- **Graph**: A non-linear data structure consisting of two sets—vertices and edges, where vertices are a set of nodes and edges consist of an edge that connects two nodes.
- **TCP**: It is a protocol that defines how to establish and maintain a connection between two devices.

CHAPTER 2

Networking Basics

Introduction

A standalone system serves no useful purpose unless a user can communicate with it. Communication between two devices over a network involves a lot of steps. This chapter aims to cover all those steps in detail so that a user can relate how information flows from one device to other over a network. Apart from that, a few important networking protocols are prominent architecture, which is used to build scalable system is discussed.

Structure

In this chapter, we will cover the following topics:

- Introduction to network
- OSI model
- Hypertext transfer protocol
- Transmission control protocol
- User datagram protocol
- Introduction to REST architecture

Objectives

After studying this unit, you will be able to understand:

- The networking basics like OSI mode.
- Different networking protocols such as HTTP, TCP, and UDP
- The REST architecture

Introduction to network

A network is an interconnection of several devices. A device in a network can be a computer, router, switch, and so on. All the devices in a network communicate using some protocols to share information with each other. For example, we can have a network consisting of computers, a fax machine, and a printer. Any computer can send a print or fax command to the printer/fax machine using the protocols consistent with the network.

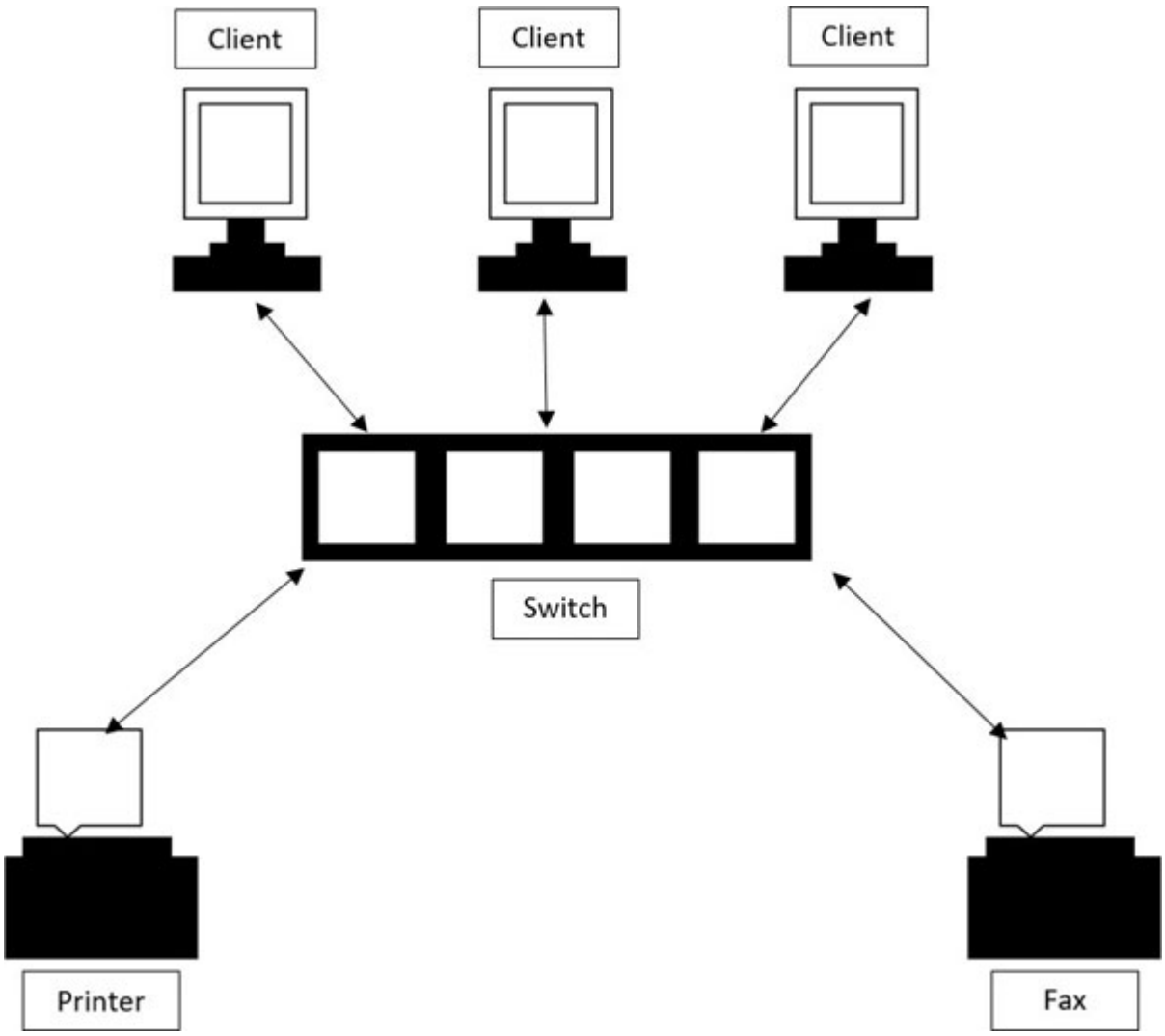


Figure 2.1: *A simple network*

In a simple term, protocol means a set of rules. A protocol defines how two entities in a network or across different networks communicate with each other. There are several networking protocols such as TCP, UDP, and HTTP. A system designer is expected to know the working of these protocols in order to choose the right protocol for a Web application.

Every device in the network has a unique address that is used when some other devices need to communicate or send some data to it. Any device, when it needs to send data, incorporates the destination address as well before sending the data. Also, data loss is very frequent during transmission, so the network has to take care of that as well. Sometimes, data is too large to send in one transmission. In such scenarios, data is segmented into smaller units and then sent. Also, upon reaching the destination, these units must be clubbed into one. There are many such manipulations that happen in the background whenever one device sends data to another device across a network. In order to understand these concepts, a conceptual reference model is known as **Open System Interconnection** (**OSI**) is developed to explain how information flows from one device to another.

OSI model

It is a conceptual model used to explain how information transfer occurs from one computer to another in a network. OSI model consists of seven layers, and each layer is assigned with a clear set of functions to perform

Following are the seven layers of the OSI model:

- Physical layer
- Data-link layer
- Network layer
- Transport layer
- Session layer
- Presentation layer
- Application layer

All these seven layers work in tandem and help in transferring the message from one computer to another across the network. The functions of each layer are described in detail in the next section.

Physical layer

It is the lowest and first layer of the OSI model. It defines the network topology, that is, how devices will be arranged in the network and the actual

physical connection between two devices. This layer is also responsible for transferring information in the form of bits between two devices. It converts the bits to electric, radio, or optical signals before transmission. Some of the devices that operate at the physical layer are the hub, repeater, modem, and cables.

The physical layer is concerned with several other things as follows:

- It determines the transmission mode, that is, whether it is simplex, half-duplex or full-duplex.
- It determines the line configuration, that is, whether it is point-to-point, multipoint, or point-to-multipoint.
- It helps two connected devices to agree over a set of parameters such as speed, duplex mode, and flow control. Because different devices in a network have different configurations, so they choose the best value of these parameters to achieve the highest performance transmission mode.
- It determines whether the communication will be in series or parallel. In series communication mode, only one bit is sent at a time, whereas, in parallel mode, multiple bits are sent.
- It specifies the bit rate, where bit rate implies the number of bits processed per unit of time.
- It ensures that the clock of sender and receiver is synchronized so that the bitstream is transmitted successfully.
- It defines the aspects and characteristics of the interface between devices and transmission media.

Note: Simplex, half-duplex, and full-duplex are different types of transmission modes. In simplex mode, communication is unidirectional; that is, only one device in a link can send data, and others can only receive it. In half-duplex, any device in a link can send/receive data, however, not simultaneously. So, at any point in time, one device can send data, and the other will receive it, and vice-versa. In full-duplex mode, communication can happen in both directions simultaneously. So, at any point in time, any device can send/receive data

Data-link layer

It is the second layer of the OSI model and is responsible for the movement of data from one node to the next in a network. The layer is also responsible for the detection and correction of the errors that may occur in the physical layer transfer. It also defines the right set of protocols which are concerned with how the connection will be established and terminated between two physically connected devices.

Data-link layer is divided into two sub-layers:

Logical Link Control (LLC)

Logical link control has the responsibility to ensure how different networking protocols can coexist in a multipoint network. Besides this, it also provides other functionalities like flow control, acknowledgment, and error notification.

Media Access Control (MAC)

The media access control layer has the responsibility to control the hardware providing the passage to the transmission medium. It basically ensures in what manner a device can get access to the medium and the permission required for transmitting data.

The data link layer provides different services listed as follow:

- It transforms the physical layer's bitstream to packets which are known as frames. It adds additional information like the source and the destination address to this frame.
- It adds additional data in the frame before sending it. By using this additional data, the receiver determines the correctness of the data. If there is any discrepancy, the receiver sends an acknowledgment for the re-transmission of the corrupted frame.
- In order to ensure that data corruption does not happen while transferring a frame from one device to another, the rate at which the sender sends data should be the same as the rate at which the receiver receives it; otherwise, a fast sender will overwhelm a slow receiver. The data link layer employs a technique known as flow control to prevent such kinds of data corruption.

- It provides protocols to determine which device has control over the channel (to which several devices are connected) at a given point in time.

Network layer

The network layer provides the means to transfer variable length packets from one device to another device located in a different network. We can define a network as a medium through which several devices are connected. All these devices in order to be uniquely identifiable, possess an address. Whenever a packet needs to be transferred, the sender node mentions the content and address of the destination node.

The network layer provides several services, which are listed as follow:

- It is responsible for finding the route from the sender to the receiver. It also has to take care of different traffic problems such as switching, routing, and so on.
- A network may be partitioned into sub-networks. The network layer is responsible for transmitting packets across these sub-networks using specialized devices like gateway or router.
- It introduces a session key between sender and receiver for security purposes.
- It ensures that the packets are delivered, so that is, the order in which they are sent is the same as the order in which they arrive at their destination.
- Unlike the data link layer that performs physical addressing, it implements logical addressing, which is also used to distinguish between source and destination.

Some important protocols operating at the network layer are Internet Protocol (IPv4 or IPv6), **Internet Control Message Protocol (ICMP)**, **Datagram Delivery Protocol (DDP)**, and so on.

Transport layer

The transport layer receives services from the network layer and is responsible for delivering the services to the application processes running

on different hosts. It serves as a logical communication path to the application processes running on two different hosts so that they can send a message to each other. Unlike the data link layer, which provides services within a network only the transport layer also provides services across networks.

Transport layer protocols are **Transmission Control Protocol** (**TCP**) and **User Datagram Protocol** (**UDP**). Layer 3 protocols, IP protocols, are responsible for the delivery of a packet from a source to its destination, hence, known as host-to-host protocols. However, transport layer protocols are known as port-to-port protocols because they are responsible for the delivery of packets from the port of the originating host to the port of the service running at the destination host.

Some important functions performed by the transport layer are listed as follow:

- It is responsible for the end-to-end delivery of the entire message from the sender to the receiver.
- Since errors are bound to occur during transmission, it retransmits lost and damaged packets.
- It provides flow control. Flow control prevents a sender from overwhelming a receiver in case a receiver is overloaded or unable to receive more messages. The transport layer controls the sending rate of data for maximizing performance and avoiding message re-transmission.
- In order to improve efficiency, the transport layer uses upward and downward multiplexing. In upward multiplexing, several transport layer connections use the same network connection, which makes it cost-effective. In downward multiplexing, one transport layer connection uses several network connections, which in turn increases throughput.
- It performs packet segmentation. Packet segmentation is the process of dividing a packet into smaller packets when the size of the packet is larger than the maximum transmission unit supported by a network.

Session layer

It is responsible for managing the connections (or dialogues) between two devices in a network. It manages the connection establishment, session maintenance, authentication, and security as well. It performs several functions, which are listed as follow:

- It creates and terminates the session between two systems and facilitates them to use the connection.
- It adds synchronization points in the data which are used for error identification. For example, if one system is sending a file with 100 pages, it will add synchronization points at the end of every 20 pages. This ensures that every transmission of 20 pages will be acknowledged if successfully received. So, if an error occurs at the transmission of the second batch, that is, 21–40 pages, then only a single batch of pages (21–40) will be resent as the first batch successful transmission receipt is already received to the sender.
- It allows the two connected devices to communicate in half-duplex or full-duplex mode.

Presentation layer

It is also known as the translation layer. Here, data is extracted from an application and transformed to the format suited for the transmission over the network.

Some of the functions performed by this layer are as follows:

- It performs data translation like ASCII to EBCDIC.
- It performs encryption/decryption of data.
- It performs data compression as well.
- It performs serialization of convoluted data structures into flat byte-strings.

The session layer can be sub-divided into two sub-layers:

- Common Application Service Element (CASE)
- Specific Application Service Element (SASE)

CASE sub-layer is responsible for ensuring services to the application layer and requesting services from the session layer. There are several application

services that receive support from CASE. Some of them are remote operation service element, reliable transfer service element, commitment concurrency, and recovery and association control service element.

SASE sub-layer is responsible for providing application-specific protocols like manufacturing messaging service, job transfer and manipulation, virtual terminal, and common management information protocol.

Some protocols that operate at the presentation layer are as follows:

- Telnet
- Network data representation
- Lightweight presentation protocol

Application layer

This layer is closest to the end-user. It serves as an interface and is responsible for displaying the information received to the end-user.

The application layer provides services listed as follow:

- It enables a user to use the network services.
- It performs message recovery and error handling as well.
- It is responsible for ensuring messages/e-mail formatting, file transfer, and providing user login services.

Several important protocols operate at this layer:

- Hypertext Transfer Protocol (HTTP)
- Simple Mail Transfer Protocol (SMTP)
- Domain Name System (DNS)
- TELNET
- File Transfer Protocol (FTP)

Hypertext Transfer Protocol (HTTP)

It is an application layer protocol that fetches resources like HTML documents. It is a request-response protocol; that is, a client requests a resource, and a server sends the response back. It serves as the foundation of any resource exchange system on the Web. HTTP being an extensible

protocol, is used to fetch not only hypertext but also images and videos as well. It is also used to post content to the server.

Some basic aspects of HTTP are as follow:

- HTTP is designed in a way that is human-readable and simple.

HTTP/2 is an efficient version of HTTP. HTTP semantics such as methods, status codes, header fields, and URIs are kept the same; however, the way data is framed and transferred between client and server is changed with HTTP/2.

- It is an extensible protocol as a new functionality can be introduced if the client and server concur to new header semantics.
- HTTP is stateless. So, no two requests have any relation between them. However, even if it is stateless, HTTP cookies can be used to make sessions stateful.
- A connection between two devices is created at the transport layer, which is needed before two devices can exchange HTTP requests/responses with each other. TCP and UDP are two protocols that establish a connection. TCP is a reliable connection, whereas UDP is not. By default, HTTP/1 opens a separate TCP connection for each HTTP request/response.
- HTTP can control how documents need to be cached. The server can instruct clients and proxies about the documents they need to cache and the duration of caching.

In a normal HTTP flow, when a client needs to communicate to a server following steps are involved:

- A client sets up a new TCP connection or reuses an existing one, or opens several connections to a server.
- An HTTP message is sent to a server. If HTTP/1 is used, then messages are human-readable; if HTTP/2 is used, then messages are encapsulated in a frame and then sent.
- The server sends a response back. The client reads the response, and it may close the connection.

Transmission Control Protocol

It is one of the most important protocols of the Internet protocol suite. It operates at Layer 4, that is, the transport layer of the OSI model. Since it is used on the top of **Internet Protocol** (**IP**), the combined stack is known as TCP/IP suite. It is useful as it solves problems with packet transfers like packet loss, corruption, and so on.

Various features of TCP protocol are listed as follows:

It is a connection-oriented protocol, implying that a connection is established between two devices before any communication occurs.

It uses congestion algorithms to prevent network congestions, and thus, reduces packet loss and improves network efficiency.

It is a reliable protocol as the sender is always notified whether the delivery of the message is successful or not.

It also ensures that data arrives in the same order as it was sent.

TCP is used extensively by the HTTP protocol.

A TCP segment consists of a header and data. TCP header may range from 20 to 60 bytes depending on the options field size.

Transmitting a packet with TCP is a multi-step process listed as follow:

1. **Establish a connection**: The two devices that need to send data to each other establish a connection using a three-way handshake mechanism. Once the handshaking mechanism is completed, they become ready to send/receive data.
2. **Send data packet**: Once the connection is established, the sender sends a packet that contains data and a sequence number. Once the destination receives the packet, it acknowledges it by setting an ACK bit inside a packet and also increments an acknowledgment number with an increment equal to the length of data. The sequence number and acknowledgment number are used to determine if the data sent is lost or received successfully.
3. **Close connection**: Any device can initiate the close connection request by sending a packet with the FINISH bit set. The other device acknowledges it by sending an acknowledgment signal and another FINISH bit.

User Datagram Protocol

UDP is another alternative to TCP in a network that uses IP. There are many differences between TCP and UDP, which are discussed as follow:

- TCP, as discussed in the previous section, is connection-oriented. However, unlike TCP, UDP is connectionless, so there is no overhead of opening, establishing, and terminating a connection. Also, UDP is quite useful for broadcast and multicast scenarios.
- Unlike TCP, UDP is not reliable as delivery of data from sender to receiver cannot be guaranteed.
- TCP ensures data arrives in order; however, UDP does not guarantee the same.
- A lost packet can be retransmitted in TCP, but it is not possible in UDP.
- The header length is variable in TCP (20–80 bytes), whereas, in UDP, it is fixed (8 bytes).
- UDP can be considered lightweight, fast, and simple than TCP.
- UDP has support for broadcasting, but TCP does not have that support.

Introduction to REST architecture

REST stands for **Representational State Transfer**. It is an architectural style that has some set of principles defining how to design loosely coupled applications using HTTP. It is often used in the development of Web services and understanding it will be useful for a system designer.

We can say that a RESTful Web application consists of two elements:

- A client who requests the resources.
- A server has the resources.

In order to create a RESTful Web service, there are six constraints that a system designer must follow. They are listed as follow:

- Client-server
- Uniform interface

- Stateless
- Cacheable
- Layered system
- Code on demand

Now, let us discuss the preceding-listed points in detail.

- **Client-server**: This constraint is as simple as it sounds—separating the client application and server application so that each can grow separately. Introducing this kind of separation helps a system designer to bifurcate the concerns; for example, user interface concerns will be no longer dependent on how one can scale the data store. Separation also helps to simplify the whole application model and makes it portable. The client just needs to know the resource's **URI** (**Uniform Resource Identifier**) in order to access it. Server-side changes can be made without affecting the client.
- **Uniform interface**: Every resource in the system will have a URI. The system should expose some APIs to the consumers to access these resources. A single resource should not be very large. Also, if needed, a resource should contain related URIs to fetch relevant information. A client, upon request, will be served with the resource from the servers' data store in the form of HTML, XML, or JSON. However, the resource representation sent to the client may not be the same as the server's internal representation. A client can get resources using common approaches like HTTP GET and can also modify those using similar approaches.
- **Stateless**: It implies making client-server interaction stateless; thus, every request to the server will be treated as new. Every client request must contain all the information necessary to process the request.
- **Cacheable**: Caching is necessary to improve site load time or access resources. So, every resource must declare if it is cacheable. Caching can be implemented on both—the client-side and server-side. Caching is immensely helpful for scaling systems.
- **Layered system**: REST architecture allows a system designer to design layers in the system. We can have one layer doing authentication; one layer can compose of proxy servers, one layer

having load balancers, one layer of data store, and one layer with application instances. An end-user may not know if it is directly connected to the server or some other layer.

- **Code on demand**: RESTful Web services also allow a server to return not only static resources like JSON or XML but also executable code as well. For example, a client may call the server to return a JavaScript code responsible for plotting a graph.

Conclusion

In this chapter, the OSI model is explained, which gives a vivid idea to a system designer on how two devices communicate over a network. Apart from it, important networking protocols such as HTTP, UDP, and TCP are discussed as well. In the end, REST architecture is covered, which is quite popular to design Web services for a system using HTTP. The next chapter will deal with how to use tools like the class diagram, HLD, LLD to draw a layout of the system.

Points to remember

- OSI is a conceptual model that is used to explain how information transfer occurs from one device to another in a network.
- HTTP is an application layer protocol, whereas TCP is a transport layer protocol.
- UDP is a lightweight, unreliable protocol, unlike TCP, which is a reliable protocol.
- REST is an architectural style that has some set of principles defining how to design loosely coupled applications using HTTP.

Multiple choice questions

1. **Which one of the following layers is responsible for converting bitstream to electric signals before transmission?**

    a. Physical layer

    b. Transport layer

c. Session layer
d. Application layer

2. **TCP protocol works at which layer of the OSI model?**
    a. Application layer
    b. Physical layer
    c. Data link layer
    d. Transport layer
3. **Which one of the following is true for UDP Protocol?**
    a. It is a connection-oriented protocol
    b. It is a reliable protocol
    c. It ensures data arrives in order
    d. It supports broadcasting

Answers

1. **a**
2. **d**
3. **d**

Questions

1. What is an OSI model?
2. What are the differences between TCP and UDP?
3. Why is HTTP protocol used?
4. What are RESTful Web services?

Key terms

- HTML stands for Hypertext Markup Language and it is used to display documents in the browser.
- JSON is a lightweight format used to store and transport data.

CHAPTER 3

Scaling Databases

Introduction

Horizontal and vertical scaling are two techniques to ensure that the system works well with increasing load. Choosing one over the other requires evaluating tradeoffs, cost, and system complexity and then weighing out all the factors to find out which one is more suitable. The chapter focuses on what is meant by horizontal and vertical scaling and what are different ways to achieve it. It also discusses various approaches for scaling databases.

Structure

In this chapter, we will cover the following topics:

- Horizontal scaling
- Vertical scaling
- Database scaling methods

Objectives

After studying this unit, you will be able to understand the different ways of scaling applications. Typically, one can categorize the scaling techniques in two buckets—horizontal and vertical scaling. The chapter discusses in detail the meaning of these two terms and how one can put them into action in their existing application. Besides this, various database scaling techniques are described, which are crucial to know for any system designer with an intent to scale the database. Generally, not all methods are applicable for every system or database. One has to look for various tradeoffs like cost, network latency, maintainability, existing system architecture, and so on before putting them into use.

Horizontal scaling

Scaling horizontally implies adding more machines to a system. For example, adding more server racks to the existing system to handle the increasing load and to process requests efficiently. Needless to say, it does involve a lot of costs to horizontally scale a system. In horizontal scaling, we add new nodes to the cluster, thus, reducing the load on each node. So, the capacity of each node is not altered; however, responsibility is decreased due to the distribution of load across server nodes.

One can visualize from the following figure that in order to horizontally scale, we can add more servers to the existing application.

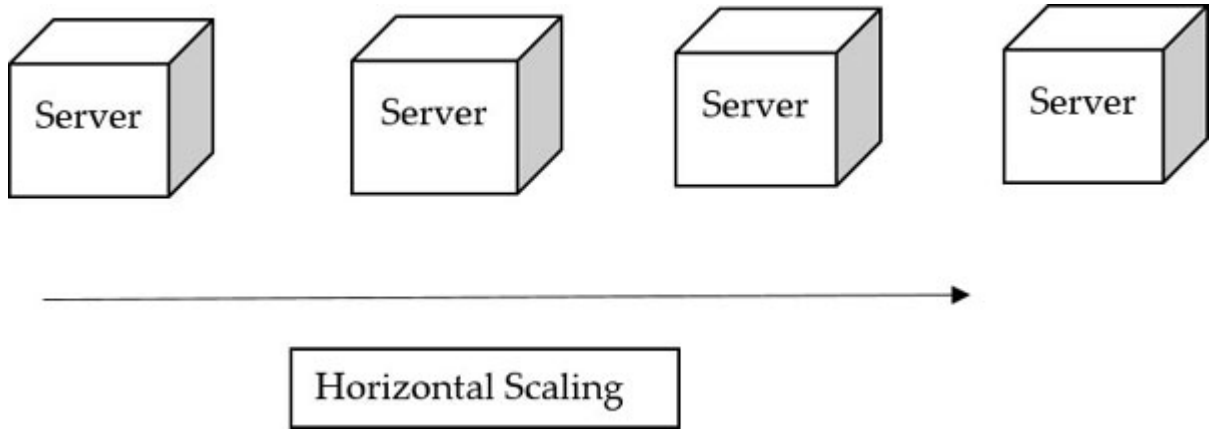


Figure 3.1: *Horizontal scaling*

We can apply horizontal scaling techniques to databases as well. Many popular databases use horizontal scaling. The need for horizontally scaling a database arises from the fact that data tend to grow faster for any system. With limited resources, memory, CPU, and storage capacity, response time for a query is bound to suffer from the increasing size of the database. So, in order to circumvent this problem, we can horizontally partition a database. Horizontally partitioning a database is also known as sharding. It is a technique where rows of a database table are held separately. Each partition denotes a part of the shard, and it may be located on a separate database server or physical location. The main advantage of this technique is the size of the database table is reduced significantly, and hence, index size is reduced. A relatively small indexing data structure decreases the turnaround time of the query and makes the system handle query in an efficient manner. However, sharding has certain disadvantages as well.

The main disadvantage of sharding is that there has to be an interconnection among different servers where the shards are placed. This interconnection is vulnerable to network failures. Also, sharding is done case-case basis and is generally one-way; that is, we may shard a database on the basis of pin code. So, queries like find people living in pin code xyz may run faster; however, queries, like find people with characteristics xyz, may run significantly slower. In short, some queries may run fast, and some queries may become slow. Additionally, there will be issues of consistency, durability, system failure because of multiple servers where database tables are located.

Vertical scaling

Vertical scaling implies adding more power to a single machine that is more CPU, RAM, or Disk than an existing machine.

The following figure illustrates that in order to vertically scale, one has to make a single machine more powerful.

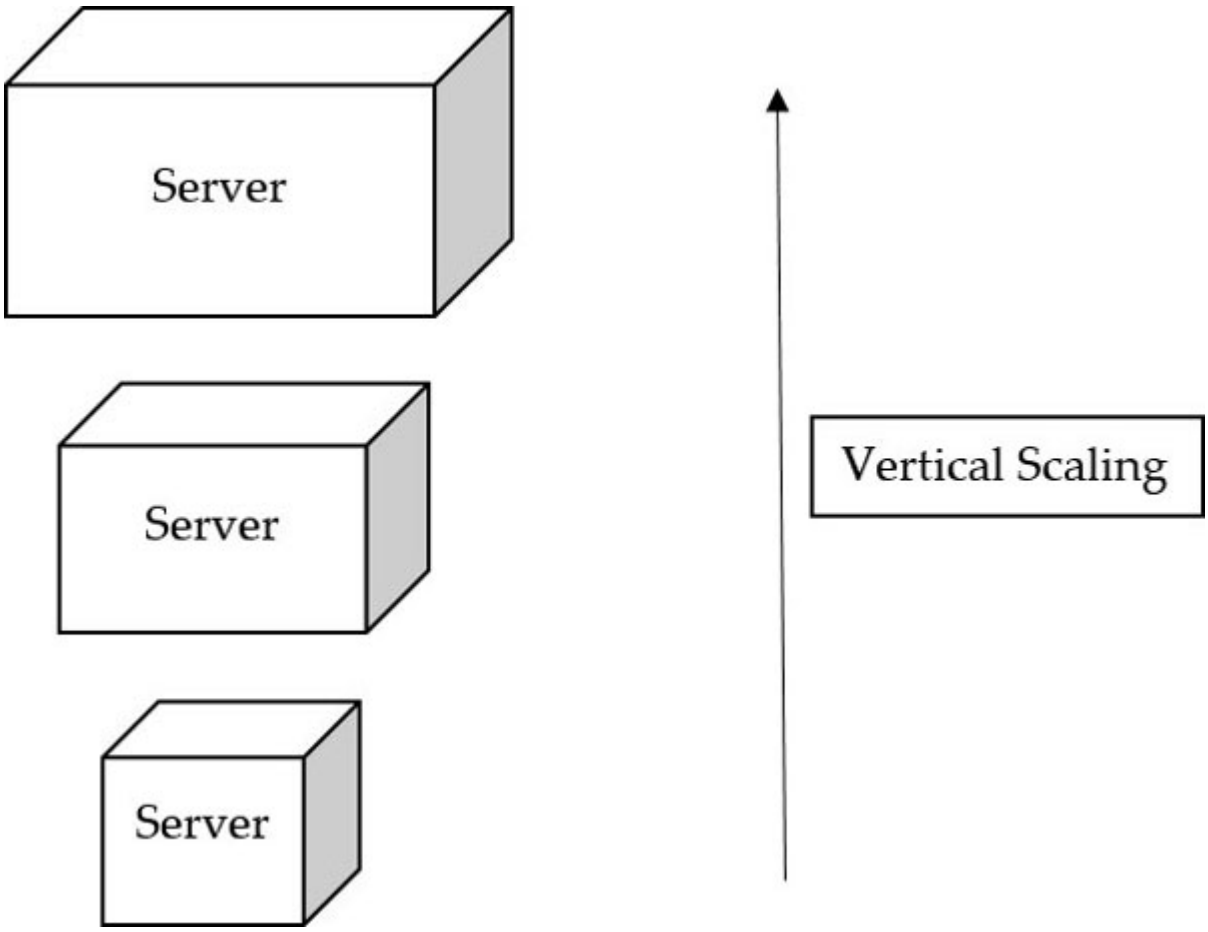


Figure 3.2: *Vertical scaling*

Vertical scaling essentially tries to increase the power of a single machine, unlike horizontal scaling, where we add more machines. Since hardware changes are required in this approach so, downtime might be required in this technique.

Any application designer should always first try to upgrade machines because buying a new server altogether would be expensive. However, it must be noted that upgrading any machine beyond a certain point makes no sense. After a certain threshold, buying a new machine is the only viable option. The system designer should always evaluate the tradeoff for both approaches and then take a call.

Database scaling methods

There are different scaling methods for the databases. Not all methods may be applicable to every database. So, readers are encouraged to understand

what level of flexibility their database supports and then use one of these techniques.

Read replicas

This technique follows a master-slave architecture. Here, database tables are distributed across different sets of nodes, of which some are known as slaves and one as master. Master node handles write queries while read queries are routed to slave nodes. The model works best if data changes are limited but read queries are large. To handle an unexpected scenario like a master failure, one of the slave nodes can be elevated to the master node. The pattern works well to scale out read queries. Let us understand this with a real-world example.

Consider a shopping website offering people a wide array of products like groceries, clothes, electronics, and so on. A majority of people visit the website to purchase products, read product specifications/reviews, casually browse and even check new gadgets or clothes. As one can see, the system is read-heavy. A system designer can deploy the read replica model for this system to make sure that read queries like find xyz properties of product abc or find the price of product abc are routed to the slave nodes which are specifically put to handle these requests, whereas, write queries like updating the price of the product are taken care by the master nodes. Since, with time the traffic to the application can grow, a system designer or database admin can add more slave nodes to the application to handle increasing traffic.

The following diagram illustrates a simple read-replica system. It has one master and two slaves. Master inputs all queries; however, the process only writes queries and route the read queries to the slave nodes.

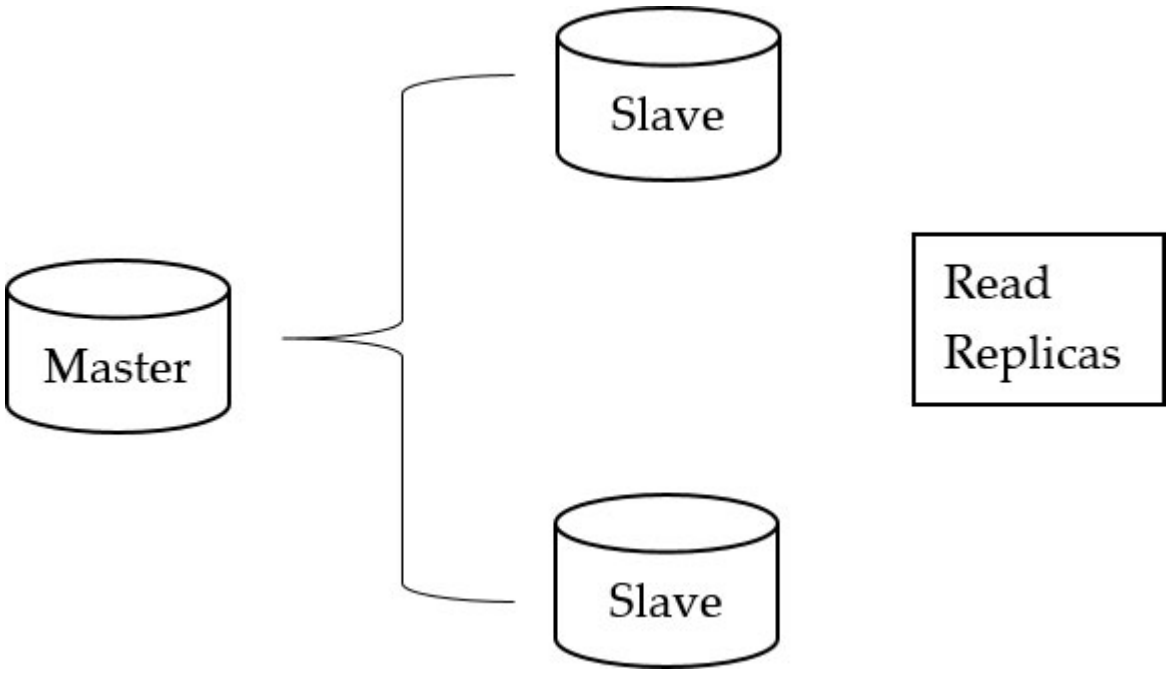


Figure 3.3: *Read replicas*

A careful reader may see that even such systems can be overwhelmed with a large number of read queries. In order to deal with this, one has to deploy more slave nodes. Also, to make sure that the system works efficiently with more slave nodes, the system designer or database administrator can put a load balancer to efficiently route read queries to a less burdened slave node.

Multi-master replica

If the need is to scale out write as well as read queries, multi-master replication can be used. In this pattern, we have multiple masters, any one of which can be used to read/write data. The multi-master replication system should ensure that any conflicts arising out of concurrent writes must be handled properly. Also, any changes made to the data must be propagated to all the other master nodes so that read queries return consistent results.

An example of a system with a good number of read as well as write requests could be a financial application. For instance, an application that allows a user to make a payment at any website given that he has valid login credentials. Now, every time a user initiates a transaction, the application must make sure that the user's account has sufficient funds. Once it is confirmed that funds are available, the transaction is initiated, the

amount is debited, and the user is informed. We can clearly see that the application involves both read as well as write operations in one single transaction. So, if the load is quite large, then the system has to deal with a large number of read and write queries. We can assume our application database to be huge for the sake of discussion. So, if the application has to work on such a huge database, then a multi-master replica can be put in use to scale out read as well as write queries. The database administrator can make use of multiple master nodes to handle read or write queries like update xyz user's account balance with amount abc or get account balance of xyz user.

The following diagram is an overly simplified version of a multi-master replica:

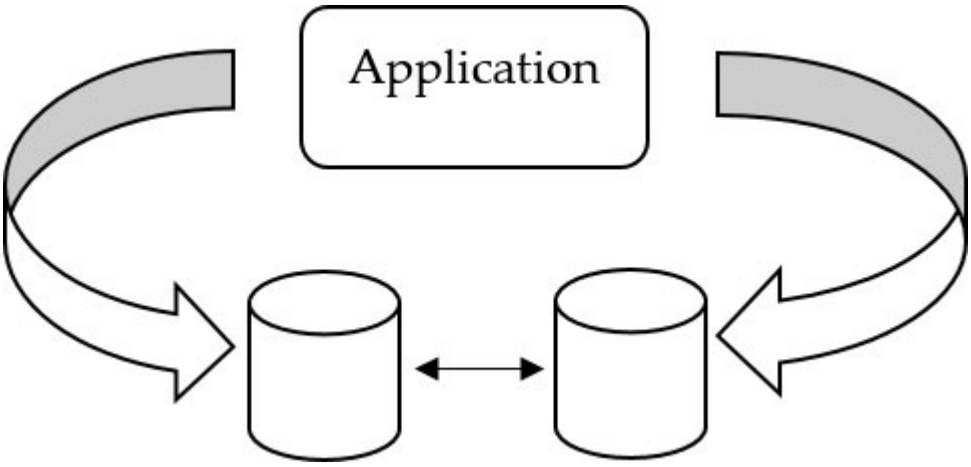


Figure 3.4: *Multi-master replica*

Connection pooling

An application first needs to establish a connection in order to communicate with a database. Then it sends a query to the database server, which computes the result and returns it back. Connection establishment is an expensive task because it involves a network overhead. So, if we want to scale our application, then establishing a connection for every database query will be a disaster. To deal with this, the application server maintains a pool of connections with the database, and these are used for each incoming query.

To understand better, consider a shopping website as discussed earlier. At any given point in time, there will be a large number of people either

browsing websites for reading product specifications or reviews or, maybe checking prices or comparing two different products. All these operations involve a lot of database queries behind the scenes. One must understand that initiating a conversation with the database involves connection establishment. This step is an expensive one, and if we keep on doing this for every database query, then a lot of time will be wasted. Also, we cannot ignore the fact that when we combine this time with database query computation time, the end-user may have to wait a lot, which, needless to say, could be detrimental for the business. In order to circumvent this problem, connection pooling can be used. We can maintain a pool of database connections, and every time the application needs to talk to the database, we can pick one from the pool. This will be immensely useful to scale database access and can help to reduce the turnaround time as well.

Connection load balancing

There is always a limit by a database vendor to the maximum number of client connections. So, in order to create a scalable system, the designer has to ensure that the database is also able to take concurrent connections. One way to achieve this is connection load balancing. This technique uses a database load balancer which routes the incoming database query to one of the available servers. Additionally, the database load balancer takes care of the task to perform re-routing if any of the servers fails.

For instance, consider the ticket booking website. At any point in time, several users might be trying to book tickets. Ticket booking involves a lot of database read and write. So, for every transaction user must connect to the database server. Now, in order to ensure that no single database server is overloaded, connection load balancing can be used. Using this technique, all the incoming database connections can be efficiently routed to a less burdened database server.

Conclusion

Horizontal and vertical scaling are two ways of scaling a system. In horizontal scaling, we add more servers to our application, whereas in vertical scaling, we increase the power of a single server by adding more RAM, memory, disk, and so on. There are tradeoffs and benefits with each

approach. Horizontal scaling involves adding a new machine, and thus, it is expensive, but it generally does not require downtime. Vertical scaling may not be more expensive than adding a new machine; however, upgrading an existing machine needs downtime. Apart from this, databases are also a crucial component of any application. There are numerous ways to scale databases as well, such as—read replicas, multi-master replica, connection load balancing, connection pooling, and sharding.

Points to remember

- Vertical scaling involves adding more machines to the application.
- Horizontal scaling involves upgrading a single machine by adding more RAM, memory, disk, and so on.
- Sharding is a technique where rows of a database table are held separately. It reduces the size of the index table; thus, the processing time of queries may improve significantly.
- Read replicas, multi-master replica, connection load balancing, connection pooling, and sharding are some of the ways to scale databases.

Multiple choice questions

1. **Which of the following is true about database sharding?**
    a. It is a technique where rows of a database table are held separately.
    b. It is a technique to add more database servers to the application.
    c. It is a technique to limit database connections.
    d. It is a technique to create a pool for database connections.
2. **Which of the following is true about horizontal scaling?**
    a. It is a technique to limit the number of concurrent connections to an application server.
    b. In this technique power of a single machine is increased by adding more RAM, CPU, memory, disk, and so on.

c. In this technique, we add more machines or servers for an existing application.
d. It is an approach to scale the system by limiting the number of HTTP connections.

Answers

1. **a**
2. **c**

Questions

1. What is vertical and horizontal scaling?
2. What is database sharding?
3. What are the different ways to scale a database?

Key terms

- CPU stands for Central Processing Unit. It is the brain of the computer.
- RAM stands for Random Access Memory. It is the primary memory of a system.
- The index data structure is used for the fast retrieval of a record from a large set of data.

CHAPTER 4

System Architecture

Introduction

At the inception of every system design, the system designer has to describe the use-cases and input/output for the system. Such an idea of the system helps a designer to visualize the system as a black box, which takes the input and gives the desired output. However, once the designer is past this stage, he may start building a system either as a big single unit or by breaking the system into several smaller pieces, where each piece is assigned a specific task and all pieces communicate with each other to produce the desired output. Now, depending upon which style a system designer adopts for the system designing, we may have either monolithic architecture or service-oriented architecture.

Generally, the call to choose either one lies with the software architect by keeping in mind the business need and maintainability of the product in the future. Each methodology has a tradeoff and advantages. The chapter discusses both approaches in detail and also focuses on how these different architectural styles can be put in use during system designing.

Structure

- Monolithic system
- Service-oriented architecture
- Microservices

Objectives

The objective of this chapter is to explain what is a Monolithic system and how they are implemented. It also explains service-oriented and microservices architecture. Additionally, it discusses how these two architectures, service-oriented and microservices, are different from each

other and what kind of practices and methodologies a software designer can follow to implement a system as per the preceding mentioned styles.

Monolithic systems

To start with simple words, monolithic systems lack modularity. Modularity implies the breakdown of an application in smaller components or modules, each assigned with a specific task, and all such modules work together to achieve the desired goal. So, monolithic systems are the complete opposite. They are designed without modularity. They are like one big box which takes the input and produces the required output.

Monolithic systems are also a single-tiered application. So, the user interface and data access layer are packed in one single application. Readers must try to relate from this discussion that such systems are not just capable of processing a single request; rather, they can perform every step to complete a given operation.

Since a single monolithic application can perform several tasks, typically, a monolithic application has a huge codebase. As more and more functionalities are added, developers tend to create more tightly coupled components. These components, when changed in the future, often require the recompilation of the entire application. Also, the onus of testing the entire application falls on the developers with every new component addition. This is true not only for a single component addition but even with a minor change that can have an impact on several other functionalities.

Tip: Choosing between monolithic style and service-based architecture is not vivid during the initial stages of product development. One should not focus solely on the architectural advantages of one over the other rather, and organization needs should be given priority. Generally, monolithic architecture suits better for lightweight applications. However, one should often dare to change the architecture to service-based if the application is evolving and becoming increasingly complex over the period of time.

Let us discuss an example to understand the monolithic systems better. Consider an application to disburse cash to the user that is used behind the

scenes of an ATM. A monolithic application like this consists of tightly coupled components.

A typical use case of dispensing cash involves steps listed in the following flowchart:

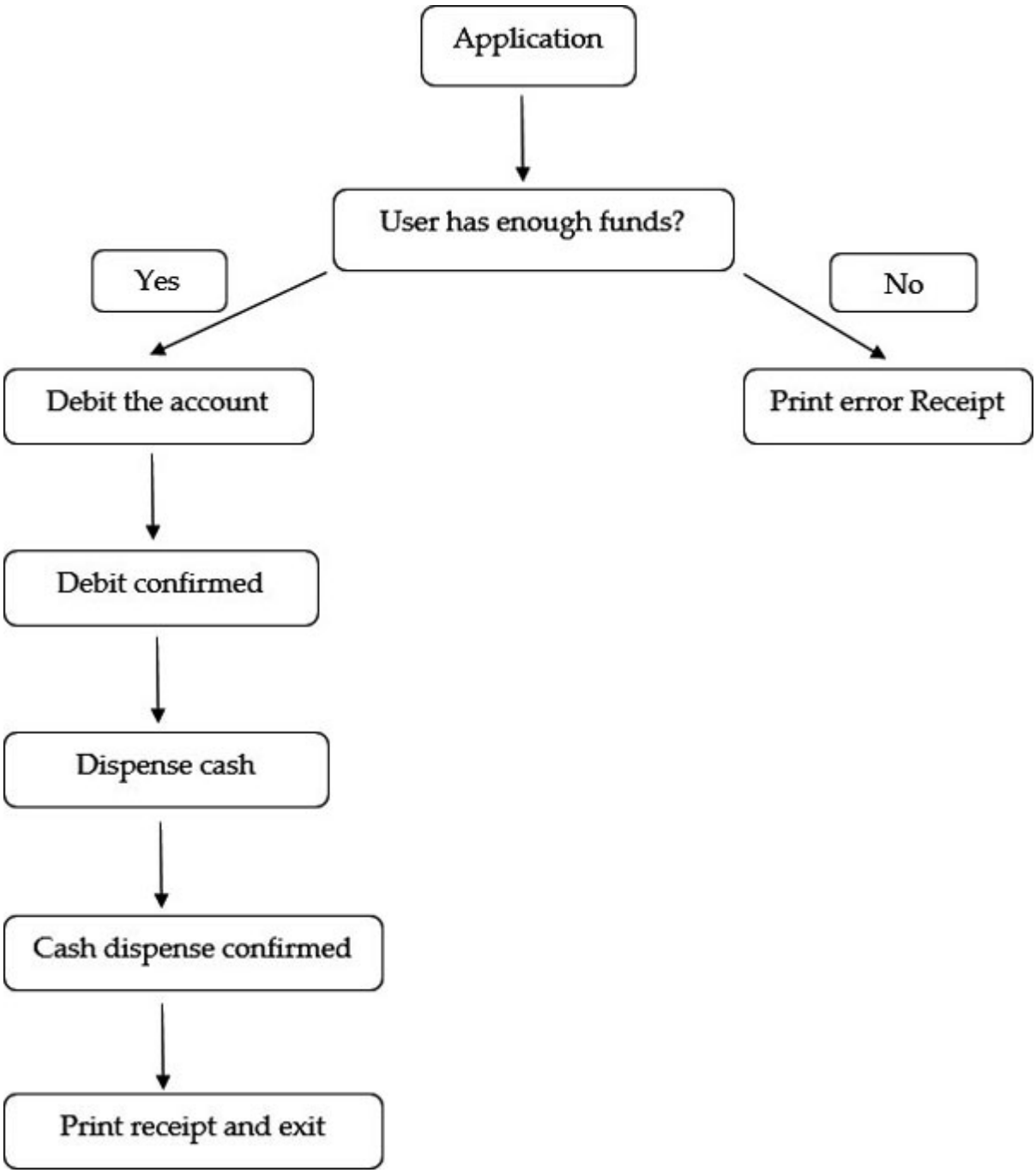


Figure 4.1: *Flowchart for a cash dispensing machine*

A system designed for this use case using a monolithic architectural style would consist of one big application responsible for performing all the operations shown preceding. The application would take care of all the functions such as checking for account balance, printing receipts, dispensing cash, and so on. Now consider if the bank wants to change their policy to withdraw the amount even if the balance is 0 and the withdrawal amount is less than 200 USD. This change involves modifying the function responsible for checking account balance. However, the impact of this modification will be to compile the entire application and redeploy it with new changes. This situation is often undesirable and generally not possible for many software products. This is where service-oriented architecture comes into the picture.

Note: A typical application can be divided into three broad components: user interface layer, data access layer, and data store. The user interface layer is the entry point of the application. The user directly communicates with this portion of the application. The data access layer wraps the data store. It is responsible for providing simplified access to the objects in the data store. The data store layer is mostly a database and is responsible for storing and retrieving data. In a monolithic application, all these layers are tightly coupled. There is usually a large codebase for either all or two out of three layers of the application.

Service-oriented architecture

Service-oriented architecture is comprised of several loosely coupled services which are highly interoperable. It is a software architecture style where application components provide services to other units. As the name implies, **Service Oriented Architecture** (**SOA**) consists of a number of services. A service in SOA is complete on its own and does not need anything else; that is, it is self-contained. A service may also contain other services if needed. A consumer of the service does not have to be aware of the inner details of the service. So, for a consumer, service is like a black box that is designed to perform a specific task.

Typically, in SOA, several services work together to provide the functionality of a large business product. Unlike Monolithic systems, SOA

follows modularity. Each service works like an independent unit and can be deployed and maintained separately. Services communicate with each other through a communication protocol over a network.

The foundation of SOA is loose-coupling between its services. The crux is to identify the core functions or tasks and then create a service to perform them. The developers then have to make sure that these services are accessible over the network so that other developers can leverage them to develop their own big applications. Every service in SOA follows a well-defined and agreed format for its output. There are six core values of SOA, which are listed as follow:

- Business value
- Strategic goals
- Intrinsic inter-operability
- Shared services
- Flexibility
- Evolutionary refinement

SOA is quite popular and possesses several useful characteristics. These desirable qualities are as follows:

- **Standard**: In SOA, a business process is partitioned into several services, and all these services are built and consumed using standards such as **Web Service Description Language** (**WSDL**), **Simple Object Access Protocol** (**SOAP**), and so on.
- **Platform independent**: Services built using SOA and the services that are consuming output from these services can be deployed independently on any platform.
- **Reusable**: Services can be re-used by other services.
- **Business function**: Each service has delegated a task from the business point of view and works in conjunction with other services of the system to form a complete business process.

Note: WSDL stands for Web Service Description Language. It is an interface description language that is used to describe the application programming interface of a product. It is based on the XML format.

WSDL is often used in conjunction with SOAP to provide Web services over the network.

In order to contrast with the monolithic system, consider an e-commerce website that offers the following functionalities:

- Check product availability for a customer.
- Allow a customer to order a product.
- Allow a customer to make payment.
- Allow a customer to check order status.
- Create a product invoice.
- Calculate shipping charges for the product.

When we are building this application in a monolithic style, we may end up creating a single application performing all these operations. However, with SOA, we will create several reusable services such as create an invoice, check product status, compute shipping charges, order product, and so on.

The following figure illustrates the difference between the monolithic and SOA of this system:

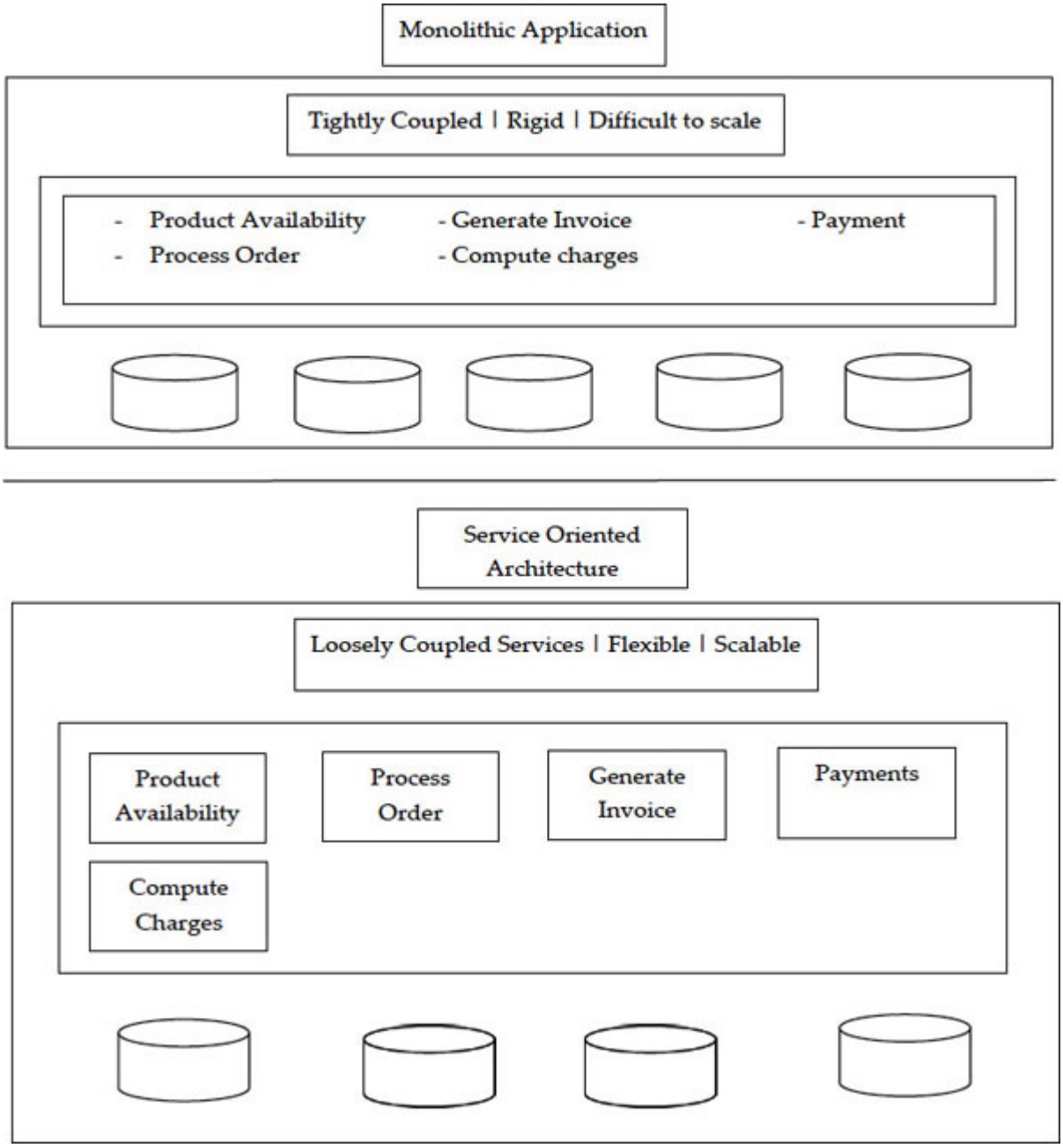


Figure 4.2: *Monolithic versus SOA*

As one can see, for a monolithic architecture, one big application is created to perform all the operations such as product availability search, process orders, generating invoices, and so on. This is in sharp contrast with the SOA, where the application is divided into different services, and each service is performing a single operation.

SOA implementation approaches

SOA can be implemented with microservices or Web services. When we use Web services-based implementation, then several components are made accessible over the standard network protocols. There are various Web services standards. One such popular standard is SOAP. SOAP typically works in the following way:

- The application sends a SOAP request to a Web server that is hosting Web service like the address database of customers. Application requests will contain the parameters to search.
- The server computes the result and returns a SOAP response—an XML formatted response containing the result.
- The client application receives the data in a standard format that can be parsed by the machine.

SOAP is widely used for the data exchange among applications because of its characteristics as follows:

- **Lightweight**: Since SOAP is based on XML, which is a lightweight data-interchange format, SOAP is considered a lightweight protocol.
- **Works with various protocols**: SOAP works with various protocols such as HTTP, SMTP, TCP, and UDP.
- **Platform interdependent**: SOAP protocol can work with any programming language-based application.

Due to these characteristics, SOAP is quite popular and hence, widely used.

Note: There are several design models for Web services. SOAP and REST are the most popular ones. Unlike SOAP, which uses XML, REST allows plain text, JSON, XML, YAML, and several other formats. REST makes use of uniform service locators to provide access to an object. To understand a RESTful Web service, consider a Web service to provide access to student-related data. Now, a RESTful Web service may host a URL like http://myschool.com. So, different URI can exist to provide access to student's data:

- **http://myschool.com/student/: Data of all the students in JSON format**

- **http://myschool.com/student/{student_name}: Data of the particular student in JSON format**

Microservices

Microservices-based architecture follows the principle of separating the functions in different services. In this architecture, a business application is developed as a collection of various independent loosely coupled services. Each service performs a specific function. Two different services communicate with each other following a pre-defined protocol. Since different services inside a particular application are independent of each other, they can be deployed separately and they can be written in different programming languages. Each service maintains its own data store. Typically, service is small in size.

Microservices based architecture offers various advantages as follows:

- Since functionalities are broken down into small pieces and assigned to a single service, so application becomes modular, easy-to-understand, and easy to develop.
- Testing becomes easy.
- Since services are developed and deployed independently, they can be scaled independently as well.
- Development of applications can be expedited by delegating the task to develop services to different teams. Each team can develop, test, and deploy their service independently.

However, there are certain disadvantages as well with microservice-based architecture:

- Different services communicate using the network calls, which is expensive as compared to the in-process calls of a monolithic application.
- If different services of a particular application are developed by different teams using different technologies, then maintenance becomes difficult in the long run as teams may change and engineers may move across different teams.

- Assigning responsibilities and managing communication may cause concerns if services are owned by different teams.

To understand better, let us consider an example of a shopping website designed using microservices-based architecture. We can have different services to perform different functions such as search, product reviews, specifications, payments, and so on. So, an overly simplified diagram for such an application is shown as follow:

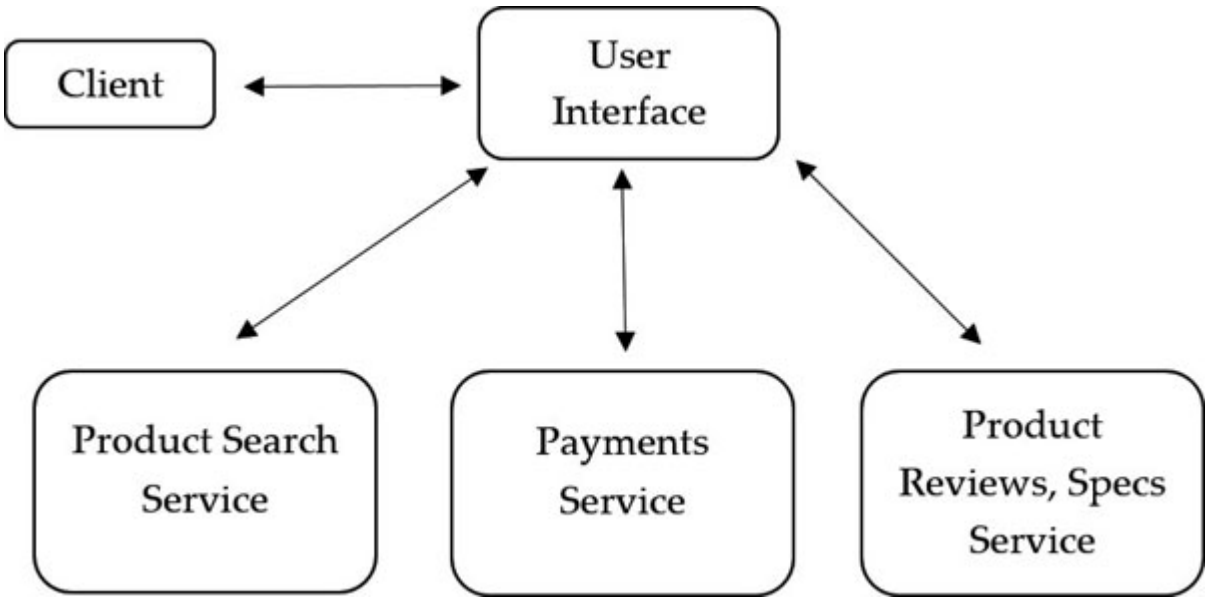


Figure 4.3: *An overly simplified illustration of a shopping website using a microservice-based architecture*

A careful reader can see that different functionalities are broken down into services. We have a product search service that searches a particular product and retrieves its information from the database, a payment service that handles all the complexities of completing a transaction via credit/debit card and product reviews, specs service to fetch the product reviews and specifications from the backend data store. Also, each service can be deployed tested independently.

Microservices versus SOA

Typically, many characteristics of microservices and SOA are similar. In both of them, a large application is broken down into several smaller pieces that are independent to develop, test and deploy. However, there are some differences between the two of them:

- SOA promotes sharing of components, whereas microservices tend to minimize sharing by designing loosely coupled services having their own data store.
- Microservices architecture emphasizes developing fine-grained, small-size services. The prefix "micro" refers to the granularity of services in microservices architecture, thereby denoting that service size in microservice-based architecture has to be smaller than SOA.
- SOA uses an **Enterprise Service Bus** (**ESB**) for communication whereas, the microservices-based architecture uses a simple and less elaborate messaging system.
- If the aim is to combine different components operating on different protocols, then SOA should be used. However, if all services of the application are accessible via the same remote access protocol, then microservice architecture suits best.
- There are uniform data governance mechanisms and standards across all services in SOA since it is based on the notion of resource sharing. As microservices-based architecture promotes independent service so, there is no uniform data governance mechanism.
- In a microservice-based architecture, services are loosely coupled and small in size. So, the deployment of an application is easier if it is built on the principles of microservices than SOA. In SOA, adding a new service sometimes result in redevelopment and redeployment of the whole application due to dependency between services.
- SOA mainly uses protocols like SOAP for remote access and **Microsoft Messaging Queue** (**MSMQ**), and **Advance Messaging Queue Protocol** (**AMQP**) for messaging. Microservices generally use **Representational State Transfer** (**REST**) and **Java Messaging Service** (**JMS**) for messaging.

Microservices versus monolithic systems

There are various parameters in deciding when one should stick to a particular architecture. This section discusses various points one can keep in mind before adopting a particular architectural style. Let us say a company has just started, and it wants to demonstrate a prototype of the product for its funding. Since the product and idea are new, its capabilities strengths are

all unproven. The key here is to create the product rapidly and demonstrate the proof of concept. Also, the implementation should be quick for raising funding and attracting customers. In such scenarios, it is ideal to start off as a monolithic system and then gradually converge the system to the service-based style.

Service-based systems require several infrastructures such as cloud services (Google Cloud, Amazon AWS), CI/CD pipeline, and so on. It is not feasible for every organization to be blessed with such tools before starting the implementation of their product. In such cases, sticking to monolithic architecture makes more sense.

There might be scenarios where the organization is already equipped with good teams, each one well-versed with different languages. Now, if expeditious development of the product is needed, then the product can be partitioned into different services, and each team can be assigned one to develop based on their expertise. Integration can be challenging, so the requirement of a DevOps engineer might be needed.

In a nutshell, organizations should always evaluate their needs, capabilities, infrastructure before adopting a particular architectural style. Each style carries a tradeoff, so it is always best to keep in mind the business needs before jumping into the implementation.

CI/CD for microservices

When we say CI/CD, we are focusing on multiple incremental processes—Continuous Integration, Continuous Delivery, and Continuous Deployment. It is imperative to understand the meaning of these terminologies for deploying microservices.

- **Continuous Integration** (**CI**): There are frequent merges of changed code into the main branch. Various automated build and test processes are developed that would ensure the new code pushed to the main branch is production-ready.
- **Continuous Delivery** (**CD**): All code changes that are through the CI process are published to a PRE-PRODUCTION environment (PRE-PRODUCTION environment is a clone of the latest live environment). In most scenarios, deployments to the live environment need manual

approvals. The objective of this process is to ensure that the code is ready to be deployed into the live production environment.

- **Continuous Deployment**: The code updates that have successfully been integrated and deployed to the PRE-PRODUCTION environment are deployed to the production in this final step.

The CI/CD process for microservices should facilitate the following:

- Each microservice can be built and deployed independently without effecting any other services.
- Configuring and enforcing quality checks at each stage of build and deployment for validations.
- The complete process should be highly reliable and quickly restorable to the previous state in case of failures.

There are different strategies for deploying code updates to the application that is in LIVE-PRODUCTION environment:

- Rolling updates

  In the rolling update method, we deploy the new instance/s of the existing microservice, and new instance/s start receiving all requests right after the successful deployment. All the old instance/s are removed.

  This strategy can be used where we need to deploy versions of service with new features frequently without changing the API endpoints.

  One of the major challenges is that even if one service fails to update to the new changes, the entire application is rolled back to the previous version. Also, during the update process, there would be a mix of old and new versions of services receiving requests, and the end user may experience inconsistencies.

- Blue–green deployment

  In blue–green deployment strategy, we deploy the new version of the service/s side-by-side with the previous version. There exists room for validations before moving the traffic to the new version. After the new version is live, we can observe the application for any errors or abnormal behavior. If anything goes wrong, we can easily swap back to the previous version providing minimal downtime to the end-user.

In a microservice architecture, since updates happen at the microservice level, we will have to deploy the new update on the same environment and use a service discovery mechanism to swap between versions of the application.

One major drawback of this approach is the need for more resources for both versions of the application to be on the same server. We may need to scale out the cluster to handle the resource consumptions as and when required.

- Canary release

  In the canary release strategy, the aim is to roll out the new changes in a controlled manner. To achieve this, we provide the new release to a limited set of clients and monitor the behavior of the application. When everything looks good, we move all the users to the new version.

  The advantage of this method is we can observe the behavior of the application, collect real data, and spot issues before it is released to all the clients.

  A canary release is much more complex to manage than blue-green deployment as we need to dynamically route a set of requests to the new version of the application.

Azure Kubernetes Service (**AKS**) can be used to simplify the deployment and management of microservice-based architecture. It helps streamline horizontal scaling, load balancing, and configuration management, all under one umbrella.

Conclusion

There are different ways of architecting a system—monolithic, SOA, or microservices-based application. In monolithic systems, every component of the application is packed in one single unit. Every piece is tightly coupled, so changing anything tantamount to redeploying the whole application again. In SOA, we have different services inside an application, with each service performing a task. Microservices-based architecture is similar, except for the fact that services in microservices-based systems are more fine-grained and micro than SOA. The chapter describes the features,

advantages, and disadvantages of each architecture type so that a reader can learn which architecture type suits best his requirements.

Points to remember

- A monolithic application contains every component tightly coupled in one single unit.
- A service in Microservice tends to be micro than service in Service Oriented Architecture.
- Generally, using services in the application makes it modular easy to understand, develop, test, and deploy.

Multiple choice questions

1. **Which of the following is true?**
    a. Monolithic applications are modular and have services to perform a specific task.
    b. Services in microservice-based architecture are larger in size than SOA.
    c. Microservices-based architecture requires the developer to bifurcate the application into tightly coupled services.
    d. There is generally uniform data governance in SOA.
2. **Which of the following is/are false for SOAP?**
    a. SOAP is based on XML.
    b. SOAP is not suitable to be used with HTTP and other transport protocols.
    c. SOAP is platform-dependent.
    d. SOAP, due to its neutrality characteristic, works well with any transport protocol.

Answers

1. **d**

2. **b, c**

Questions

1. What is a monolithic application?
2. What are the differences between a monolithic and a microservice-based application?
3. What is the difference between SOA and microservice-based architecture?
4. How is SOAP different from REST?

Key terms

- XML stands for Extensible Markup Language. It defines a set of rules to encode documents.
- ESB stands for Enterprise Service Bus. It is a middleware tool often used in distributed computing to distribute work among different components of an application.

CHAPTER 5

Introduction to OOPs

Introduction

In software engineering, there exist numerous ways to approach a problem, whether it is a mathematical or any real-world problem. Paradigm denotes a technique that is used to solve a problem or perform a task. Similarly, a programming paradigm means how to solve a problem using a particular programming language. Programming paradigms classify various programming languages on the basis of their features. Some commonly used paradigms are imperative and declarative. The imperative programming paradigm instructs the computer to change its state based on the commands (or statements) that a programmer writes. The imperative programming paradigm is further divided into procedural and object-oriented. In procedural programming, a program is built from one or more routines or functions (commonly known as procedures). C programming language can be considered to be based on a procedural paradigm. On the other hand, an object-oriented paradigm is based on the concepts of objects and classes. Programming languages like C++ and Java are based on this paradigm. It is indispensable for a system designer to have a thorough understanding of object-oriented programming principles because, as a system designer, he has to draw classes, visualize attributes and define functions of various actors involved in the system. The aim of this chapter is to explain the OOPs principles in detail and how one can use these principles during a system design. The chapter contains various code snippets in C++ language to provide users a reasonable platform to understand the OOPs concepts.

Structure

In this chapter, we will cover the following topics:

- Introduction to OOPs

- Features of OOPs

Objectives

After studying this unit, you will be able to understand what is object-oriented programming is and the various concepts in the OOPs.

Introduction to OOPs

Traditional procedural-oriented languages such as C and FORTRAN have some serious drawbacks. In procedural-oriented languages, programs are made up of procedures or functions. Functions are generally non-reusable. It is not easy to copy and paste functions from one point to another as functions may contain a reference to other variables, functions, or headers. Additionally, traditional languages focus more on defining functions and managing memory, whereas object-oriented languages enable high-level abstraction for solving real-world problems. So, object-oriented language allows a user to focus on the business problem rather than other irrelevant issues.

In an object-oriented language, users can define components in terms of classes and objects, as object-oriented programming is based on the concept of objects and classes. So, it is important to understand the meaning of these terms to apply OOPs principles to a real-world problem. Procedural-oriented languages like C have a concept of structure that is used to define a user-defined data type. Similarly, in programming languages such as C++ and JAVA, there is a concept of class that is used to define the user-defined data type. A class consists of different variables and functions that operate on those variables. A class is a blueprint or template that possesses some properties and behaviors that are common to all objects of that class. The term “object” refers to the instance of the class, and it simulates a real-world entity. For example, consider a class Student with properties such as name, roll number, grade, address, and so on. We may define several member functions for this class to get the name of the student, print marks, and so on.

An overly simplified code for such class in C++ could be defined in the following manner:

```
1. Class Student {
2. private:
3.   string name;
4.   int age;
5.   string roll_number;
6.   double grade;
7.
8. public:
9.   string getName() {
10.    return name;
11.   }
12.   Void printMarks() {
13.    cout<<grade<<endl;
14.   }
15. };
```

Now, we can define various objects of the class Student, and all such objects will have the same properties and behavior. For instance, consider two instances or objects of the class **`Student`**, say, **`s1`** and **`s2`**:

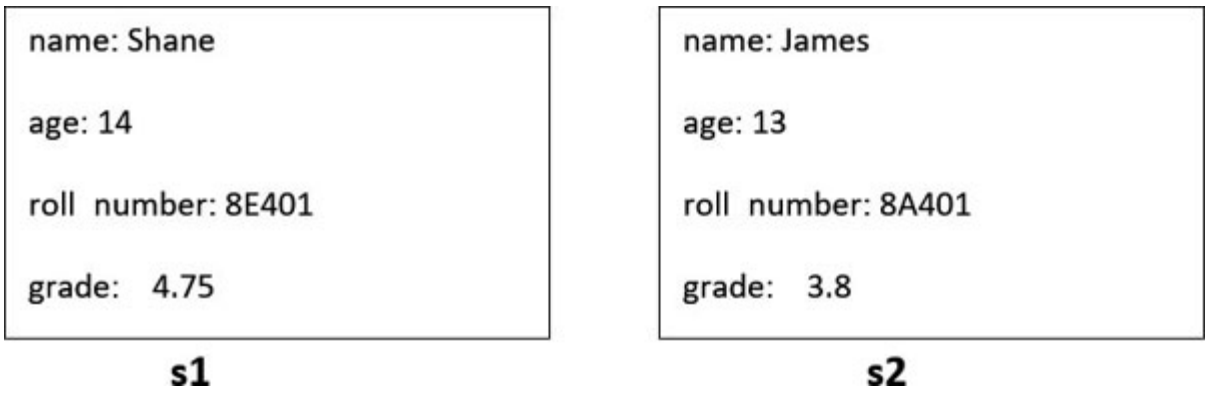


Figure 5.1: *Instances of class Student*

We can clearly see that both the objects possess the same properties (name, age, roll number, and grade), albeit values are different for those properties.

OOPs features

Object-oriented programming is a programming paradigm that uses classes and objects to design a program. It eases software design and maintenance by providing several features as follows:

- Class
- Object
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Class

A class is like a template or a blueprint using which we can create an object. It symbolizes a user-defined data type and consists of data (variables) and functions that operate on this data.

For instance, consider a class **`Person`**:

```
Class Person {
private:
   string name;
   string address;
   string phone_number;
   string profession;
   int age;
public:
   void set_name(string pname) {
    name = pname;
   }
   void set_address(string paddress) {
    address = paddress;
   }
   void set_phone_number(string phone_number) {
```

```
16.    phone_number = phone_number;
17.   }
18.   Void set_profession(string profession) {
19.    profession = profession;
20.   }
21.   void set_age(int age) {
22.    Age = age;
23.   }
24.   string get_name() {
25.    return name;
26.   }
27.   string get_address() {
28.    return address;
29.   }
30.   string get_phone_number() {
31.    return phone_number;
32.   }
33.   string get_profession() {
34.    return profession;
35.   }
36.   int get_age() {
37.    return age;
38.   }
39. };
```

We can observe from the preceding code that class **`Person`** has different data members such as **`name`**, **`address`**, **`age`**, **`profession`**, and **`phone number`** and various member functions such as **`get_age`**, **`set_age`**, and so on, which operate on those data members.

Object

Objects are the instances of the class. They generally correspond to a real-world entity. For example, we may have an object **Car**. Objects may also represent abstract things like the file in the computer system.

Consider the same class **Person**, which is defined previously. Let us consider two objects of this class **Person**:

```
Person p1("John", "New Orleans", "504-123-4567", "Software
Engineer", 38);
Person p2("Hitesh", "India", "91-1234567890", "Painter", 28);
```

So, we can see that the two objects of class **Person**—**p1** and **p2** represent a real-world living being. One must note that on the same lines, objects can be visualized to define several other real-world entities.

Inheritance

It is the ability of the class to acquire the properties and behaviors of its parent class. Inheritance is useful because it supports code reusability. The inherited class is known as a subclass or derived class, and the class from which it is inheriting is known as a parent or super or base class. The different types of inheritance are described next, along with a sample code in C++ language.

Single inheritance

In this type of inheritance, a subclass inherits the features of one parent class. A simple pictorial representation could be as follows:

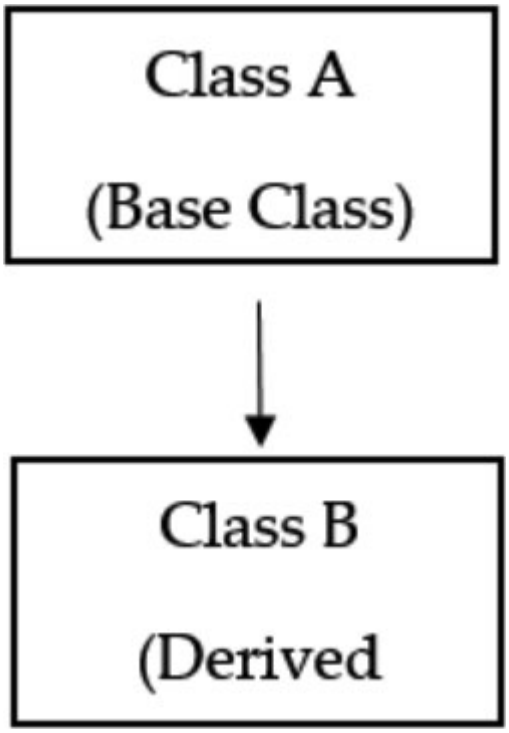


Figure 5.2: *Single inheritance*

It is evident from *figure 5.1* that in single inheritance, a subclass inherits from one base class.

A C++ code for the same:

```
/* Base Class */
Class A {
        Int a_id;
        Void a_fun() {
        ….
        }
};
/* Derived Class */
Class B : public A {
        Int b_id;
        Void b_fun() {
        ….
        }
```

```
14. };
```

Multi-level inheritance

In this type of inheritance, a derived class is inherited from another derived class. A simple pictorial representation is as follows:

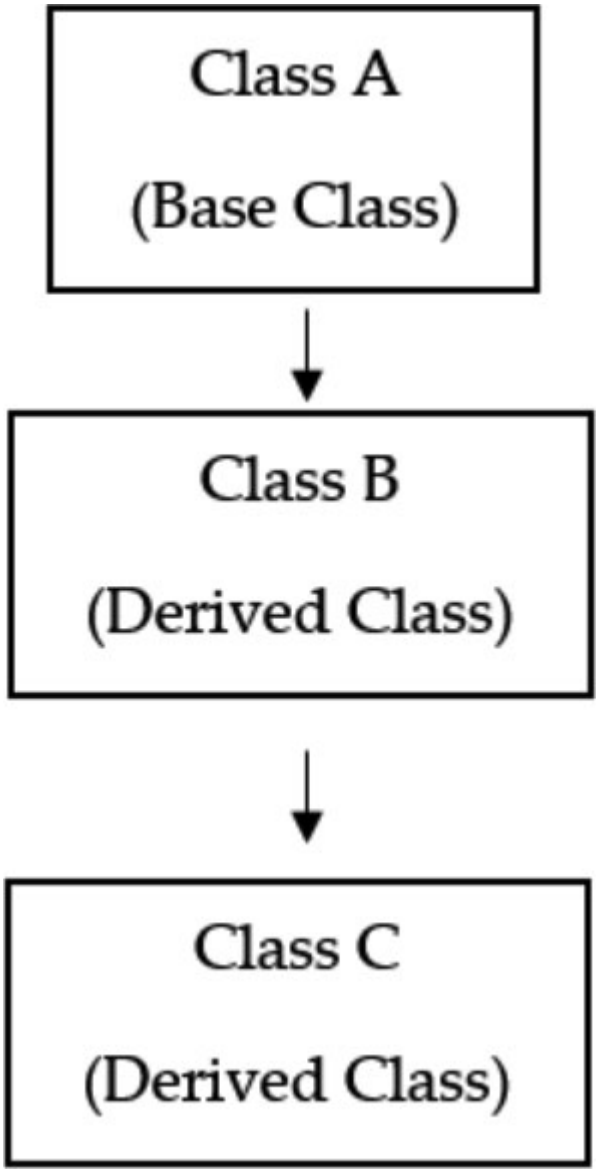


Figure 5.3: *Multi-level inheritance*

Readers can see that `Class B` is derived from the base **Class A** and then **Class C** is derived from **Class B**. So, Class B inherits features from **Class**

A, and **Class C** inherits features from **Class B**. So, the class will contain features of both classes—A and B.

```
/* Base Class */
Class A {
            Int a_id;
            Void a_fun() {
            ….
            }
};
/* Derived Class 1 */
Class B : public A {
            Int b_id;
            Void b_fun() {
            ….
            }
};
/* Derived Class 2 */
Class C : public B {
            Int c_id;
            Void c_fun() {
            ….
            }
};
```

Multiple inheritance

In this inheritance, a derived class is inherited from more than one base class. A simple pictorial representation is as follows:

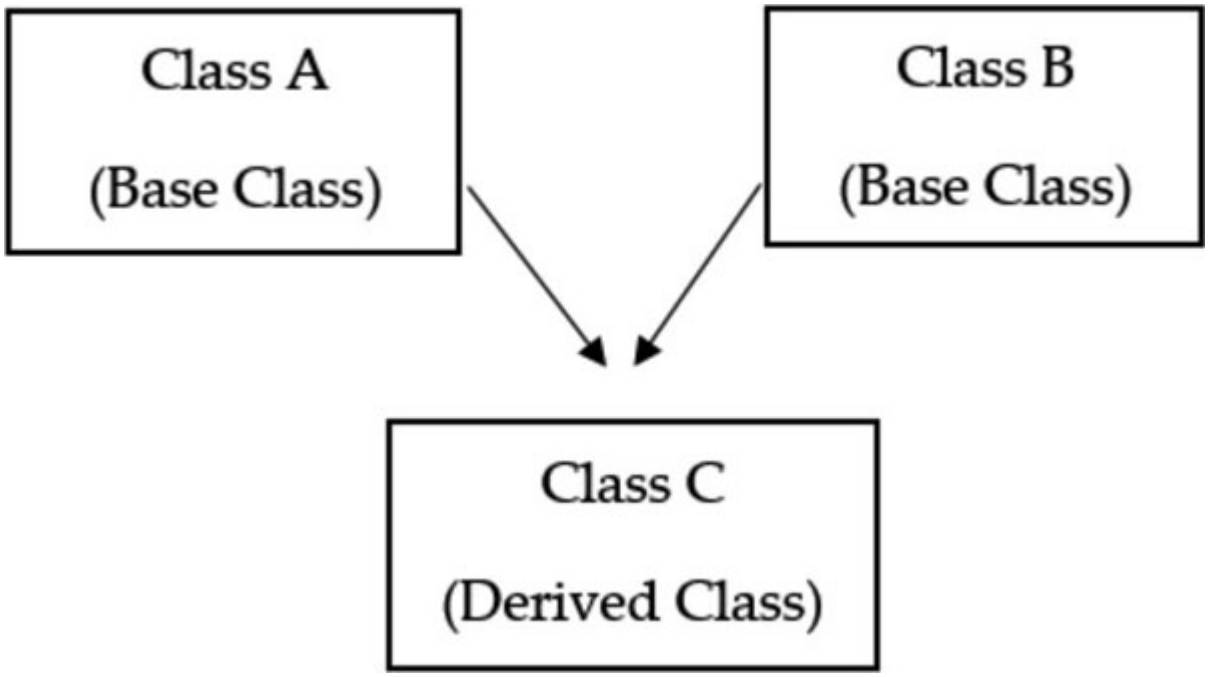


Figure 5.4: *Multiple inheritance*

In the preceding case, **Class C** is derived from **Class A** and **Class B**. So, **Class C** derives its features from **Class A** and **Class B**.

```
/* Base Class */
Class A {
          Int a_id;
          Void a_fun() {
          ….
          }
};
/* Derived Class 1 */
Class B {
          Int b_id;
          Void b_fun() {
          ….
          }
};
/* Derived Class 2 */
```

```
Class C: public A, public B {
            Int c_id;
            Void c_fun() {
            ….
            }
};
```

Hierarchical inheritance

In this inheritance, more than one subclass is inherited from one base class. Consider the following figure to understand it better.

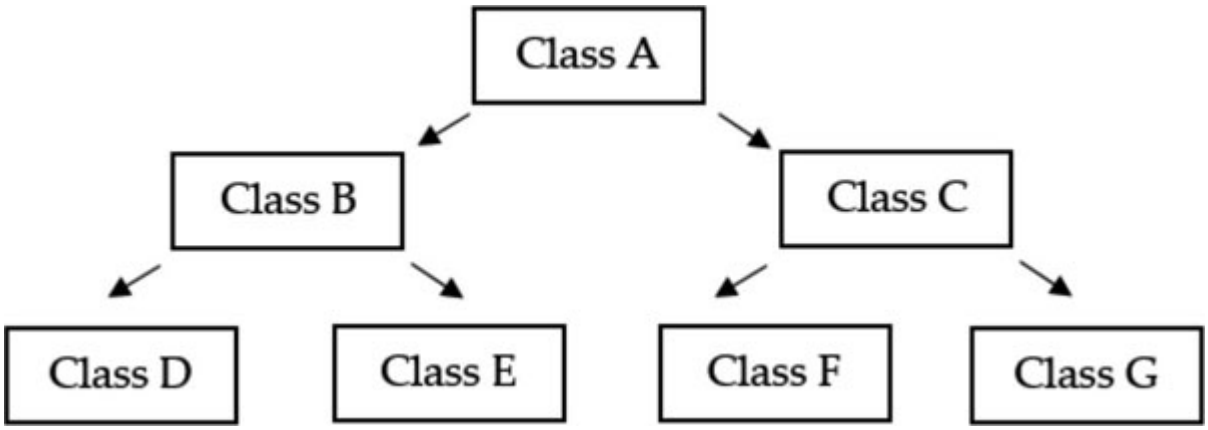


Figure 5.5: *Hierarchical inheritance*

Readers can clearly see that **Class A** acts as a base class for **Class B** and **Class C**. **Class B** then acts as a base class for **Class D** and **Class E**. Similarly, **Class C** acts as a base class for **Class F** and **Class G**. Consider, the following code snippet in C++ language for the preceding case.

```
Class A {
            Int a_id;
            Void a_fun() {
            ….
            }
};
Class B : public A {
```

```
8.          Int b_id;
9.          Void b_fun() {
10.         ….
11.         }
12. };
13. Class C: public A {
14.         Int c_id;
15.         Void c_fun() {
16.         ….
17.         }
18. };
19. Class D : public B {
20.         Int d_id;
21.         Void d_fun() {
22.         ….
23.         }
24. };
25. Class E : public B {
26.         Int e_id;
27.         Void e_fun() {
28.         ….
29.         }
30. };
31. Class F : public C {
32.         Int f_id;
33.         Void f_fun() {
34.         ….
35.         }
36. };
37. Class G : public C {
```

```
38.         Int g_id;
39.         Void g_fun() {
40.         ….
41.         }
42. };
```

Hybrid inheritance

It is a mixture of more than one type of inheritance. For instance, we can combine multiple and hierarchical inheritance. Consider the following figure.

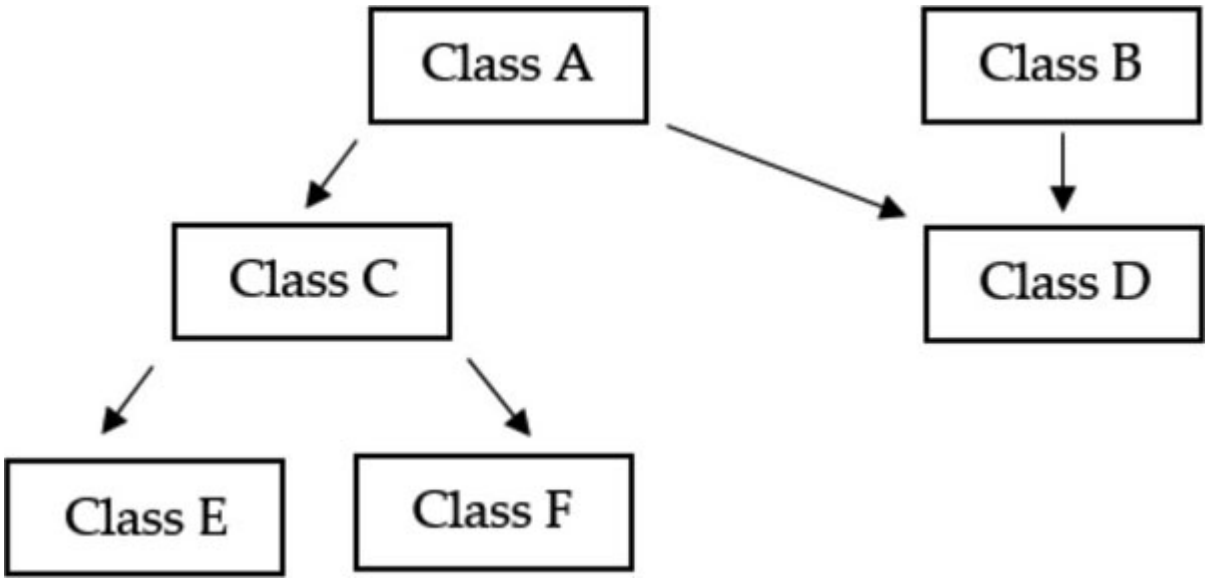


Figure 5.6: *Hybrid inheritance*

From *figure 5.5*, it is evident that **Class D** has two base classes (**Class A** and **Class B**) which is multiple inheritance. Additionally, **Class C** has two subclasses (**Class E** and **Class F**), which is a hierarchical inheritance. The following code snippet in C++ illustrates the same:

```
1. Class A {
2.          Int a_id;
3.          Void a_fun() {
4.          ….
5.          }
```

```
};
Class B {
          Int b_id;
          Void b_fun() {
          ….
          }
};
Class C: public A {
          Int c_id;
          Void c_fun() {
          ….
          }
};
Class D : public A, public B {
          Int d_id;
          Void d_fun() {
          ….
          }
};
Class E : public C {
          Int e_id;
          Void e_fun() {
          ….
          }
};
Class F : public C {
          Int f_id;
          Void f_fun() {
          ….
          }
```

```
};
```

Polymorphism

It denotes the ability of a message to be processed in more than one form. For instance, consider a class **Shape** that has a function named as **draw** to draw a particular shape. Now, we may have different classes, such as **Circle**, **Square**, **Hexagon**, and so on, which derives features from this class **Shape**. So, all these subclasses should define their own implementation of function **draw** because each shape is different. So, the manner in which they are drawn must be defined by their corresponding class only. Look at the following code in C++ to understand this example.

```
Class Shape {
            Void draw() {
            ….
            }
};
Class Circle : public Shape {
            Void draw() {
            ….
            }
};
Class Square : public Shape {
            Void draw() {
            ….
            }
};
```

Polymorphism in C++ is categorized into two types:

- Compile-time polymorphism
- Runtime polymorphism

Compile-time polymorphism could be implemented using function overloading.

In function overloading, we have multiple functions having the same name but different signatures. Different function signature implies that the number and types of argument are different for overloaded functions. Readers must note that return type plays no role in function overloading. Consider the following example of function overloading.

```
1. Class DisplayValue {
2. Public:
3.   Void display (int val) {
4.   Cout<<"Integer data type with value "<<val;
5.   }
6.   Void display (double val) {
7.   Cout<<"Double data type with value "<<val;
8.   }
9.   Void display (string val) {
10.  Cout<<"Integer data type with value "<<val;
11.  }
12. };
```

Runtime polymorphism is implemented using function overriding.

Function overriding: In function overriding, a derived class has a definition of one of the member functions of the base class. Look at the following example of function overriding.

```
1. Class A {
2. Public:
3.   Void display (string msg) {
4.   Cout<<"Printing message from base class A "<<msg;
5.   }
6. };
7. Class B : public A {
8. Public:
```

```
9.  Void display (string msg) {
10. Cout<<"Printing message from derived class B "<<msg;
11. }
12. };
```

Abstraction

It refers to hiding internal details and showing relevant information. Let us see a real-life example to understand this concept. Consider a car driver who knows he can increase speed by applying an accelerator and applying brakes to stop the car. However, the mechanism behind the working of the accelerator and brakes is irrelevant to him, and he need not know all those details to drive a car. So, one can clearly see that only relevant information like applying accelerator increases the speed and applying brakes stops the car is given to the user, and other internal details are hidden. It is an abstraction. Similarly, consider library functions such as sort, pow, log, and so on in C++. A user knows how to use these functions, but the underlying algorithm is hidden from him.

In C++, abstraction can be achieved using classes and header files.

- **Abstraction using class**: Class acts as a container for data and member functions. A class can control the visibility of a member using an access specifier. For instance, a private member is not accessible directly outside the class. So, it remains opaque to the user.
- **Abstraction using header files**: A header file contains a declaration of functions. So, a user just needs to know the function signature to use it. For example, consider a function to sort the data. A user needs to know how to pass the arguments, and the underlying sorting algorithm is completely hidden from him/her.

Encapsulation

It is the wrapping up of data and functions that operate on this data in a single unit, that is, a class. Encapsulation hides the internal representation of the object. This is different from abstraction as abstraction hides the implementation details, and encapsulation is about hiding the information. Let us consider an example of a C++ class that looks like the following:

```
1. Class Adder {
2. Private:
3.   int value1;
4.   int value2;
5. Public:
6.   int get_value1() {
7.   return value1;
8.   }
9. int get_value2() {
10.   return value2;
11.   }
12.   void set_value1(int value) {
13.   value1 = value;
14.   }
15. void set_value2(int value) {
16.   value2 = value;
17.   }
18.   int addValues() {
19.   return (value1+value2);
20.   }
21. };
```

In the preceding example, we can see that the internal representation of the object of class **`Adder`** will be hidden from the user. This is achieved by making the class data—**`value1`** and **`value2`** as private. So, the outside world can access class data using the getter and setter function only. So, this is encapsulation. Also, if we see that the user needs to call the **`addValues`** function to generate the sum and the underlying algorithm for finding sum is hidden from the user, this is how abstraction is achieved.

Conclusion

In this chapter, we learned about different programming paradigms, such as procedural and object-oriented programming. The chapter explained what object-oriented programming is and delineated various OOPs features—class, object, inheritance, abstraction, encapsulation, and polymorphism. A class is a blueprint or template for a user-defined data type, and an object is an instance of the class, and it generally simulates a real-world entity. There are various features in OOPs which are quite useful to solve a real-world problem. Object-oriented programming enables high-level abstraction, so it allows a user to focus on the business problem rather than the programming language complexities like memory management. So, understanding object-oriented programming principles is a must for any system designer.

Points to remember

- A class is a template for a user-defined data type, and it contains data and member functions that operate on those data.
- An object is a class instance that defines a real-world or an abstract entity.
- Object-oriented programming has various features such as abstraction, encapsulation, polymorphism, and inheritance.
- Abstraction implies hiding implementation details and showing relevant details only.
- Encapsulation refers to the wrapping up of data and functions that operate on it into a single unit known as class. It enforces information hiding.
- Inheritance is the ability of a class to reuse features of another class. It promotes code reusability.
- Polymorphism is the ability of a message to be processed in more than one form.

Multiple choice questions

1. **Which of the following is true about inheritance?**
    a. It implies hiding the underlying algorithm of a function.
    b. It implies hiding the internal representation of a class object.

c. It denotes the ability of a class to reuse features of other classes and promotes code reusability.
d. It denotes the ability of a message to be processed in more than one form.

2. **Which one of the following OOP features enforces information hiding?**
    a. Abstraction
    b. Encapsulation
    c. Inheritance
    d. Polymorphism
3. **Function overloading is used to implement which of the following OOP features?**
    a. Abstraction
    b. Encapsulation
    c. Inheritance
    d. Polymorphism

Answers

1. **c**
2. **b**
3. **d**

Questions

1. What are the differences between procedural and object-oriented programming?
2. What are the various features of object-oriented programming?
3. What is the difference between function overloading and function overriding?

Key terms

- Public, private, and protected are the access specifiers in C++.
- Constructor is a member function of a class that is used to initialize objects of the class.

CHAPTER 6

Design Patterns

Introduction

System design is a daunting task and requires various steps. One of the crucial steps in system design is choosing the right strategy to implement the project so that the codebase of the system is readable, flexible, and optimal for the developer. Oftentimes, the developers resort to bad code practices; although they expedite project deliverables but in the long run, it becomes cumbersome and troublesome for the developers. Design patterns are useful for the developers because they provide tested and proven patterns to implement the project. Additionally, design patterns help developers to develop a more maintainable codebase. The aim of this chapter is to explain various design patterns and the scenarios in which they can be used.

Structure

In this chapter, we will cover the following topics:

- Design patterns and their uses
- Types of design patterns

Objectives

After studying this unit, you will be able to understand what a design pattern is and its uses and what are the different types of design patterns.

Design patterns and their uses

Most of the system designers can relate that several problems have a common repeatable solution. So, it makes sense to develop a technique for such commonly occurring problems as it will help save a lot of

development and testing time. Design patterns are created to serve the same purpose. They are proven and tested techniques to solve similar problems. An ad-hoc problem-solving approach might result in several bugs during the production phase, so using design patterns may help prevent such issues.

Types of design patterns

Design patterns are of three types: creational, behavioral, and structural. In the next section, we will see some important creational, behavioral, and structural types of design patterns.

Creational

Creational design patterns are related to object creation. They are generally used to provide a mechanism to create an object based on certain criterion. Some of the important types of creational design patterns are object pool, prototype, and singleton.

Object pool

Object pool design pattern is used when the cost of initializing the object is expensive, and the rate of instantiation is also high; however, the number of objects in use at any point in time is low. Because the cost of creating a new instance is high, it makes sense to create a pool of objects. Whenever a client demands a new object, one of the free objects is returned from this pool. It might be possible that the pool is exhausted; that is, all the objects in the pool are already given. In such a case, the pool can create new objects because it is not desirable for a process to wait for the release of an object. To understand the object pool design pattern, consider a real-world example. Let us say a company exists in your area. Whenever a new employee is hired, the supervisor has to give a proper workstation (computer, desk, and so on) to the new employee. Now, for a company, it is better to keep a pool of workstations in their warehouse so that any time a new or existing employee needs a workstation, they can readily give it to them. Also, it may happen that an employee is leaving the company. So, his/her workstation will be returned back to the company's warehouse, and hence, it will be added back to the pool.

Now, let us see how we can implement an object pool design pattern using C++:

```
Class Resource {
  int value;
  public:
    Resource() {
      value = 0;
    }
  Resource(int value) {
    this -> value = value;
  }
  resetResource() {
    this -> value = 0;
  }
};
Class ObjectPoolDP {
  list < Resource * > objectPool;
  static ObjectPoolDP * ObjectPoolDPInstance = nullptr;
  public:
    static ObjectPoolDP * getInstance() {
      if (ObjectPoolDPInstance == nullptr) {
        ObjectPoolDPInstance = new ObjectPoolDP();
      }
      return ObjectPoolDPInstance;
    }
  Resource * getObject() {
    if (objectPool.empty()) {
      /* Create a new object */
      return new Resource();
    } else {
```

```
29.         /* Return already existing object from the object
    pool */
30.       Resource * resource = objectPool.front();
31.       objectPool.pop_front();
32.       return resource;
33.     }
34.   }
35.   void returnObjectToPool(Resource * resource) {
36.     resource -> resetResource();
37.     objectPool.push_back(resource);
38.   }
39. };
```

Prototype

Consider a situation when a user has to create a copy of an existing object. A trivial way to deal with this problem is to create a new object of the same class, one by one, copy all the field values of the existing object to the new object, and return the new object. However, there are some pitfalls to this approach. First, class fields may be private. So, a user might not be able to access those fields directly. Second, user code becomes dependent on that class which may be problematic. A better way to deal with this problem is to use a prototype design pattern. This design pattern delegates the responsibility of cloning the object to the actual object itself. Readers must note that the object to be cloned is known as a prototype.

Consider the following C++ code snippet that implements the prototype design pattern using the preceding strategy.

```
1. Class PrototypeDP {
2.   int x;
3.   int y;
4.   public:
5.     virtual PrototypeDP * clone() = 0;
6.   PrototypeDP(const PrototypeDP & obj) {
```

```
7.      this -> x = obj.x;
8.      this -> y = obj.y;
9.    }
10. };
11. Class PrototypeDPDerived1: public PrototypeDP {
12.    public: PrototypeDPDerived1(const PrototypeDPDerived1 &
    obj): PrototypeDP(obj) {
13.    }
14.    PrototypeDP * clone() {
15.      return new PrototypeDPDerived1( * this);
16.    }
17.    /* Other Functions */
18.    …
19. };
20. Class PrototypeDPDerived2: public PrototypeDP {
21.    public: PrototypeDPDerived2(const PrototypeDPDerived2 &
    obj): PrototypeDP(obj) {
22.    }
23.    PrototypeDP * clone() {
24.      return new PrototypeDPDerived2( * this);
25.    }
26.    /* Other Functions */
27.    …
28. };
```

Singleton

The intent of the Singleton design pattern is to ensure that the class has only one instance and to provide global access to this object. For example, consider the office of the Prime Minister of India, which is a singleton. Because there can be at most one active Prime Minister at any point of time, so at any point of time, the Prime Minister of India will be a global point of

access, and it will return a single name. Consider another example of a database connection object. Creating a database connection object can be quite expensive for some applications. So, a developer has an option of using the Singleton design pattern, and every time a connection is needed, the class will always return a single object.

A developer can use the following strategy to implement a singleton design pattern:

Make the constructor private to prevent the client from using **`new`** operator to create a new object.

Expose a new static function, say **`getInstance`**, which delivers a class object to the client. Behind the scene, this method will call the class constructor when it is called for the first time, it will save that object in a static field, and then all the subsequent calls to **`getInstance`** will return this cached object only. Please note that the client is unaware of this implementation (OOPs concept).

Consider a C++ code snippet that implements the singleton design pattern using the preceding strategy:

```
Class SingletonDP {
  private: static SingletonDP * SingleDPInstance = nullptr;
  SingletonDP() {
    //Default constructor
  }
  public: static SingletonDP * getInstance() {
    if (SingletonDPInstance == nullptr) {
      SingletonDPInstance = new SingletonDP();
    }
    return SingletonDPInstance;
  }
};
```

Behavioral

Behavioral design pattern identifies the pattern of communication between objects, thus, making communication flexible among these objects. Some of the important Behavioral design patterns are a chain of responsibility, mediator, and observer.

Chain of responsibility

It is a behavioral design pattern where we have a chain of handlers. The request passes through this chain of handlers, and each handler decides if it should process the request or pass it to the next handler in the chain. If a handler decides to process the request, then it does not send the request to the next handler in the chain. Careful readers may observe that this design pattern can be used when multiple handlers can handle a request. The design pattern is useful for scenarios when the program is expected to handle different types of requests, but the sequence and exact kind of request is unknown.

A real-life problem that can be solved using the chain of responsibility could be the support center of a product. For instance, you purchased a brand new laptop. You unboxed it, and you quickly downloaded a few games to play. Once the games are downloaded, you start one of them. However, you realize that the game has begun, but there is no sound. You tried to find out what was wrong with the sound but ended up with no luck. So, you finally decided to call the customer care number. Now, if you observe it carefully, there are a series of things that happen afterward. First, the call is connected to an automated system that presents you with different options like Press 1 if you want to know the price of xyz…, or Press 2 if you wish to talk to an executive, and so on. Second, you are connected to an operator who asks you to explain your query and suggests a few steps to help you. However, it may happen that the audio is still not coming even after trying all the suggested steps by the operator. Because the operator is unable to resolve your issue, he/she transfers your call to a tech specialist who is an expert in handling the issues. So, finally, the call is transferred to him, and within a minute, he is able to find the root cause of the issue and suggests you a solution for the same. If we see it carefully, we have three different handlers in this situation, and if at any stage a particular handler is unable to process the request, it sends it to the next handler in the chain. All the three handlers are connected next to each other in the following fashion:

Automated system -> Operator -> Tech Specialist

Also, it might be the case that the customer just wants to know about the price/details of a product which could be handled by the automated system or operator, or the concern is too complex to be resolved over the call. In that case, none of the handlers would be able to process the request. To summarize, the chain of responsibility design pattern could be applied to use cases like this where sequence and kind of requests are unknown, and the program is expected to handle different kinds of requests.

Mediator

Consider the air traffic controller room, whose function is to coordinate with the pilots and inform them about any potential traffic or points of concern. Now, imagine if the air traffic controller room ceases to exist. In such a scenario, pilots have to communicate with each other to know about the potential traffic they may encounter in their route. This would be chaotic, and it is undesirable. If we observe carefully, the air traffic controller room acts as a layer between any two pilots and a pilot instead of communicating directly to another pilot, interacts with the air traffic controller room. The mediator design pattern is used to serve such kinds of use cases. It reduces the chaotic dependencies between objects and restricts any direct communication between them by forcing objects to collaborate and communicate via the mediator object. Consider the following code snippet that attempts to showcase the use of mediator design pattern on the air traffic controller:

```
1. Class Item
2. class TrafficController {
3.   virtual void broadcast(string from, string msg) = 0;
4.   virtual void message(string from, string to, string msg)
   = 0;
5. };
6. class Pilot {
7.   string pilot_name;
8.   TrafficController * traffic_controller;
9.   vector < string > msg_log;
```

```
10.  Pilot(string pilot_name): m_name(pilot_name) {}
11.  void broadcast(string msg) {
12.    traffic_controller -> broadcast(pilot_name, msg);
13.  }
14.   void personalMsg(string destination_pilot_name, string
     msg) {
15.     traffic_controller -> message(destination_pilot_name,
     to, msg);
16.  }
17.  void receive(string source_pilot_name, string msg) {
18.    string s {
19.      "
20.       Receieved a message from" source_pilot_name + ": \""
     + msg + "\""
21.    };
22.    msg_log.push_back(s);
23.  }
24. };
25. class TrafficControllerHQ: TrafficController {
26.  vector < Pilot * > pilots;
27.  void broadcast(string source_pilot_name, string msg) {
28.    for (auto p: pilots)
29.      if (p -> pilot_name != source_pilot_name)
30.        p -> receive(source_pilot_name, msg);
31.  }
32.       void   message(string   source_pilot_name,   string
     destination_pilot_name, string msg) {
33.    auto target = find_if(begin(pilots), end(pilots),
34.      [ & ](const Pilot * p) {
35.        return p -> pilot_name == to;
36.      });
```

```
37.          if (target != end(pilots))( * target) ->
   receive(source_pilot_name, msg);
38.   }
39. };
```

Observer

The observer design pattern is used when there are one to many dependencies so that if one object changes its state, then all its dependent objects are notified. For instance, consider a new phone that is about to launch and people all over the world are excited to buy it. People are visiting the official website of that phone company, and the website provides an option Notify, when available. When a user clicks on this option, the website asks for the user's email address. So, people who really want this phone will be subscribing to this option so that as soon as the product is available, they are notified, and they can immediately go to the website to purchase it. To support this scenario, an observer design pattern can be used.

Consider the following C++ code of an overly simplified version of observer design pattern:

```
1. Class Observer {
2.   Public: Void notify() {
3.     /* Logic to notify the user */
4.   }
5. };
6. Class Subject {
7.   list < Observer > observersSubscribedToSubject;
8.   Public:
9.     void registerObserver(Observer & observer) {
10.      /* Logic to register the observer */
11.      observersSubscribedToSubject.push_back(observer);
12.    }
13.  void notifyObservers() {
```

```
14.     /* Logic to notify the observers */
15.     for (Observer & observer: observersSubscribedToSubject)
    {
16.       observer.notify();
17.     }
18.   }
19. };
```

Structural

Structural design pattern attempts to simplify the design by identifying the easy route to realize relationships among objects. Some of the important structural design patterns are bridge, decorator, flyweight, and proxy.

Bridge

The bridge design pattern is used when the intention is to separate abstraction from the implementation so that the two can vary independently. It allows us to divide a large class into two separate dimensions—abstraction and implementation, which can be developed independent of each other. To understand it better, consider a program that lets you draw two different shapes of rectangle and circle of three different colors say, red, yellow, and green. Now, a trivial solution is to create a class **`Shape`**, extend two classes – rectangle and circle and then further create colored classes like **`RedSquare`**, **`RedCircle`**, and so on; that is, we will be creating six different subclasses. Furthermore, adding new colors and new shapes means we will end up in creating subclasses in an exponential manner. Bridge design pattern attempts to solve this problem in a much cleaner way by making a switch from inheritance to object composition. It essentially implies that we can extract one of the dimensions in a different hierarchy itself. So, using this approach, we can bifurcate the hierarchies in the following manner:

- Create a base class **`Color`** with three subclasses – **`Red`**, **`Yellow`**, and **`Green`**.
- Create a base class **`Shape`** with two subclasses – **`Circle`** and **`Rectangle`**.

Using the preceding division, we can have the Shape class to possess a field pointing to a color object. Also, adding new colors will not pose any problem as no change will be needed in **Shape** class and its subclasses. Consider the following code snippet that uses a bridge design pattern for the same problem:

```
1. Class Shape {
2.   Color;
3.   Public:
4.     virtual void draw() = 0;
5. };
6. Class Circle: public Shape {
7.   Int radius;
8.   Public:
9.     void draw() {
10.      /* Logic to draw circle */
11.      color.fillColor();
12.    }
13. };
14. Class Rectangle: public Shape {
15.   Int length;
16.   Int breadth;
17.   Public:
18.    void draw() {
19.      /* Logic to draw rectangle */
20.      Color.fillColor();
21.    }
22. };
23. Class Color {
24.   Void fillColor() = 0;
25. };
```

```
Class RedColor: public Color {
  Void fillColor() {
    /* Fill red color */
  }
};
Class YellowColor: public Color {
  Void fillColor() {
    /* Fill yellow color */
  }
};
Class GreenColor: public Color {
  Void fillColor() {
    /* Fill green color */
  }
};
```

Decorator

A decorator pattern is a structural design pattern that is used when we want to attach additional functionalities to the object at runtime. It provides a clean and flexible alternative to creating several subclasses for extending the object's functionality.

Consider a simple example to understand decorator design patterns.

Let us say we want to implement a class that should be able to draw different figures such as rectangle, triangle, and so on. Now, the user comes with a new requirement that the figure to be drawn can be of different colors. A trivial way to handle this problem would be to create a class such as **`Triangle`**, **`Rectangle`**, and so on, and then use inheritance to create more subclasses such as colored triangle or colored rectangle. However, an elegant solution is to apply a decorator design pattern to this problem. The following C++ code snippet applies decorator design pattern to this problem:

```
Class Shape {
```

```
Public:
  virtual string getShapeType() = 0;
  virtual void draw() = 0;
};
Class Triangle: Shape {
Int base;
Int height;
Public:
  String getShapeType() {
   return string("A triangle with base and height as ") + to_string(base) + string(", ") + to_string(height);
  }
  void draw() {
  /* Logic to draw a triangle */
  }
};
Class Rectangle : Shape {
Int length;
Int breadth;
Public:
  String getShapeType() {
   return string("A rectangle with length and breadth as ") + to_string(length) + string(", ") + to_string(breadth);
  }
  void draw() {
  /* Logic to draw a rectangle */
  }
};
Class ColoredShape : Shape {
Shape* shape;
```

```
30. String color;
31. Public:
32.   String getShapeType() {
33.   return shape->getShapeType() + " and filled with color "
    + color;
34.   }
35.   Void draw() {
36.   /* Logic to draw a colored shape */
37.   }
38. };
```

Flyweight

The flyweight design pattern is used when the system has to support a large number of similar objects, and these objects contain similar fields that can be shared. The main point to note here is that these objects, if stored individually, would eat up the entire memory. So, the sharable data (the one which is common to these objects) is stored in an external data structure, and it is passed to the object when needed. To understand it better, consider a text editor like MS Word. In a text editor, we write characters such as A, B, and so on, where each character generally contains attributes like font size, color, and other formatting data. Now, most of the characters will have the same size, color, and so on. So, for a very large document containing several thousands of characters, if we store these attributes individually, we might end up in exhausting the whole memory. Flyweight design patterns are useful in such scenarios. We can create an object that will store the formatting data of a character, that is, it will store size, color, and so on, and each character object can contain a reference to this object. In this manner, we only have to store the character position and a reference field pointing to this object in the character object. It will tremendously reduce the memory footprint of the program.

Let us consider a simple program to understand the flyweight design pattern. The program should be able to render millions of stars on a drawing board, and as a developer, our intention is to reduce the memory footprint of the program.

```
1. Class StarType {
2.   String texture;
3.   String color;
4.   StarType (string texture_u, string color_u) {
5.   texture = texture_u;
6.   Color = color_u;
7.   }
8.   Static Void draw(drawing_board dobj, int x, int y) {
9.   /* Logic to draw the Star at coordinates x and y */
10.  }
11. };
12. Class StarFactory {
13.  Map<pair<string,string>, StarType > StarTypes;
14.    Static StarType findStarType(string texture, string
    color) {
15.  If(StarTypes.count({texture,color}) > 0) {
16.   Return StarTypes [{texture,color}];
17.    }                          StarType sType = new
    StarType(texture,color);
18.  StarTypes.add(sType);
19.  Return sType;
20.  }
21. };
22. Class Star {
23.  Int x;
24.  Int y;
25.  StarType star_type;
26.  Void draw(drawing_board dobj) {
27.  StarType::draw(dobj,x,y);
28.  }
```

```
29. };
30. Class Galaxy {
31.   Map<star,star> stars;
32.   Void addStar(int x, int y, string texture, string color)
    {
33.   StarType sType = StarFactory.findStarType(texture,color);
34.   Star star_u = new Star(x,y,sType);
35.   Stars.insert(star_u,star_u);
36.   }
37.   Void draw(drawing_board dobj) {
38.   for(auto s: stars) {
39.    s.draw();
40.   }
41.   }
42. };
```

Proxy

A proxy design pattern provides controlled access to the original object, thus, allowing the user to perform certain operations before or after the request goes to the original object. The proxy acts as a wrapper to the original object so that all client requests are routed to it, and behind the scene proxy access the original object. The proxy serves the following purposes:

- It can check if the caller has permission to access the actual object.
- It can check if certain preconditions are satisfied before accessing the actual object.
- It can prevent object creation if the object is expensive to create by caching the original object.

The following is a simple example to understand the proxy design pattern:

```
1. class IPassenger {
2.   public:
```

```
3.     virtual~IPassenger() {
4.       cout << "IPassenger destructor!" << endl;
5.     }
6.   virtual void canBoardPlane(int flight_number) = 0;
7. };
8. class Passenger: public IPassenger {
9.   public: void canBoardPlane(int flight_number) {
10.        cout << "Flight" << flight_number << "has been boarded!" << endl;
11.   }
12. };
13. class ProxyPassenger: public IPassenger {
14.        public:    ProxyPassenger(int    booked_flight_num): booked_flight_no(booked_flight_num) {}
15.   void canBoardPlane(int flight_num) {
16.     if (flight_num == booked_flight_num) {
17.       real_passenger -> canBoardPlane();
18.     } else {
19.        cout << "Sorry, the given passenger cannot board the plane" << endl;
20.     }
21.   }
22.    private: unique_ptr < IPassenger > real_passenger_ = make_unique < Passenger > ();
23.   int booked_flight_no;
24. };
25. int main() {
26.    unique_ptr < IPassenger > passenger = make_unique < ProxyPassenger > (307);
27.   passenger -> canBoardPlane();
28.   passenger = make_unique < ProxyPassenger > (601);
```

```
29.   passenger -> canBoardPlane();
30. }
```

In the preceding example, **`Passenger`** class has a proxy class **`ProxyPassenger`**. If we observe carefully, first, the call to **`Passenger`** class is routed to the proxy class, which is performing a check and then routing the call to the original class. So, the proxy class acts as a wrapper and controls access to the original class.

Conclusion

Design patterns are useful for every system designer because they provide well-proven and tested approaches to commonly occurring problems. Design patterns can be creational, structural, and behavioral. Creational design pattern enables the creation of an object in a controlled manner and subjects it to the fulfillment of certain criterion. Structural Design pattern simplifies the design by identifying the relationship among objects to form a larger structure. Behavioral design pattern identifies the common communication pattern between the object to find out the pattern. Some of the important creational design patterns are object pool, prototype, and Singleton. Behavioral design patterns are a chain of responsibility, mediator, and observer, and structural design patterns are bridge, decorator, flyweight, and proxy. In the upcoming chapter, we will discuss about object-oriented analysis and design methodology. A systematic way to approach a system design problem, and along with it, different tools and techniques available to a designer will be covered in the upcoming chapter.

Points to remember

- Design patterns provide a well-proven and tested approach to commonly occurring problems.
- Design patterns can be creational, structural, and behavioral.
- Creational design pattern controls object creation based on certain criterion.
- Structural design pattern identifies the common characteristics among objects to form larger structures.

- Behavioral design patterns identify the common communication pattern among objects.

Multiple choice questions

1. **Which of the following is a creational design pattern?**
    a. Singleton
    b. Proxy
    c. Mediator
    d. Chain of responsibility
2. **Which of the following is a behavioral design pattern?**
    a. Bridge
    b. Flyweight
    c. Mediator
    d. Object pool

Answers

1. **a**
2. **c**

Questions

1. What is a design pattern, and why it is important for a system designer to learn them?
2. What are the different types of design patterns?

CHAPTER 7

Object-Oriented Design Methodology

Introduction

Object-Oriented Programming (**OOP**) is based on the concept of objects and classes. Objects are containers for data, and they possess attributes or properties, or fields. Class is a blueprint or template for a user-defined data type. A class contains data (they are the same as the various fields that an object possesses) and functions that operate on these data. Object-oriented analysis and design is a technique to analyze and design a system or application by applying OOPs principles. This technique also enables using visual modeling to guide the product stakeholders and the different developers involved in the product development.

A good system designer never jumps directly to writing code. It is imperative to first layout the design of the whole application diagrammatically or uses some other tools in order to get a vivid understanding of the whole system. Object-oriented design methodology refers to using object-oriented design principles and techniques to develop a system or application. The aim of this chapter is to explain about various tools that are available for developers which they can use to define the system pictorially in terms of several building blocks and at the same time also show various users who are going to interact with the system. The chapter is important for those readers who are preparing for the system design interviews because in every system design interview, a candidate is expected to describe the application's high-level design, low-level design, class diagram, and so on, and in this chapter, we will see how to use these tools in the most efficient manner.

Structure

In this chapter, we will cover the following topics:

- Insight into object-oriented design methodology

- Performing object-oriented analysis and design

Objectives

After studying this unit, you will be able to understand what is meant by object-oriented analysis and design. The chapter also teaches different tools available for a developer to efficiently design a system based on object-oriented design principles.

Insight into object-oriented design methodology

Based on the popular waterfall model, the software development lifecycle can be described in the following manner:

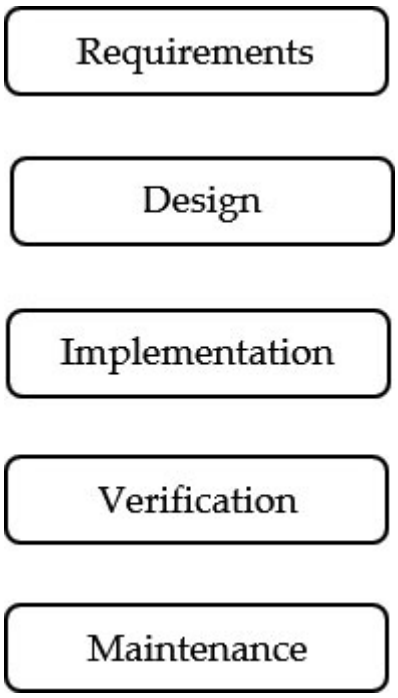


Figure 7.1: *Software development life cycle*

If we see carefully, the software lifecycle starts with a rough description of the problem for designing, coding, testing, and finally deployment. The initial phases of this lifecycle constitute analysis and design. It is a prudent technique to first model the application using tools like UML and then jump to the implementation part as it helps create the systems which are more extensible, maintainable, and flexible. Object-oriented analysis is about

figuring out “what to do”, and object-oriented design is about finding out how to do it.

The first step in system designing is to gather the requirements. It essentially implies defining the problem we are trying to solve and deciding what the system will do. We may also decide what are the necessary features or must to have and desirable or nice to have features that our system should possess. A good example could be, consider a bank app with the following features:

- See the account balance
- Transfer the fund to another bank account
- Report an issue

Needless to say, these are the must to have features for the bank app. Now, we may list some desirable or nice to have features like:

- Send an alert to the user if his/her account is debited
- Send an alert for any upcoming maintenance

In a similar manner, a system designer has to list essential and desirable features for his application. One must note that the aim here is to get started by listing some essential/desirable requirements and also at the same time not to go too deep into it so as to prevent getting stuck in this requirement phase. We should list the bare minimum set of requirements for our product. Also, it is perfectly okay to miss some of the requirements at this stage, as we can always update the requirements list later if needed.

FURPS+ requirements

FURPS+ is an acronym for functionality, usability, reliability, performance, and supportability. It is one of the most commonly used models for classifying software quality attributes. It serves as a checklist while defining requirements.

- **Functionality**: It refers to the core features of the system. It defines what the customer wants.
- **Usability**: It refers to how intuitive the application is to the user. Usability signifies if the application is easy to use or not.

- **Reliability**: It is concerned about parameters such as availability, recoverability, and accuracy. It revolves around questions such as acceptable downtime, measures to recover the system, and so on.
- **Performance**: It includes aspects like system throughput, response time, startup time, and time to recover from an erroneous state.
- **Supportability**: It revolves around parameters like whether the application can be tested, serviced, installed, and configured.

In addition to the preceding requirements, the FURPS+ model allows us to specify more constraints like design, implementation, interface, and physical.

- **Design constraints**: It specifies constraints on how the system must be built because of limitations like the system requiring the relational database.
- **Implementation constraints**: It specifies limitations on coding like implementation language, platform, standards, or design methodologies.
- **Interface constraints**: It is a requirement when the system needs to interface with an external system.
- **Physical constraints**: It is related to the hardware on which the system will be deployed.

Readers must realize that the FURPS+ model is just to give direction to the users to find out the key requirements. It might be possible that only a few of the constraints are relevant for a particular project.

Performing object-oriented analysis and design

Object-oriented analysis and design is a technique to decompose systems into objects and identify relationships between them. It is essential that a developer can use the output of the analysis and design phase to write code in their favorite object-oriented language. The section covers different techniques and tools available to perform the analysis and design phase of the system in the most efficient manner.

Defining system requirements

It is time for a system designer to define the requirements. As we have seen in the previous section, the FURPS+ model is there to help us in finding the right direction to list the requirements. Let us put it into use to define the requirements for a musical Jukebox. A jukebox is a music-playing device. Some of the functional requirements that the Jukebox must possess could be the following:

- Maintain the library of songs and albums
- Allow a user to navigate songs and albums
- Allow a user to select and play a song or album
- Maintain a queue of songs to play
- Allow a user to create a playlist
- Allow a user to add new songs and albums

Now, some non-functional requirements could be:

- It should consume less power

Identifying the use cases and actors of the system

Once the requirements are gathered, the next step for a system designer is to define the use case. At this stage, the system designer has to identify the actor (or user) and how he is going to fulfill the goal. A typical use case should have three elements:

- **Goal**: What the system is trying to achieve
- **Actor/user**: Who is going to interact with the system
- How we are going to achieve the goal

We may add some additional pointers to the use case, like what is going to happen if the normal course of events does not happen or some error creeps in.

Identifying the actor or user is a crucial and indispensable step before writing the use cases. At times, identifying them is easy, and at times it is not. For example, for a Jukebox, the primary actor is going to be the person who is interested in listening to music. A use case may have more than one actor; however, one must note that the primary actor, that is, the one who initiated the task, will always be one, though we may have multiple secondary actors.

Once we are done writing use cases and identifying actors, we need the means to describe the interaction between these use cases and the actors using some modeling tool. UML provides a useful tool—use case diagram for this purpose. Use case diagrams are no replacement to writing use cases; they act as a supplement to it. Use case diagrams are non-technical in nature and hence, can be shared with business users.

User stories are another tool in the arsenal of a system designer. They tend to be smaller and simpler as compared to a use case. They are generally one or two sentences long with the intent to be quick to read and understand. User story and use case may seem the same, but they are different in the aspect that user story acts as a placeholder for conversation, that is, a reminder for getting into the depth of something. However, the use case acts as a record of the conversation that has already happened and delineates how a particular goal can be achieved. One must note that both user story and use case can be used, and the order in which they can be used is flexible. Both of them are useful tools to describe the goal of the system and provide useful information for the next stage of analysis and design.

Let us apply these learnings to our Jukebox example. We have already stated the Jukebox functional and non-functional requirements in the previous section. Let us move on to how we can write use cases and user stories for the system. The primary goal of Jukebox is to play a song. So, the primary actor is going to be a user, the one who will be playing the song or any album. Now, let us see some use cases for the Jukebox:

Title: Play a song

Primary Actor: User

Success Scenario: User browses a library of songs and selects a song to play. System plays the selected song.

Title: Select multiple songs to play

Primary Actor: User

Success Scenario:

1. User browses a library of songs.
2. User selects a song to play.

3. Jukebox plays the selected song.
4. User selects another song from the library.
5. Jukebox adds the song to a play queue.
6. Jukebox picks up the next song from the queue and plays it.

Title: Create a playlist
Primary Actor: User
Success Scenario:

1. User selects create a new playlist icon on the jukebox.
2. User adds a name to the playlist.
3. User clicks on add a new song to the playlist.
4. User browses a library of song and add songs to the playlist.
5. User clicks on finish.

Let us create user stories now. A user story may have different formats, but we are going to stick to a common format like:

As a <role>, I want <feature>, so that <receive benefit>.

Using the preceding format, we can write a few use cases for Jukebox like:

As a user, I want to list all the songs of a movie so that I can listen to all the songs of my favorite movie.

As a user, I want to sort songs based on the release date so that I can listen to the latest songs.

Readers can see that on similar lines, we may write different use cases and user stories for the Jukebox. Similarly, these principles can be applied to any system design problem. After writing user stories and use cases, we are done with the analysis part of the system design. Now, we have to jump into the design stage. At this stage, it is imperative to decide how we are going to define our solution. We have to define a conceptual model to identify the most important objects in our system and the relationship between these objects. In order to identify the potential objects in our system, we can follow a simple rule. We can look at the user stories and use cases and try to pick up the relevant noun in them; post this, we can list all the nouns, and

generally, these nouns would be our potential objects. So, for the Jukebox example, the nouns that can be our objects are User, Song, Album, Library, Playlist, and Queue. Once the nouns (or actors) are identified, we need to identify the relationship among them. The key to identifying the relationship is to see the verb phrases in our use cases or user stories. For example, the user browses the library, selects a song, creates a playlist, and so on. When we list such verb phrases, it becomes intuitive to establish a relationship among the nouns. Let us put these findings into a conceptual diagram:

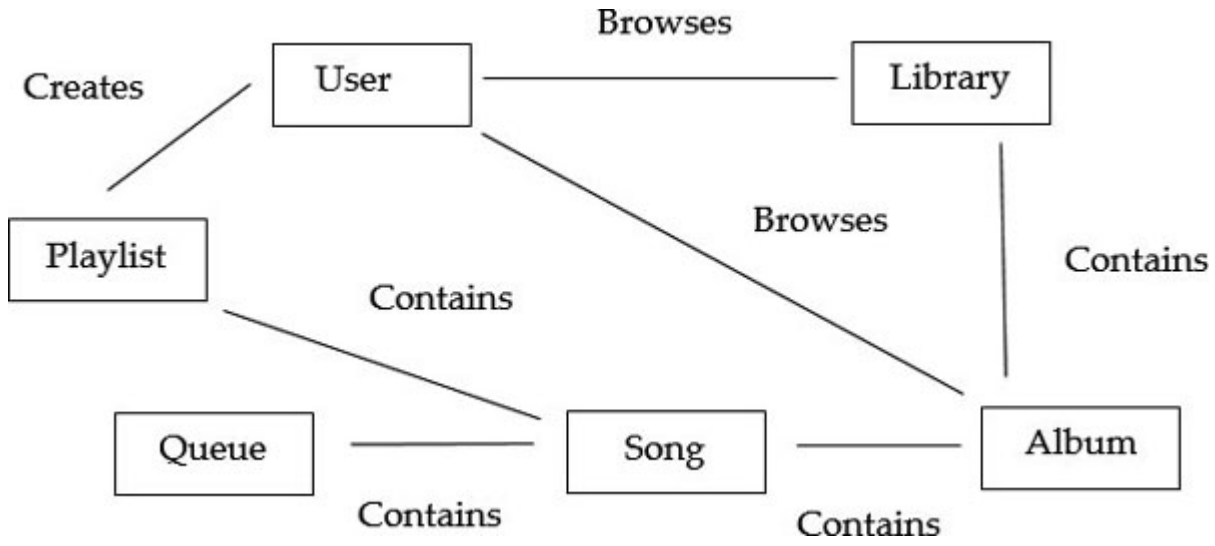


Figure 7.2: *Conceptual diagram*

The preceding diagram simply contains objects and the relationship between them. As we inferred from the use cases and user stories that **User** can create a playlist, and those playlists essentially contain songs. These two informations are represented by drawing an undirected line between User and **Playlist** and **Playlist** and **Song**. On similar lines, we have drawn an undirected line between different objects. Careful readers can see that now we have identified major objects for our system. So, let us say if we are using object-oriented language like C++. Then, we know that we need to have a class for **User** object, **Song** object, and all the objects we have identified so far. Thus, analysis and design methodology helps us to identify key building blocks in our system and provides the necessary inputs to the implementation phase of the system design.

Class diagrams

In the previous section, we saw that we can start with use cases and user stories for a system design. Then, we can pick nouns from both of them, and these nouns could be our potential objects. The verbs in the use cases and user stories help us to identify the relationship between the objects. Finally, using all these informations, we can create a conceptual model of the system. However, the conceptual model is kind of rough. Fortunately, there are UML-based tools to depict the same information in a much cleaner and standard way. UML class diagram is one such tool that we will be discussing in this section.

We can represent a class in the following manner:

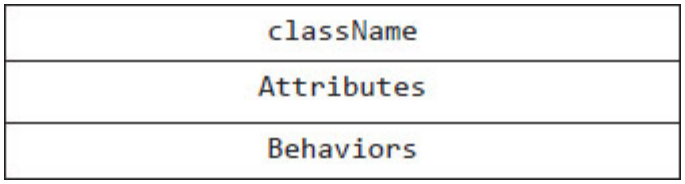


Figure 7.3: *Class representation*

One must note that **`className`** should be singular only, and the convention is to keep the first character of **`className`** in uppercase. Attribute names can follow your favorite programming convention like camel case. They are written in the following manner:

```
<attribute_name> : type = <default_value>
```

Figure 7.4: *Attribute representation*

Similarly, behaviors can be written as a simple function name and arguments followed by the data type that represents the return value type.

```
<behavior_name>(parameter type) : <return_type>
```

Figure 7.5: *Behavior representation*

Let us take an example of Jukebox design and represent a class for **`Song`**:

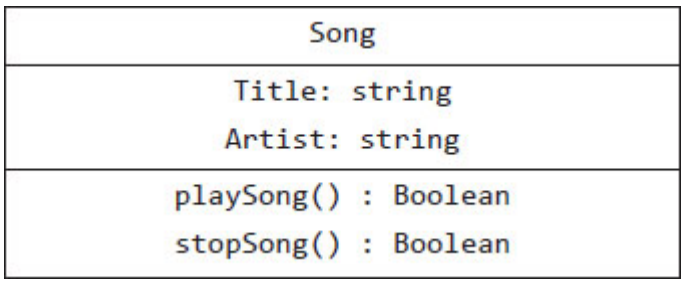


Figure 7.6: *Song class representation*

To show visibility of attributes and behaviors, that is, whether they are private or public, we can add a + or − sign before the attributes. Minus sign − is indicative that the attribute is a private member, and plus sign + implies that the attribute is a public member.

So, for the same **`Song`** class, we can declare both attributes of class **`Song`** as private by preceding them with – sign, and we can declare both the behaviors as public by preceding them with a plus + sign.

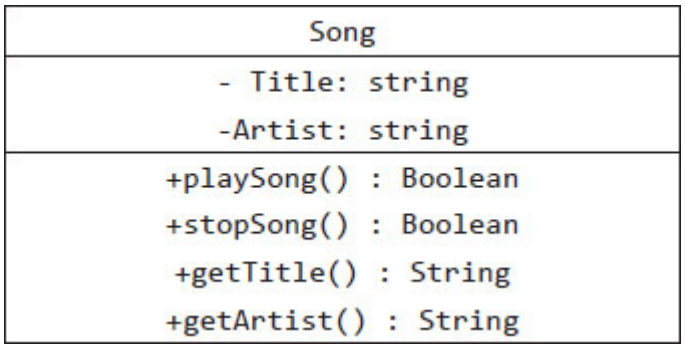


Figure 7.7: *Representing access modifier in the song class*

With the class diagram in place, it becomes easy to transform it into code. For example, the Song class in a very simplified manner can be represented in C++ in the following way:

```
1. Class Song {
2. Private:
3.   String title;
4.   String artist;
5. Public:
6.   Boolean playSong() {
7.    /* Logic to play the song */
```

```
8.    }
9.  Boolean stopSong() {
10.     /* Logic to stop the song */
11.   }
12. };
```

Now, let us create a class diagram for other objects identified earlier:

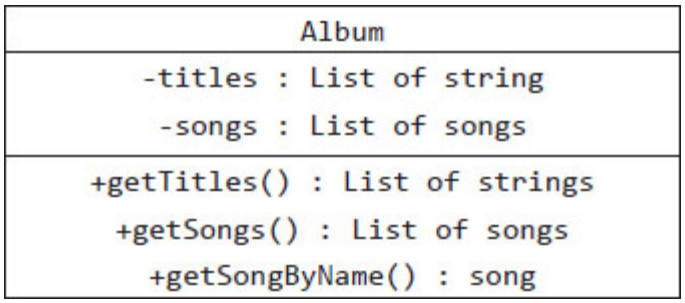


Figure 7.8: *Album class representation*

The point to note here is a data type; that is, whether **`getTitles`** returns a list, array, or any other data structure is not relevant in the UML class diagram. The intention is to show that we need to return a collection of strings from **`getTitles`** function. Choosing between array or list or any other data structure is a matter of discussion during the implementation phase.

On the same lines, the playlist class diagram can be represented in the following manner:



Figure 7.9: *Library class representation*

If there are any static variables or methods, then they are just underlined in the UML class diagram. We may also have to represent inheritance in our class diagram. Inheritance is determined if there is *a* relationship and the

way to represent it is to draw an arrow from derived class to base class. To illustrate, consider an example of a college portal. We can have different types of people logging into the portal, such as **`Student`**, **`Teacher`**, and **`admin`**. Now, each of these can be represented using the class diagram in the following manner:

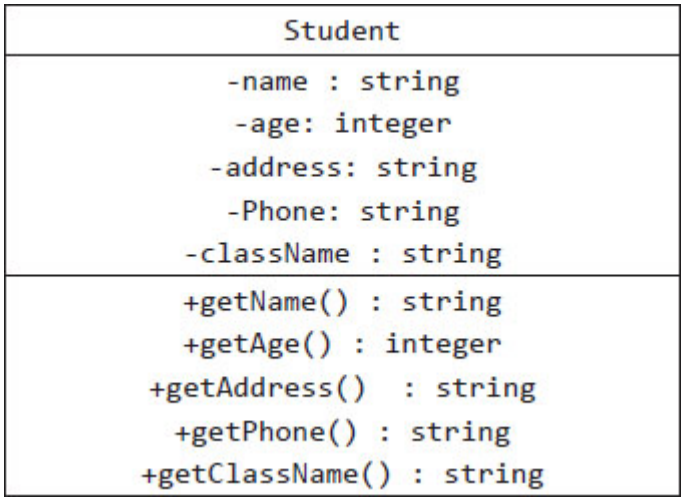


Figure 7.10: *Student class representation*

In the same way, **`Teacher`** class can be represented in the following manner:

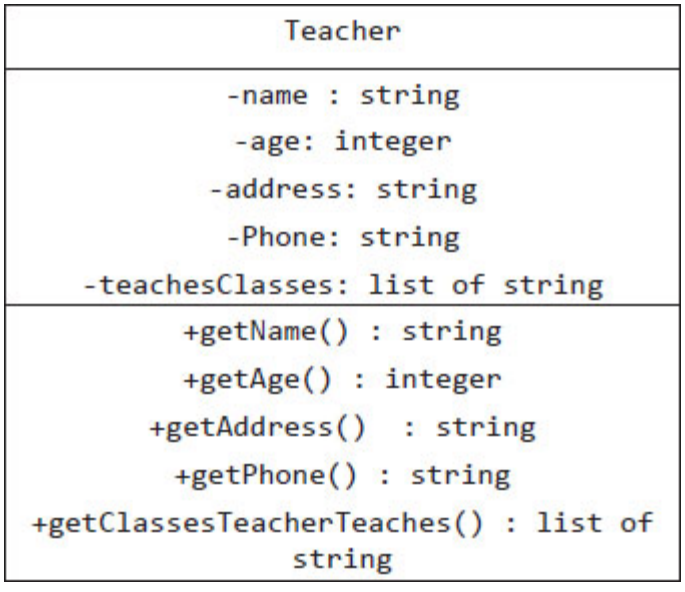


Figure 7.11: *Teacher class representation*

Following the same way, our **`Admin`** class will look as follows:

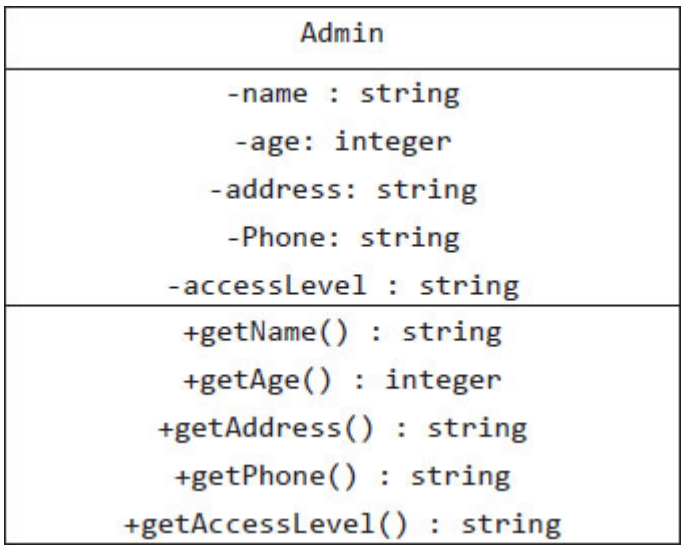


Figure 7.12: *Admin class representation*

If we carefully see, all these classes have several common attributes such as **`name`**, **`age`**, **`address`**, and so on. Also, we can establish *a* relationship in the following manner: **`Student`** is a **`Person`** or **`Teacher`** is a **`Person`**, or **`Admin`** is a **`Person`**. So, we can create a base class **`Person`** that possesses attributes common to these classes, and these classes derive from this base class. In the class diagram, this can be represented in the following fashion:

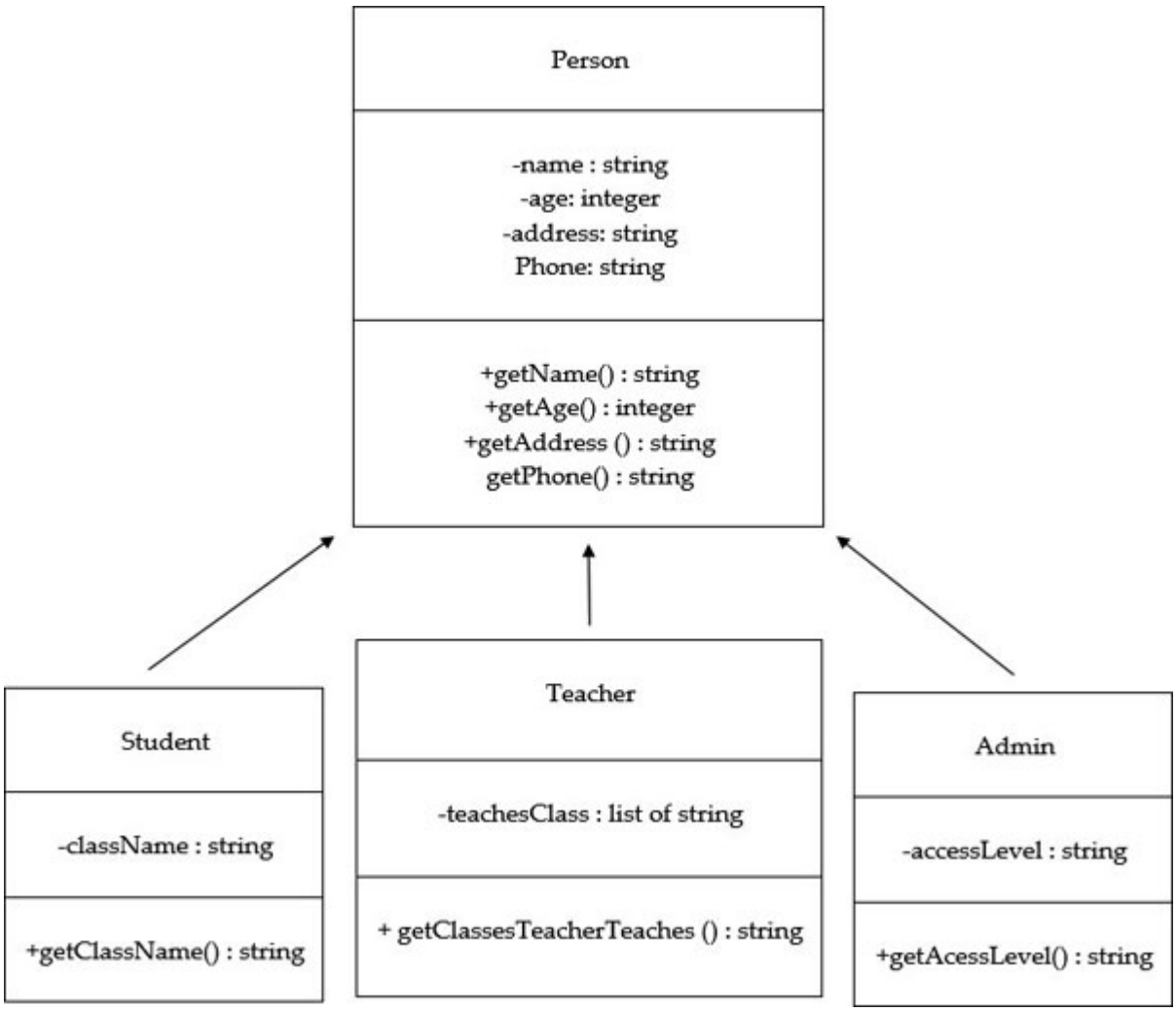


Figure 7.13: *Representing is a relationship*

There are some more concepts centered on UML class diagrams which are recurring in many system design problems. For instance, we may have abstract classes in our design. We represent the abstract class in the class diagram in a similar way as normal classes, with the only difference being that the abstract class name is written in the italicized form. We may have protected members in our system design. They are preceded with a `#` sign.

Inheritance is one type of relationship that can exist between the two objects. Similarly, we have aggregation, which is another concept and it specifies *has a* relationship between two objects. For example, a **`Car`** has **`Wheel`**. One must note that in aggregation, both the objects exist independently. So, if one is destroyed, the other one still exists. To depict aggregation using the UML Class diagram, we use the following notation:

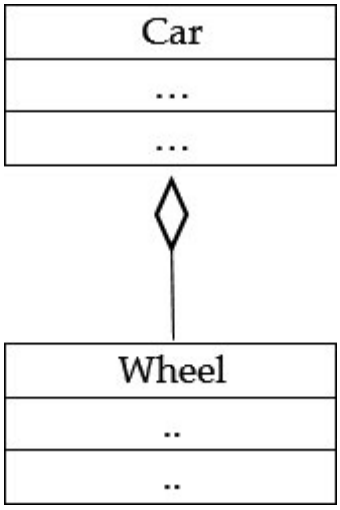


Figure 7.14: *Aggregation*

A composition is a specific form of aggregation. It is also centered around *has a* relationship, but it lays emphasis on ownership. For instance, a car has an engine, but the engine is inherent to the car in such a way that if the car is destroyed, then the engine is destroyed too. In other words, an engine has no meaning or purpose without a car. We represent such relationship in the UML diagram in the following manner:

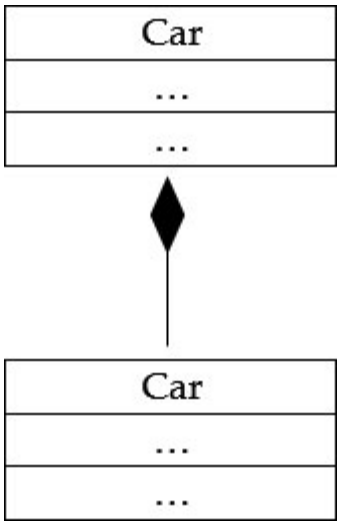


Figure 7.15: *Composition*

Now, we have seen different methodologies of the UML class diagram. This places us in a good place to start analysis and design of any system design problem.

Conclusion

New developers often commit the mistake of jumping directly to the implementation phase. It is always a prudent decision to first write the requirements using user stories, use cases, and any other tool you find interesting, then define a rough conceptual model and finally jump to create a class diagram. This way, apart from the attributes that the class possesses, we know what kind of behaviors it should have. Once the class diagrams are drawn, it becomes easy to transform them into code. The chapter discussed several methodologies and styles to write user cases user stories, and for drawing class diagrams. These tools can be used by system designers to perform analysis and design of an application. Once we have a class diagram in place, we can transform it easily into the code using any of the favorite object-oriented languages.

Points to remember

- Object-oriented analysis is about figuring out what to do, and object-oriented design is about how to do it.
- In the analysis phase of system design, we should gather requirements and list them using tools like user stories or use cases.
- In the design phase of system design, we should leverage information and requirements gathered in the analysis phase by transforming them into class diagrams.
- Class diagrams are UML-based tools that help a designer to write code during the implementation phase.
- We can list out the nouns in the use case and user story to find the potential objects in our system.
- We can list out the verb phrase in the use case and user story to find the relationships between two objects.
- Inheritance, aggregation, and composition are different types of relationships that exist between two objects of a system.

Multiple choice questions

1. **Which of the following is true for the analysis phase of system design?**
    a. In analysis phase, a developer is supposed to write code for the whole application.
    b. In analysis phase, one has to gather the requirements of the system.
    c. In analysis phase, a developer has to decide which language to use for implementation.
    d. In analysis phase, objects and classes are supposed to be identified.
2. **Which of the following is false for an inheritance?**
    a. Inheritance implies is a relationship between two objects.
    b. Inheritance promotes code-reusability.
    c. Inheritance implies has a relationship between two objects.
    d. Inheritance implies if the base class object is destroyed, then the derived class object is also destroyed.

Answers

1. **b**
2. **c**

Questions

1. What is the meaning of the analysis and design phase of system design?
2. What are the differences between inheritance, composition, and aggregation?

Key terms

- UML stands for Unified Modeling Language, and it is used to model the system design.

- An abstract class is a class with one or more methods that are simply declared and not defined.
- Public, private, and protected are keywords in object-oriented languages used to define the accessibility and visibility of classes, attributes, and behaviors.

CHAPTER 8

Approaching System Design Problems

Introduction

In the previous chapters, we have seen various methodologies, techniques, and tools that help during the system design. However, learning all these things is of no use if we cannot put them to use in a systematic and clean manner. The aim of this chapter is to pave the way for system designers to use all this stuff in a sophisticated way and to learn how we can approach a system design problem thrown at us, be it in some interviews or in real life. We will take a sample problem and perform analysis and design and model system using class diagrams. We will do some back-of-the-envelope calculations, and we will see how to choose between relational and non-relational databases and how to scale the system. The chapter will act as a guideline by providing us with a series of steps that will be helpful enough to build confidence to approach any system design problem. So, subsequent chapters can essentially follow the same steps to design the system.

Structure

In this chapter, we will cover how to approach a system design question with the help of an example.

Objectives

After studying this unit, you will be able to understand how to approach a system design problem. You would be able to decide how to choose between relational and non-relational databases. Apart from this, you will also learn how to scale the system so that it works perfectly for a large number of users.

Getting started

To begin with, one must note that system design is an art that comes with a lot of practice. There is no single solution to a particular system design problem. There could be many different ways to design a system. Because the nature of system design problems is subjective, it is imperative for a system designer to spend some time learning how other products are built. It helps shape the judgment so that a system designer can take the best route for designing the system.

Requirement gathering

Let us say you are given the following problem statement: design a tiny URL.

Readers must be aware of the fact that problem statements given during the interview or by the client can be vague. It is easy to fall into this trap and just start off with the statement itself. It is easy, as said earlier, but it is incorrect. One should always ask clarifying questions as follows:

- Who is going to use the system?
- What is the expected input and output to the system?
- How many requests we are expecting the system to handle?
- Do we need this system to be available 24×7?

And so on. Such follow-up questions are always needed to figure out what exactly we need to design, and it also helps to define the scope of the problem. Coming back to our problem statement, let us define the system requirements. We have seen in the previous chapter that we can define the functional and non-functional requirements of the system. So, let us define some functional and non-functional requirements for our problem statement as well.

Functional requirements:

- The system should take an input URL and generate a small or tiny URL as output.
- Whenever a user clicks on a tiny URL, he/she must be redirected to the original URL.
- The user should be able to copy and paste the tiny URL.

Non-functional requirements:

- The system should be highly available.
- The system should be able to redirect to the original URL with minimal latency.
- The system can expose some API through so that other systems or applications can easily use it.

Once the requirements are defined, let us write some use cases. One must observe that the primary actor for this system is going to be a client or user who needs a tiny URL for a big URL.

Title: Generate a tiny URL

Primary actor: User

Success scenario:

- User gives a URL to the system which generates a tiny URL for the user.
- User can copy and paste this URL anywhere he wants.

Title: Redirect tiny URL to original URL

Primary actor: User

Success scenario:

- User clicks on the tiny URL.
- System redirects the user to the original URL.

Title: Redirect tiny URL to original URL

Primary actor: User

Success scenario:

- User clicks on the tiny URL.

Back of the envelope calculations

A good system designer should be aware of Back-of-the-envelope calculations. These calculations help to choose the best design, database, and components for the application. To understand what it exactly means, let us dissect our problem statement. Our system is going to be a read-heavy system; that is, most of the queries to the system will be of URL redirection from a tiny URL to the original URL.

Now, let us assume that our URL shortener application is going to receive 300 million URL shortening requests. Because our system is read-heavy, let us assume that read to write ratio (read requests/ write requests) is 200:1.

URL redirection request (redirect tiny URL to the original URL) would be:

200 × 300 million = 60 billion

Queries per second for our application will be:

300 million/(30 days × 24 hours × 3,600 seconds) = approximately 116 URL shortening per second

URL redirection requests per second will be:

200 × 116 = 23,200 URL redirection per second

Now, let us discuss about the storage requirements. Consider, we plan to store every request to generate a tiny URL for 5 years in our database. So, the total number of objects we will end up storing will be:

300 million × 5 years X 12 months = 18 billion

Let us assume we need 500 bytes to store each object. So, the total storage needed per month will be:

18 billion × 500 bytes = 9 TB

Now, we have done back-of-the-envelope calculations. As we studied in the previous chapter, it is always good to define a conceptual model of the system. The key to drawing one is to identify the nouns which may qualify as the system actors and verb phrases that define the relationship between these nouns.

The nouns in the use cases are User, System, and URL. However, URL is more like a result of an action that the user triggers. So, we can create an overly-simplified conceptual model in the following way:

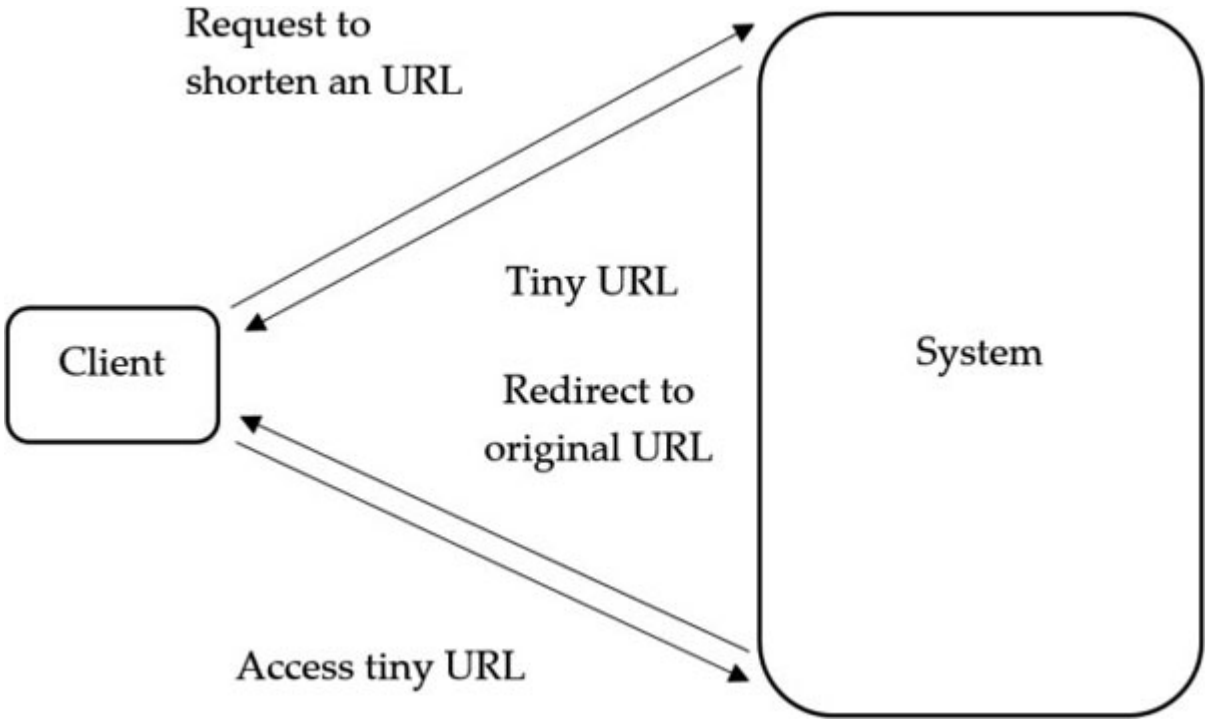


Figure 8.1: *Conceptual diagram of tiny URL*

Scaling the design

As mentioned earlier, the system we designed is just a very simplified version, and it is not at all practical to design a system like this in the real world. The arrangement may work for a very limited number of users, but it is not scalable. Let us move on to the scalability aspect of the system. We have seen in previous chapters that there are many technologies available to a system designer, such as cache, load balancer, and so on. Let us see how we can use those technologies.

We are good with such a naïve system when users are saying, 10, 20, or maybe 30, but if say, the number of users accessing the system are of order millions, then, needless to say, one single instance of the system is not enough to handle all the requests. It is good to have several instances of applications up and running on different servers. To distribute the load evenly across different servers, we can use a load balancer. Also, we would need databases to store the actual and tiny URLs. Furthermore, our system is read-heavy. So, querying the database, again and again, can degrade the turnaround time. So, it would be a better idea to cache the frequently accessed tiny URL so that every time a user clicks on a tiny URL, we can

quickly retrieve the original URL from the cache instead of the database. Now, based on the things we just discussed; we can improvise the conceptual diagram in the following manner:

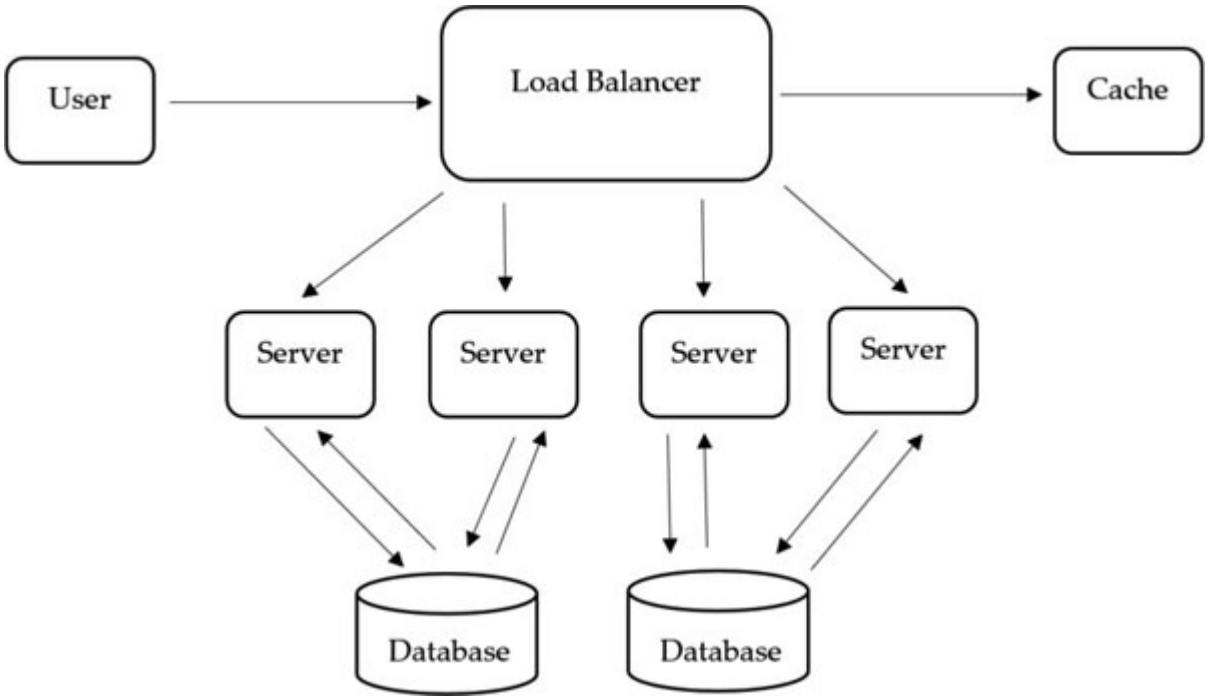


Figure 8.2: *Scaling the tiny URL system*

The preceding conceptual diagram makes more sense. It is capable of handling several user requests at a single point in time. We have placed a cache to reduce the turnaround time, and our persistent layer consists of databases to store URL-related things.

SQL versus NoSQL

The database in our system can be SQL or NoSQL but before we come to that question, let us see how we can store data in the database. We first need to define the schema that we need for our tables. We may consider the following tables for our system:

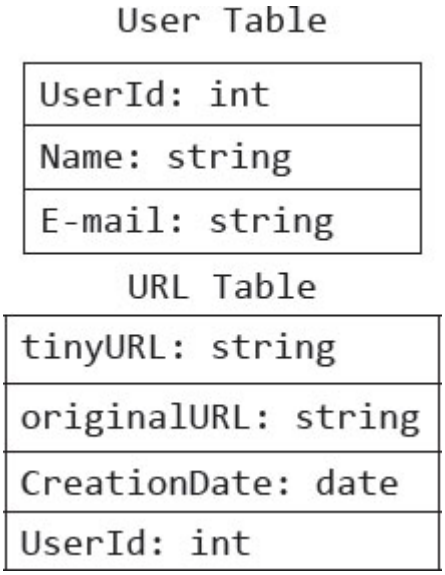


Figure 8.3: *Table schema for tiny URL*

Now, our system is going to store several millions or even billions of URLs. Moreover, we do not need any relation between different rows of our table. Our use case will mostly deal with queries like—getting the original URL from a tiny URL; that is, most of the queries will try to retrieve the value (original URL) from a key (tiny URL). So, a no-SQL database would be a better choice. Readers should also note that SQL databases are vertically scalable; that is, we can add more power to the single server, such as adding more RAM, CPU, and so on. However, NoSQL databases are horizontally scalable. So, we can shard a database, that is, store data across different database servers. So, instead of querying a big large chunk of data, we will be querying a small set of sharded data. It essentially implies a better turnaround time. This is also justifying our choice of NoSQL as our system will deal with a large amount of data. One must ask questions like this to choose between a SQL and NoSQL Database. There are several other parameters one can keep in mind while choosing between a SQL and NoSQL database, like whether we need ACID properties or whether the schema is fixed or dynamic. ACID properties stand for Atomicity (transaction is atomic, so it is either successfully executed or fails), Consistency (transaction brings database from one valid state to another), Isolation (parallel run of the transaction will bring the database in the same state if the transactions are run sequentially) and Durability (a transaction once committed will remain in the database even if the database goes to a failed or crashed state).

Identifying the algorithm

We are now at a place where we have identified the requirements, conceptualized the system diagrammatically, made a few calculations, and estimated the database requirements. However, the system will work with the right algorithm and logic. So, let us see a few approaches to solve the actual problem—generate a tiny URL for a given URL.

Approach 1: Encode the original URL.

We can feed our actual URL to a hash function, say MD5. The hash function will provide us with a hashed value. Then, we will convert this hashed value to base 62 because we have assumed that our tiny URL is going to contain characters from the set [a-z] [A-Z] [0-9]. Assuming we have decided that our tiny URL is going to be eight characters long, but if we use the MD5 hashing function, then hashed value after conversion to base 62 is definitely going to be more than eight characters. We can choose the first eight characters from the base 62 string. However, the disadvantage of this approach would be collision because different URLs, when hashed and converted to base 62 string, may have the same first eight characters. To circumvent this problem, we can append either user id or timestamp to the original URL and then feed it to the hash function. Once it is done, we can convert it to base 62 and extract the first eight characters. We can additionally put a check to see if a similar tiny URL already exists in our database or not. If a tiny URL already exists, we can generate the tiny URL again. We can keep doing it until we get a unique tiny URL. This approach has some disadvantages as follows:

- Choosing a good hash function that ensures a random URL along with minimum collision is a difficult task.
- The turnaround time of the system may increase if the system is not able to generate a tiny URL in one go.

Approach 2: Generate a random tiny URL.

We can use a random eight-character key generator service to generate a tiny URL. Whenever there is a request to generate a tiny URL, we can call the random key generator to retrieve a tiny URL. In order to make things fast and avoid any concurrency issues later, we can generate all the possible eight-character keys offline beforehand and put them in memory or

database. So, in the future, whenever there is a request to shorten an URL, we can query the database to get a fresh tiny URL. We have to make sure that once the tiny URL is assigned to an URL, it must be marked so that it cannot be used again. To make things simple, we can maintain two tables—one for maintaining all the unused tiny URLs and one for the used tiny URLs. The moment we pair a tiny URL with a URL, we will remove it from the unused table and add it to the used table. We can also place a cache memory to avoid querying the database for a fresh tiny URL. There is an issue with this approach that two instances of the service may pick the same unused URL from the database. In order to ensure it does not happen, we can use database locks.

Approach 3: Counter-based approach.

We have decided to use eight characters long tiny URL for our system. We can map all the possible eight-character strings to an integer value lying in a range, say, 0 – *n*. Now, we can maintain a counter so that any time a user asks for a tiny URL, we will return the current counter value and increment the counter. For example, if the current counter value is 0, it means our tiny URL is going to be aaaaaaaa, and we will return the counter value and increment the counter to 1. So, next time a new request comes, we will return 1, and a tiny URL can be aaaaaaab, and so on. We can diagrammatically represent the counter-based approach in the following manner:

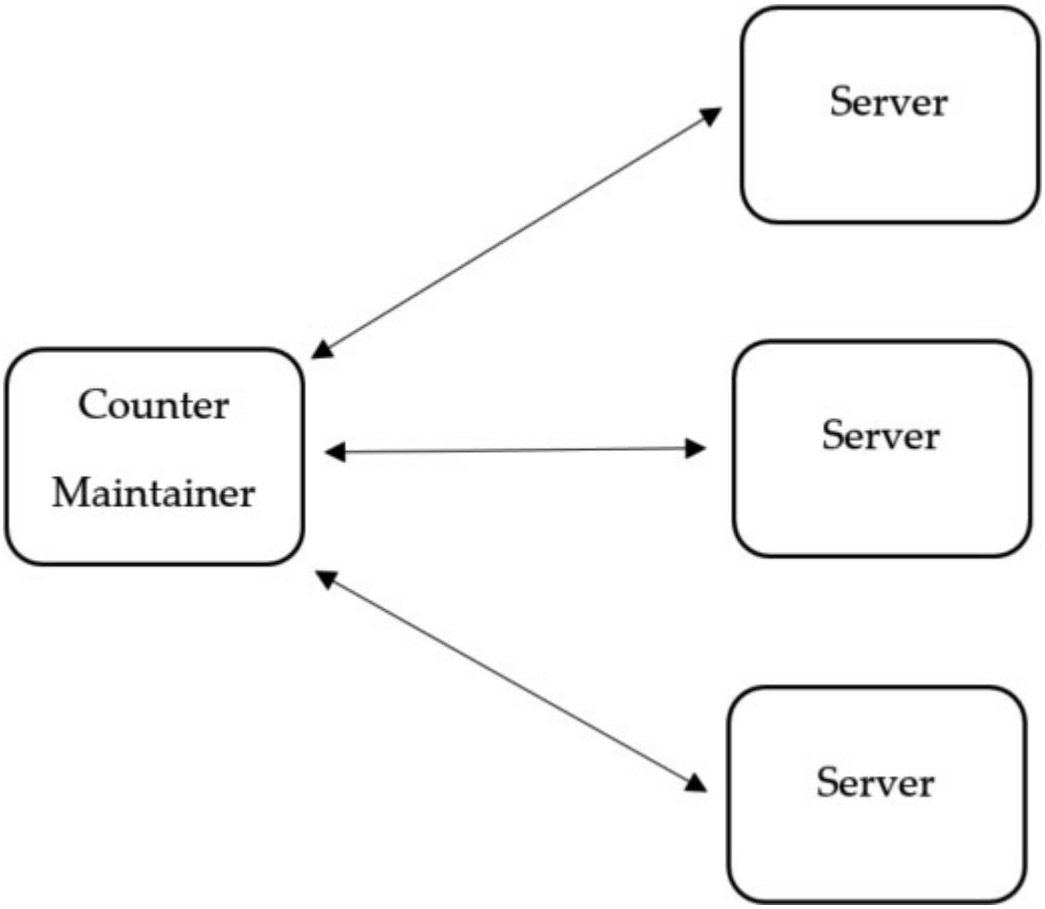


Figure 8.4: *Counter-based approach for tiny URL system*

There are some problems with this approach. First, tiny URLs are predictable because they are generated in a sequential manner. It could pose a security threat. To make tiny URLs more random, we can modify the mapping between the value returned by the counter and the tiny URL. Say, we will map a six-character string to a counter value, and the rest of the two characters can be random in our tiny URL. The other disadvantage of this approach is that counter maintainer is a single point of failure. We are giving too much power to a single machine which could be deleterious. To circumvent this problem, we can have multiple counter maintainer machines. To ease the functionality, we will divide the range and assign it to different counter maintainer machines. Say, counter Maintainer 1 works with range 0–1 million, counter Maintainer 2 works with range 1,000,001–2 million, and so on.

We can also keep a load balancer between these counter maintainer machines and server to avoid stressing out a particular machine in the

following manner:

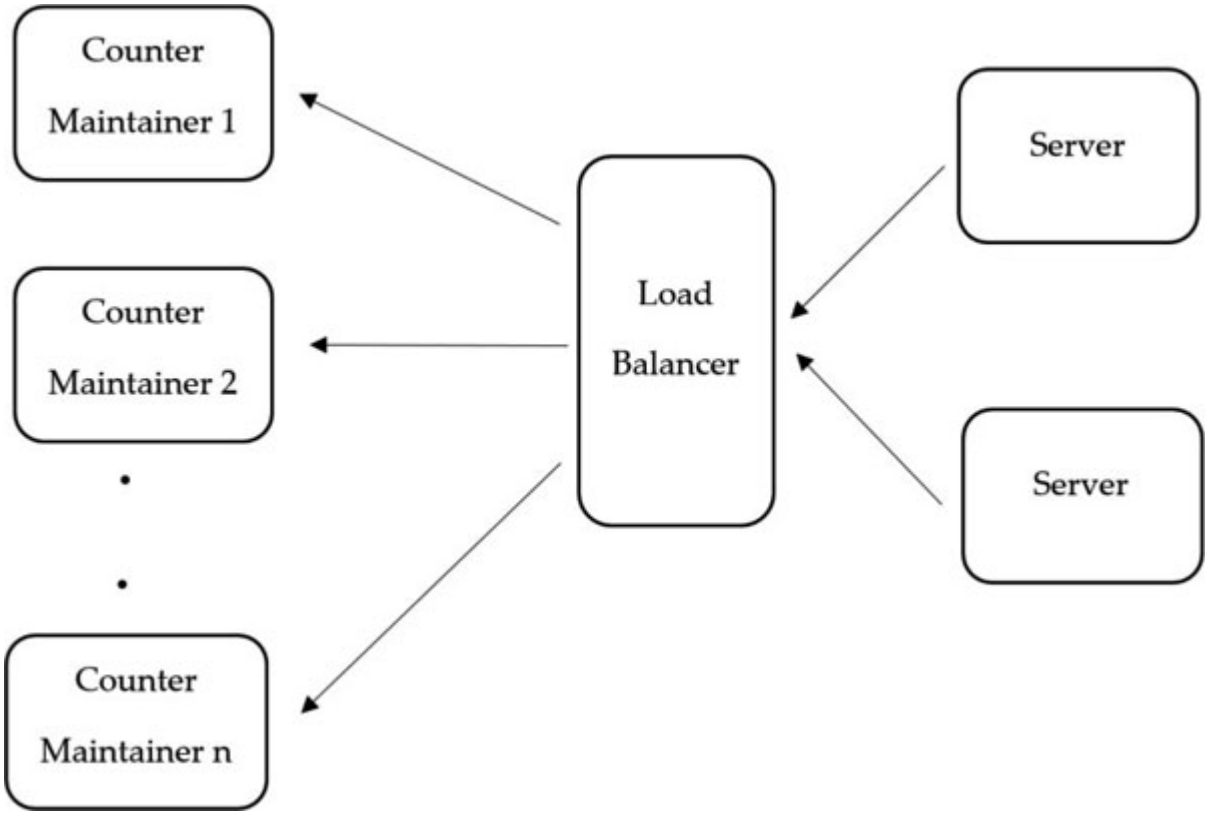


Figure 8.5: *Multiple-counter-based approach for tiny URL system*

This arrangement offers some advantages. First, there is no single pointer of failure. Second, each counter maintainer works with its own range. So, at any point in time, it is quite difficult to predict which Maintainer machine will be serving the request; hence, a tiny URL will be random.

Partitioning the data

System design discussions are often quite elaborate, time-consuming, and follow no particular direction. Each system is fairly different from the other. For example, if we are working with the banking system, then security is a must. However, if we are discussing a tiny URL system, then security might not be a hot topic during the discussion. So, one must understand the key aspect of the system and then try to drive the discussion in the direction that serves utmost to the system in the picture. As far as tiny URL system is concerned, we must work on the aspects like how to scale a database and how to maintain the cache.

Readers must realize the fact that our tiny URL system is going to store several billions of URLs in a short span of time. So, there is a need to scale the database as well. We can employ the following methods to scale the database:

Partition the data based on the range: In this method, we will take, say, the first two letters of the tiny URL. Then we store all the URLs having the same first two letters in one table. We may also place less frequently accessed URLs in one table. The main disadvantage of range-based partitioning could be that we might have too many URLs with the same first two letters, and in that case, it leads to unbalanced division.

Hash-based partitioning scheme: In the approach, we will compute the hash value of the object to be stored. The hash value will point us to the partition where we need to store the object. Now, whenever there is a query to retrieve the object, we can again compute the hash, which will point us to the data block storing our object so that we can quickly retrieve the object. We should employ the hash function that will distribute the object randomly and uniformly across several partitions so as to avoid any overloaded partition. Readers must note that hash-based partitioning schemes may suffer if the database servers are added or removed because in those cases, we have to change the hash function so that it can ensure the uniform distribution of the data. Let us see in detail how this can cause the problem and how it can be solved.

A hash table is a data structure to store key-value pairs. In order to store a key-value pair, we provide the hash table with a key, and the hash table uses its hash function to compute an integer value. This integer value points to the index in the hash table where this key-value pair will be stored. Now, the hash function has to ensure a uniform distribution of value, and they do not spit out a value that points to an index in the hash table directly. Typically, the hash function outputs a value, say val, which is recomputed again with val % N, where N denotes the number of buckets or entries in the hash table. The result of val%N then points to an entry in the hash table. Each entry may point to a link list as multiple results of val%N can result in the same value. Now, if the hash table is distributed, then it will be split, and different parts will be stored in different servers. So, for a distributed hash table, key-value pairs are stored in different servers.

Now, how does the distribution of key-value pairs occur? We can use the previous strategy with little modification. We will first feed the key to the hash function and say the result is val. Then we will perform val%N, where N denotes the number of servers. So, we will have the server where this key-value pair will be stored. Once we have the server, we will perform the same steps as mentioned earlier to compute the hash table index. Readers should note that the hash function for each server must be the same. Let us assume we have four servers. So, we will perform hash(key)%4 to get the server number. Say one of the servers crashed. So, our computation formula for a server will change to hash(key) % 3. Readers must observe that if this happens, then we will have to recompute this for every key-value pair, and everything may move to an entirely different server altogether. This will be a huge blow to the system's performance and may even crash the remaining servers. In order to solve this problem, we use a consistent hashing scheme. It is a distribution scheme that does not depend directly on the number of servers; thus, ensuring that the addition and deletion of the servers minimize the number of key-value pairs that need to be relocated. Consistent hashing assigns the object a position on an abstract circle or hash ring; that is, it maps the hash output range on the circle's edge. So, the minimum possible hash value would be 0, which corresponds to angle 0, and the maximum possible value, which corresponds to an angle of 360 degree. All the remaining hash values will fit between these two angles only. So, the working is simple: compute the hash value of the key, and assign it a position on the edge of a circle as follows:

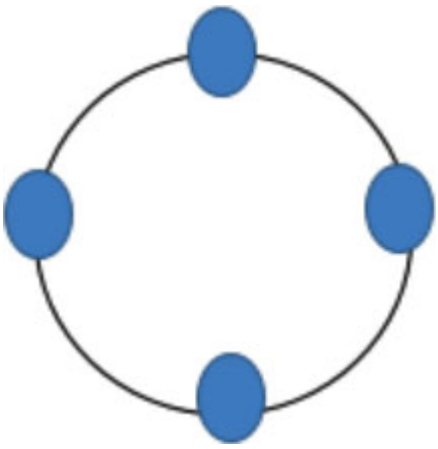


Figure 8.6: *Consistent Hashing - basics*

Now, let us say we assign the position to the servers in the same circle. So, the representation will look as follows:

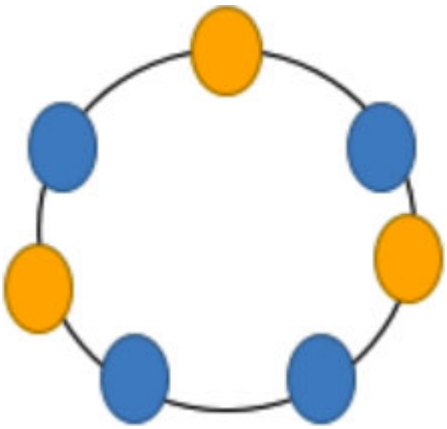


Figure 8.7: *Consistent Hashing – objects & servers*

Here, we have four objects (blue circle) and three servers (yellow circle). Now, to associate the object to the server, we will move in anti-clockwise order from the object and put the object in the first server we encounter. In our example, it would look as follows:

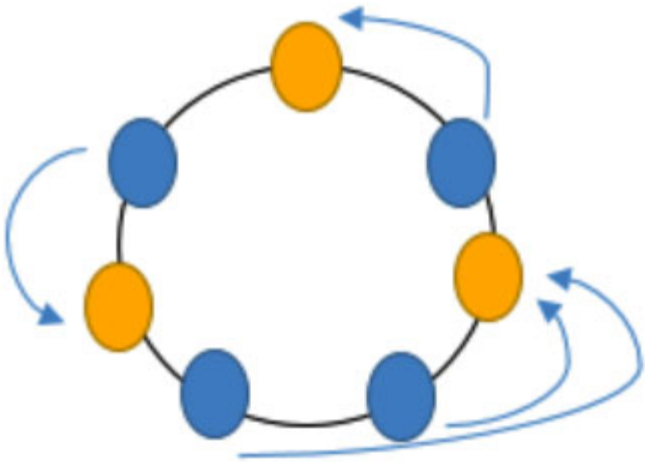


Figure 8.8: *Consistent Hashing – object & servers association*

Programmatically, we can keep a sorted list of angles that correspond to the servers or objects and find the nearest server for an object; we can perform a linear or binary search to find the first server with an angle value greater than the object's angle value. Now, we have to ensure uniform distribution of load to each server. In order to achieve this, we will assign "*n*" labels or angles to each server based on the load a server can handle. If a server is twice as powerful as compared with the other servers, then it will be

assigned $2n$ labels or angles. So, let us assume we choose $n = 4$, then each server will have four angles (considering all the servers are the same). We will name our three servers as A, B, and C, and they will be represented in the following way:

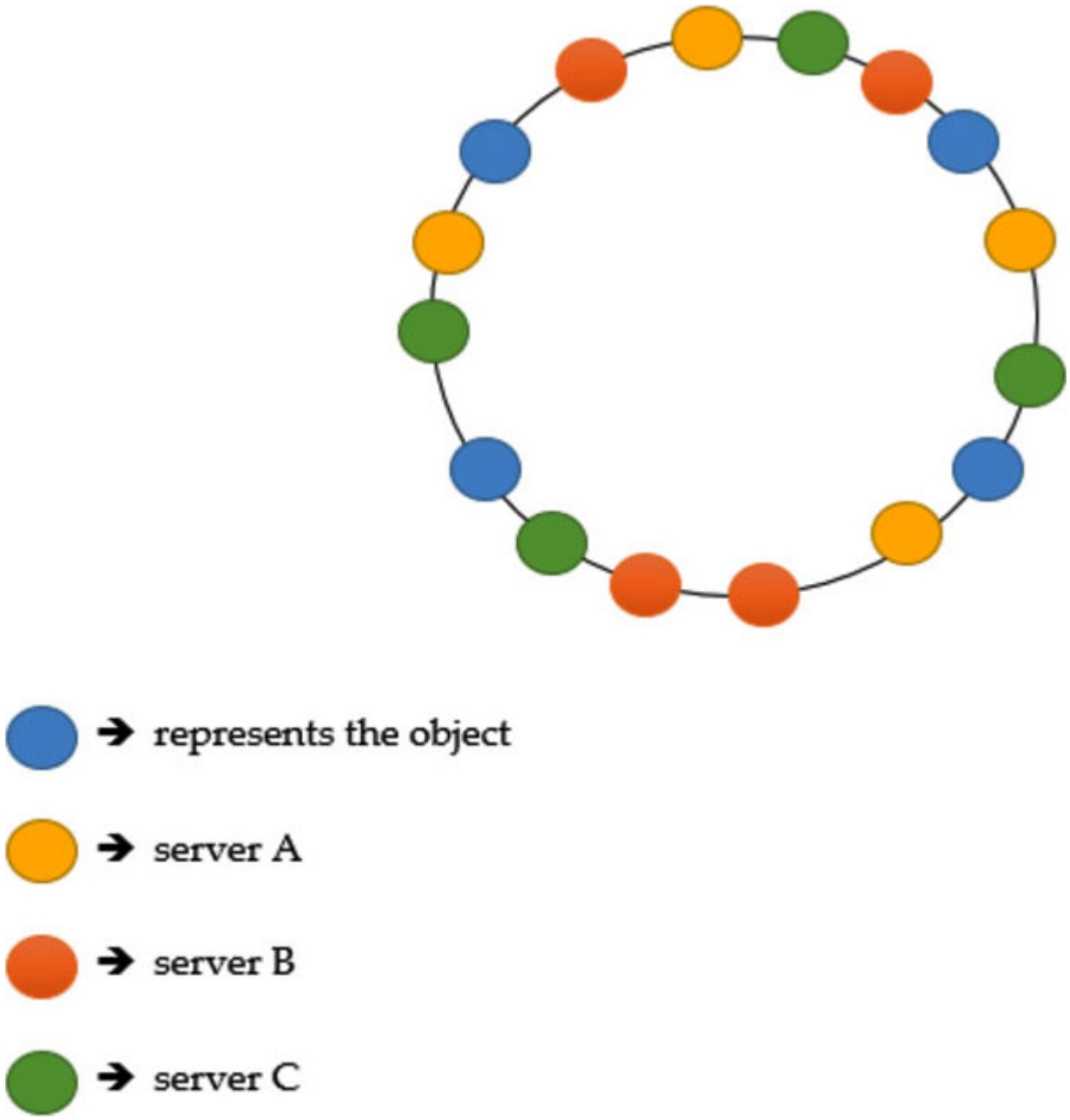


Figure 8.9: *Consistent Hashing*

Now, what is the benefit of using this consistent hashing technique? Imagine we remove Server C from this hash ring. In that case, we only have to change the mapping of objects preceding the Server C only, and all other

objects will remain unchanged in their mapping. The same thing we can imagine if a new server is added, In that case, the object preceding the new server will have to be remapped, and all other objects will be untouched. That is the beauty and advantage of consistent hashing.

Cache

We have discussed earlier about placing cache memory between the server and database. The intention was to speed up the retrieval time of frequently accessed tiny URLs by placing them in the cache. However, cache memory has less storage capacity as compared to the database. So, at one point in time, our cache memory will be full, and we need to eliminate some of the existing entries. Now, we must think about proper eviction strategies as well. In order to ensure the optimal usage of cache memory, we can place the most frequently used URLs in the cache. We can use the Least Recently Used policy for our cache. In this policy, we eliminate the URL which is least recently used. We can use the heap-based data structure, where we keep the last accessed URL at the root. In this manner, the URL, which is used least recently, will be at the last node of the heap. Now, whenever the cache miss occurs, the server has to query the database. We can keep distributed cache servers as well. These caches will be located in different geographical locations, with close proximity to the user. There can also exist a load balancer between the server and cache.

Database cleanup

The tiny URLs created are not going to be alive forever. They must have some expiration date, say 3 years or any other defined by the user. Now, the question is, what do with the expired URLs? It is obvious that expired URLs need to be removed. However, keeping track of when a particular URL is going to expire and then removing it from the database is going to be expensive. Instead of that, we can use the lazy cleanup technique. The lazy cleanup service will run periodically, and it will remove all the expired URLs in the database and cache. There are a few things we can do additionally:

- When a user accesses an expired URL, it must throw an error and delete all the entries of the expired URL.

- We can trigger a cleanup service periodically when traffic is very low to remove all the expired URLs.
- It must be ensured that URLs deleted after expiration are put to use again.

Now, we can improvise the system high-level design based on our discussion in the following manner:

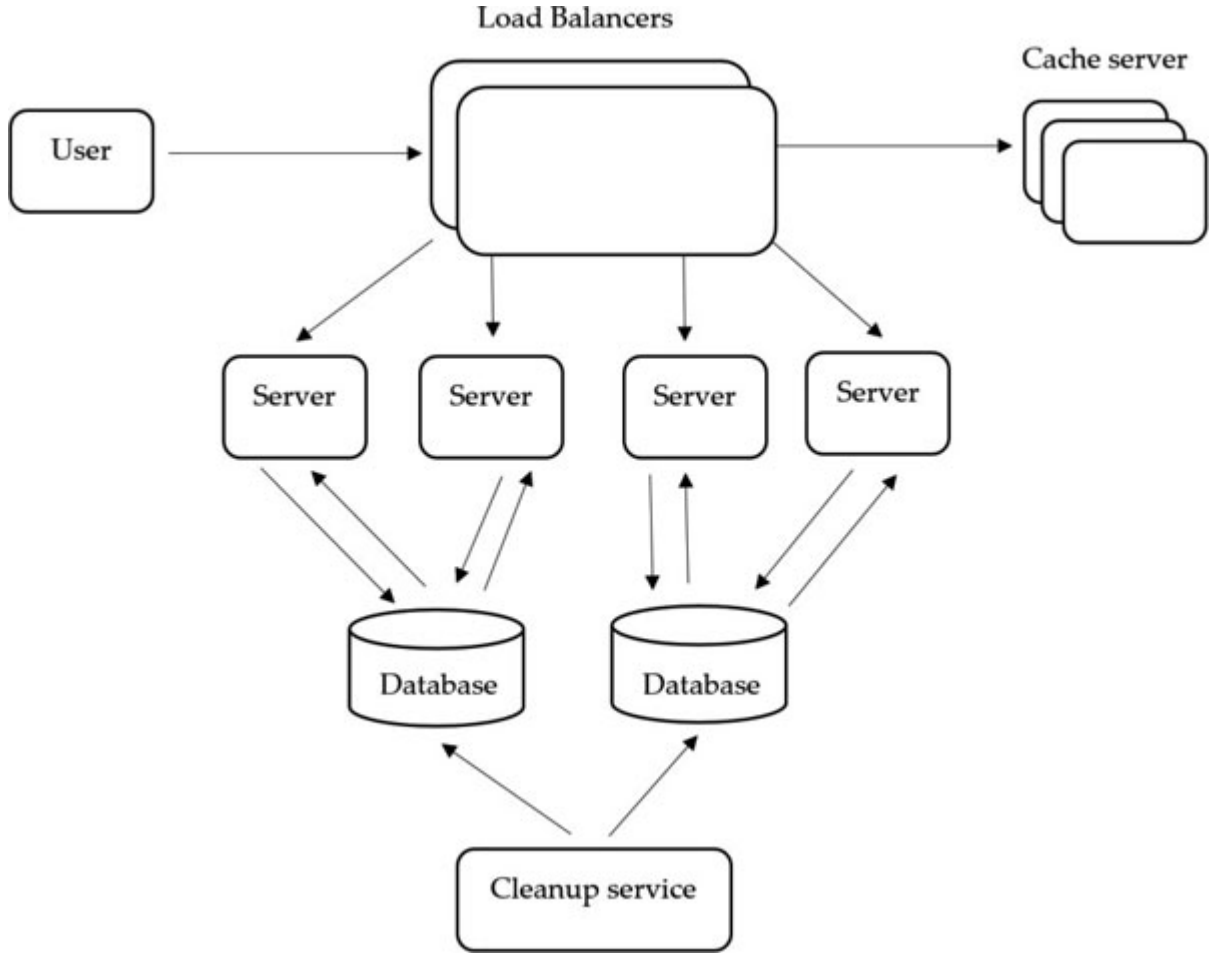


Figure 8.10: *Tiny URL system*

Apart from all the preceding discussions, we can keep some statistical data to better serve users. Which URLs are getting accessed very frequently, geographical location with a significant amount of traffic, the time when user traffic is very high and very less, and so on. All this data can help us to visualize better where we can put technologies like cache and load balancer and when we can schedule our run for clean-up service.

If we have a use case where a URL can be accessed by a limited number of users only, then we need to have a database where we store rows like URL and user id, which essentially implies that the user id has access to the URL. So, if any unauthorized access to the URL is made, it will throw an access denied error.

Conclusion

The initial stage of system design is often very struggling because the requirements are not clear. A designer must begin with identifying requirements. The initial step is to get clarity on what we need. Once requirements are clear, one should write use cases or user stories as it helps in identifying major actors of the system and the relationship between them. We can draw conceptual diagrams to pictorially visualize the system. Then, we can perform back-of-the-envelope calculations in order to figure out the expected user traffic our application can expect. Additionally, back-of-the-envelope calculations will generate approximate storage requirements numbers, and it will help us to take a prudent decision in choosing between SQL and NoSQL databases as well. Once we have a rough sketch of the system, we can work on scaling it so that it works even for a large number of users. At this point, we may work on adding various technologies such as a load balancer, cache, and so on. One must note that almost every system needs a database to work, so we must encounter a question when we have to choose between a SQL and NoSQL database. Choosing either of them is dependent on various parameters like how much data we are going to store, whether there is fixed schema or not, and so on. To summarize, all the discussions will shape the final view of the system.

Points to remember

- System design problem statements are intentionally vague. So, first, requirements must be gathered and written using tools like user stories and use cases.
- After gathering all the requirements, a system designer should identify the key actors in the system and then identify the relationship between them.

- Every system is unique, and it is up to the different stakeholders of the project to identify the areas in which system designing discussion should be driven. So, for some, system memory would be the key thing, and for some, security would be a must to have.

Multiple choice questions

1. **Which of the following is true for cache?**
    a. Generally, it has slower access time than databases.
    b. It is faster to access cache memory than accessing database.
    c. Cache is used to distribute traffic evenly across different servers.
    d. Cache is a database scaling technique.
2. **Which of the following is false for SQL database?**
    a. They generally follow ACID properties.
    b. They have well-defined schema.
    c. They have dynamic schema.
    d. They can be scaled vertically.

Answers

1. a
2. c

Questions

1. What is the difference between SQL and NoSQL database?
2. What is a load balancer, and why it is used?

Key terms

- Load balancer is used to evenly distribute traffic across different server nodes.

- Cache is a high-speed memory placed between a server and database to reduce the time to access the data.

CHAPTER 9

Design a Key-Value Data Store

Introduction

A key-value data store is a database where each key is associated with one value in a collection. In general, the query language is not required for a key-value data store. It supports operations such as get, insert, and delete. Its simplistic nature makes it easy to use, scalable, and flexible. The chapter is designed to discuss how a scalable key-value store can be designed. Different technologies and methodologies are discussed to create a scalable key-value store.

Structure

In this chapter, we will cover the following topics:

- Defining the system requirements
- Creating a scalable system

Objectives

In this chapter, we will be discussing how to design a scalable key-value data store. We will list out the system requirements and then discuss how different technologies can be put in use to scale the system. The main intention here is for the readers to understand the approach for system designing questions and, at the same time to understand how this approach and different technologies can be put in use while doing a system design for any real-world use case.

Defining the system requirements

A key-value store is synonymous with the key-value distributed database where we store a key-value pair. In order to retrieve the value, we supply

the database a key, and it extracts out the key-value pair for us. The key-value database is a non-relational database, and key and value can be anything from a simple primitive data type like an integer to a complex object. A good key-value data store is one that possesses these characteristics:

- **Durable**: We cannot lose any data unless it is deleted by the user itself.
- **Available**: The key-value data store must be highly available.
- **Performance**: Latency must be minimum.

Based on the above discussions, let us define the functional and non-functional requirements for the key-value store.

Functional requirements:

- Store a ke1y-value pair.
    - Update value if key-value pair already exists in the database.
- Retrieve the value corresponding to a key.
- Delete the key-value pair.

Non-functional requirements:

- The system should be highly available.
- The system must return consistent results.
- The system must be durable so as to avoid any data loss during an emergency.
- The system should have minimum latency.

Now, let us write the user stories:

Title: Store a key-value pair

Primary actor: User

Success scenario:

User provides a key-value pair to store in the database.

If the key-value pair already exists then the value is updated in the database.

Title: Retrieve the value from a key

Primary actor: User

Success scenario:

- User provides a key and the system returns the value corresponding to it.

Title: Delete the key-value pair

Primary actor: User

Success scenario:

- User provides a key and the system deletes the corresponding key-value pair.

Back-of-the-envelope calculations

We have written user stories. Let us move on to the part where we can roughly estimate the constraints on the system and make some assumptions. The one point readers must note is that during system design discussion, it is always encouraged to discuss things, such as the amount of data we need to store, queries per second, expected system traffic, and so on. For our case, we are making a few assumptions that:

- The system will store around 100 TB of data.
- The maximum size of value can be 1 GB.
- Queries per second can be of order 10,000.

Assuming that each machine can hold 5 TB of data, we would need (100/5) = 20 machines. However, actual machine numbers can be more considering we might ensure data redundancy.

Creating a scalable system

Consistency or availability

It is always nice to finalize the system design goals based on what we expect from our system. This is where we will use CAP Theorem. We have read in previous chapters that a distributed system can be either CA, AP, or CP. However, AC systems are practically non-existent as network failures are bound to occur. Sao, we are left with CP or AP. To choose between availability and consistency, we will choose consistency here because it is okay to serve with correct data rather than supplying incorrect data.

High-level design

A very trivial way to store that data in our system would be to dump the entire data in one single database. We can have a client, server, and database. The client will send a request to the server to say store a key-value pair, and the server will put that in the database. In that case, we can roughly visualize the interaction of the client with the database in the following manner:

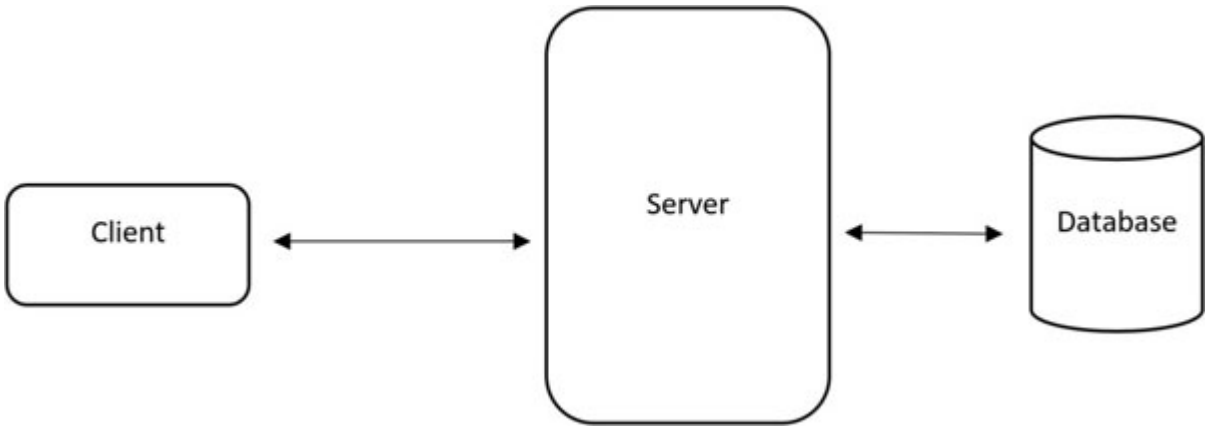


Figure 9.1: *A trivial high-level design of the key-value store*

Scaling the database

We can clearly see some problems with this architecture. First, keeping all the data in one single database can be detrimental to the system's performance. Second, we are dealing with a massive amount of data. Also, our schema is dynamic so using a NoSQL database makes more sense. We can use database sharding in this case. We can horizontally partition the data and store them in different databases. This way, we will be distributing the load, so data lookup will be fast.

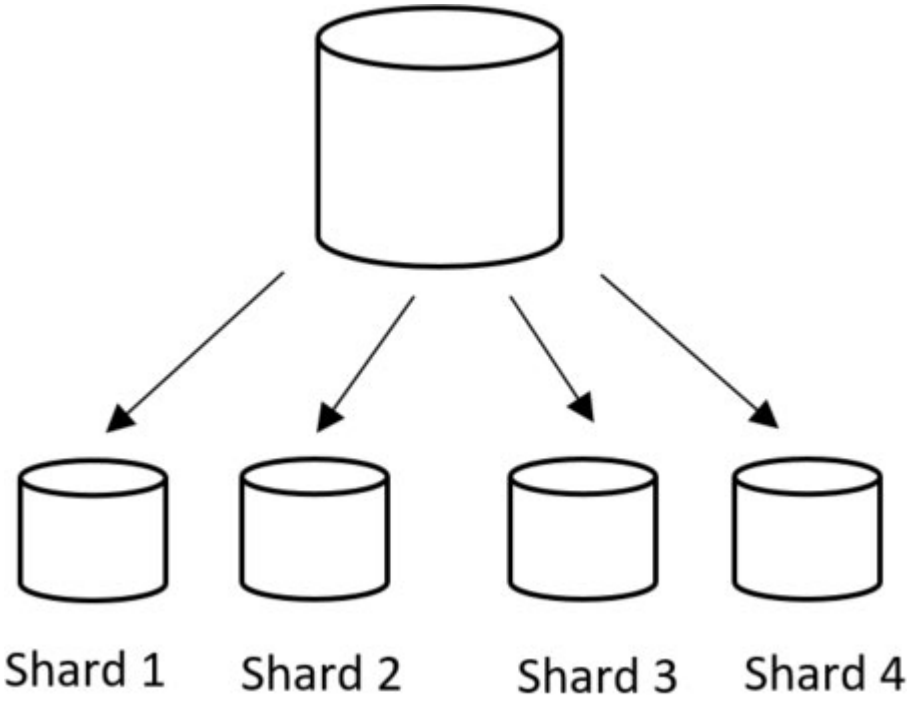


Figure 9.2: *Database sharding*

Durability and consistency

Now, we must answer the question, *Is it okay to keep one copy of data?* Keeping a single copy of data can have issues if one of the shards goes down or gets corrupted due to any reason. In that case, we will lose the only copy of data, and losing data is generally not a good idea if it is critical to business. So, data replication must be done. There is more to this. We must ensure that the copy of the data kept is up to date. We should not keep any stale data in our database. To ensure this, we can employ different strategies like:

Let us say we keep four copies of the data in different machines. Now, it is very unlikely that all four databases will go down or get corrupted. To solve the problem of consistency, we will ensure that a particular database writer operation is successful only if it is written to every machine. The downside of this approach is that database write operation will become slow.

We can improve the previous technique with the mast-slave concept. We can have a master machine, and every read and write request passes through it. Whenever there is a write operation, we can declare the write operation as a success even if the data is written in a single machine. The master

machine will keep track of which machine has the consistent data. Now, for every read operation, the master will ensure (using the same information only) that the read will get the latest write only. We can run a background job when the load on the system is low to ensure that all the machines carry the latest data.

The preceding technique still possesses some problems. What if the master dies or gets corrupted due to some reason? In that case, we would lose some crucial data. To counter this, we can keep a standby master, but the standby master still needs to be updated so that it can take over in case the active master fails. Now, to keep the standby master up-to-date, we can maintain a shared network file system. Whenever there is any insert/edit/delete operation, then the active master will log entry in the file present in this shared directory. This shared file would then be used by the standby master to keep itself up-to-date. Now, if a failure occurs, then the standby master can become the active master in a minimum time frame.

The following figure is a visual representation of the active and standby master-concept we just discussed. Both the master interacts with the shared file system. Active master writes a log file in the shared file system, and the standby master uses this log to keep itself up-to-date.

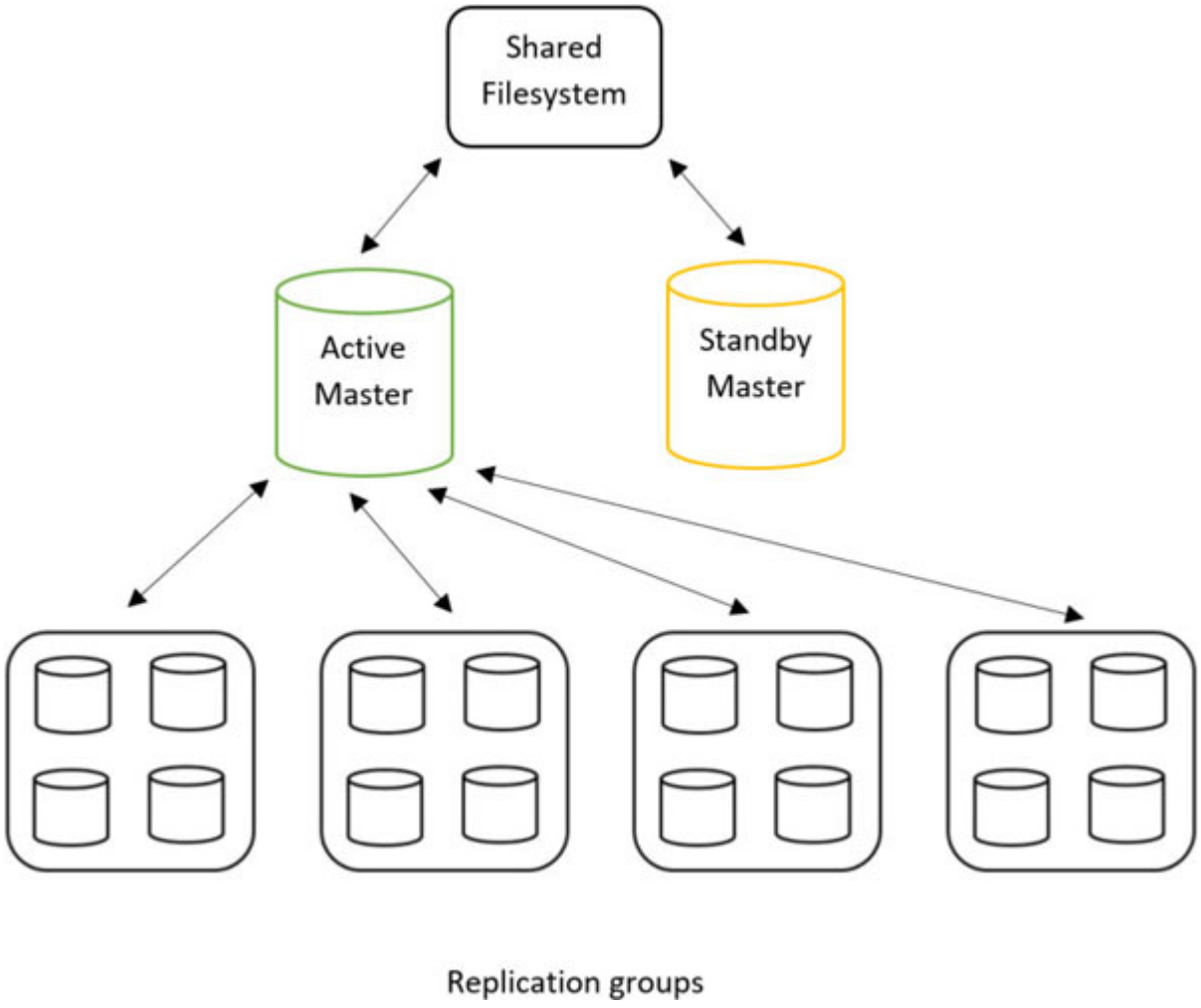


Figure 9.3: *Master-slave and replication group concept for data durability*

We must also note that there are four replication groups in the above arrangement. Inside each replication group, we have four copies of the same data that reside in a particular shard.

Re-defining the high-level design

Our replication groups contain the data which can belong to different tables created by the different users. Whenever a client sends a request, we need the information of which replication group to go for fetching the data. We can introduce a component, say, Replication Group Manager, which will keep this information. The replication group manager will have to elect a leader node in the group. A leader node is a node in which if data is written, then the write operation is considered successful. All the other nodes will ultimately sync with it. It is our choice to decide if we want the write

operation to be successful if data is written to only the leader node or if data is written to the leader and one other node. In our case, we chose the former.

Note: Replication Group Manager can be implemented using Apache Zookeeper, which offers a hierarchical key-value store. It is a service for distributed systems, and it is highly available.

Now, the replication group also has a capacity, and if that limit is exhausted, then they need to be split into two replication groups. This mechanism must be done for the performance aspect as well. We can have a controller service for this that can perform the split.

We will have a service responsible for handling the user's request. In order to maintain high availability, we will have multiple servers hosting these services. A load balancer can be introduced between clients and servers for even distribution of the traffic.

Based on these inputs, we can re-define the high-level design of the system in the following way:

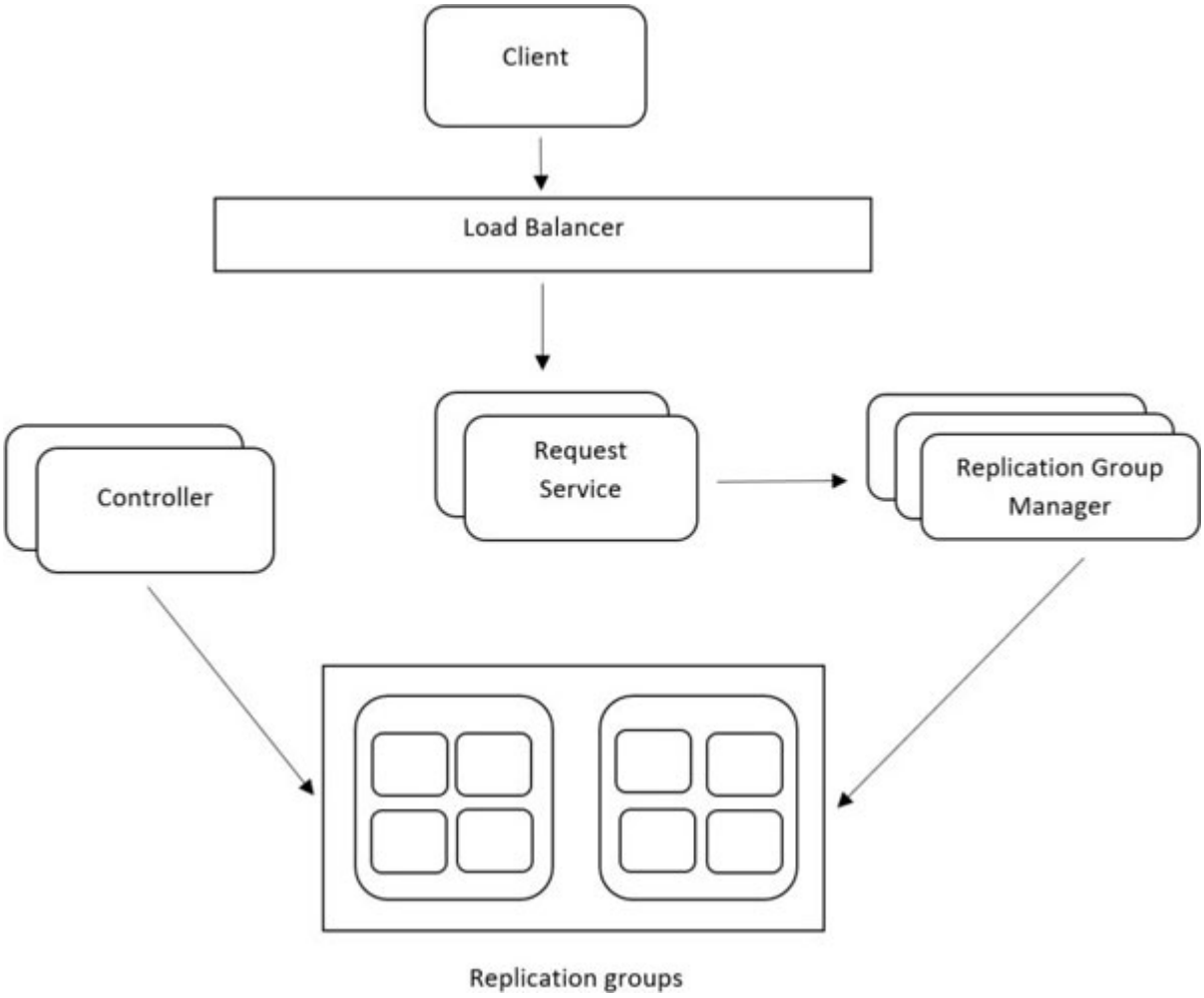


Figure 9.4: *High-level design of key-value datastore*

Conclusion

In this chapter, we discussed how to design a scalable key-value data store. A key-value data store is essentially a no-SQL database. We identified the key steps for any system design question in the previous chapter, and the same steps are followed in this chapter. First, requirements are listed, then back-of-the-envelope calculations are done, and post that, a high-level design is drawn. We identified the core components of the system and then worked on how to scale them. The upcoming chapter will discuss another system design problem, and we will use the same steps to find out the solution.

Points to remember

- CAP Theorem says that any distributed system can have only two out of these three properties: consistency, availability, and partition tolerance.
- Generally, relational databases have fixed schema, whereas non-relational databases have dynamic schema.
- Sharding is a technique in which a large data set is split into smaller parts. It is a useful technique to scale the database.
- Load balancing refers to the process of distributing the traffic across a number of servers so as to make sure that all the servers receive almost the same number of traffic and none of the servers is overloaded as compared to others. It tries to make the overall computing process efficient.

Multiple choice question

1. **Which of the following is generally false for SQL databases?**
    a. They are ACID compliant
    b. They have a fixed schema
    c. They are horizontally scalable
    d. They are dynamic schema

Answer

1. **d**

Questions

1. What is a load balancer?
2. What are different ways for horizontally partitioning the data?
3. What is database sharding?

Key term

- Database sharding refers to the horizontal partition of data in which the table rows are split and kept in different tables.

CHAPTER 10

Design a File Hosting Service

Introduction

This chapter aims to design a file hosting service like Google Drive or Dropbox. File hosting service enables the user to store their documents online on some remote server. It provides the flexibility to access those documents anytime, anywhere, and from any device, given that the user has authentication and authorization for accessing the data. The chapter starts with deciding the system requirements and then proceed toward creating a scalable system.

Structure

In this chapter, we will cover the following topics:

- Defining system requirements
- Creating a scalable system

Objectives

In this chapter, we will be discussing how to design a file hosting service like Google Drive or Dropbox. We will be using our learnings from previous chapters to approach this problem statement.

Defining system requirements

File hosting services are quite popular these days because they provide a very simplified ecosystem to store and access documents digitally, anytime, and anywhere. Such systems are used in almost every organization in some or the other way as the employees are distributed across various geographical locations, and a platform is needed to exchange files and

documents. File hosting service also provides various useful features as follows:

- Multiple copies of data are placed in different data centers. So, it is reliable and durable.
- The user can access/share files anytime/anywhere.
- There is always a scope to increase the storage space of remote servers. So, there are no disk space issues.

Let us define the system requirements:

- The user should be able to upload and download a document or file.
- The user should be able to decide with whom he/she will be sharing the document.
- Once a file is edited by one user, then it must be updated for all the users with whom this file is being shared.

Once we have defined the system requirements, let us try to figure out some design considerations:

The system is going to both read as well as write heavy system.

Before storing the file, we can divide the file into several smaller chunks.

Whenever there is a file edit operation, we can transfer only those chunks which are changed.

If the file upload/download operation fails, then only those chunks will be transferred, which could not be transferred in the earlier attempt. In case the remaining chunks cannot be transferred, then we will invalidate the entire operation. So, the file upload/download operation will be atomic.

A user can make an offline edit, and as soon as he comes online, the changes will be synced, and all the others will be able to see the updated document.

Let us try to do some back-of-the-envelope calculations:

Assume that there will be 5,00,000 users.

Every user will have approximately 400 files.

Let us assume that every file is worth 500 kB.

So, total storage would be 20, 00, 00,000 × 500 kB = 100 TB.

Creating scalable system

In order to draw the high-level design, let us define some user workflows:

- The user will create a workspace folder on their device. He may create several more folders inside it in any manner he wants.
- The same view will be shown in the file-hosting application.
- The user may add/modify/delete files present inside the workspace folder or in the application, and the changes will be reflected to all the users.

Now, before we jump to high-level design, let us focus on some key aspects of the design. It always makes sense to have a modular application with a single responsibility principle. So, based on the same principle, we will have a service with the dedicated responsibility to update the file metadata. The metadata information for a file can be File Name, File Size, path, last modification timestamp, type, users with which it is shared, and so on. We will have a service to notify all the interested users whenever an update occurs in a file and, similarly, a service to upload/download the file from cloud storage. As we have defined the core components of the system, let us design a high-level design of the system:

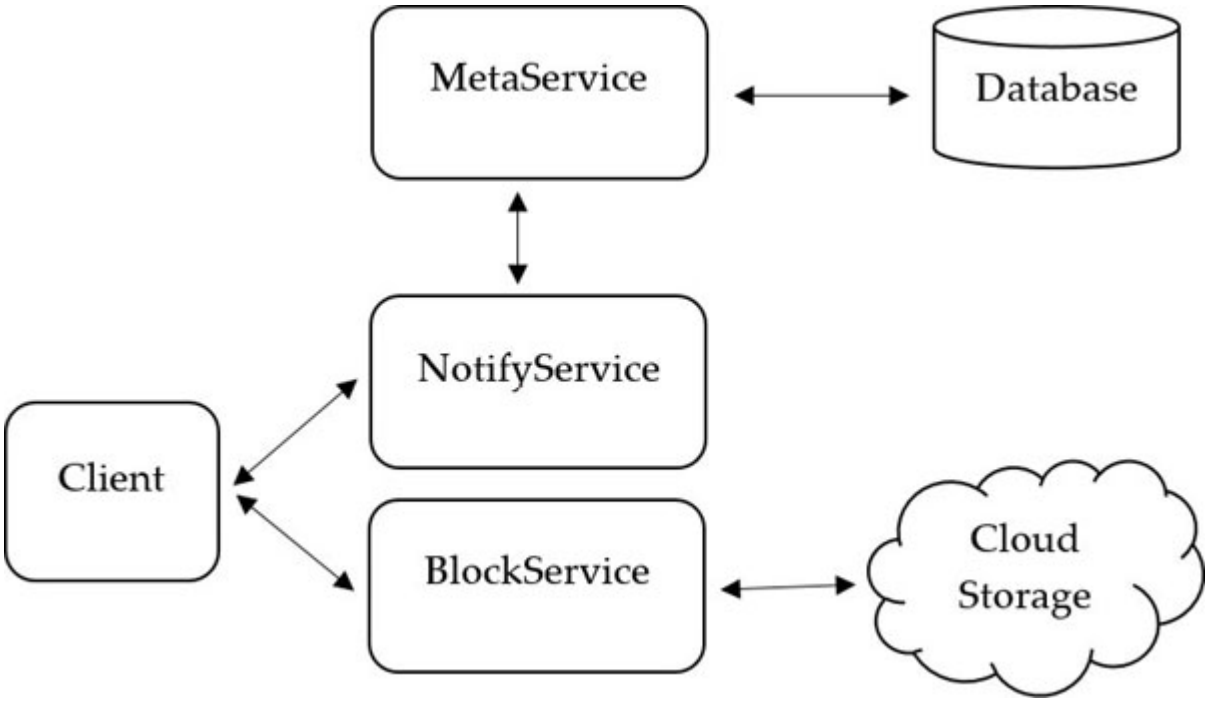


Figure 10.1: *A trivial high-level design of the file hosting service*

In this simplistic version of our high-level design, we have:

- **MetaService**: It will update the file metadata.
- **NotifyService**: It will take care of notifying users whenever there is an update on file.
- **BlockService**: It will help users to upload/download a file from cloud storage.

Let us dive deep into the design considerations. The client specifies the workspace in his machine, and the files inside the workspace will be synced across all the users. He may also upload/download the file. So, there are a few questions we must consider before discussing how to build the client part of the application.

First, how to send changes made to a file to all the users?

A very naïve way would be to transfer every change made to a file across the network and propagate the changes to all users with whom this file is shared. There are multiple problems with this. First, not every time a user edits the whole file, and second, the user may make changes when they are offline, and in that case, our approach will fail. Also, multiple users may be working on the same file, and in that case, we might have some inconsistencies with the final version of the file.

Second, how to transfer a file in the most optimal way?

It just does not make sense to transfer the whole file with some small information of what is changed across the network. A better way would be to divide the file into small parts and then transfer only those parts which are changed across the network. It will save bandwidth and a lot of computing resources.

Third, how can a user receive an update about a file?

A user cannot repeatedly raise the request to see if there is any file update. It will be detrimental to the system's performance. We can use HTTP long polling for this. In HTTP Long polling, the client sends a request to the server with the understanding that the server may not respond immediately. The server will hold the request until it has the data, and then it will send the response back to the client. The client, upon receiving the response, can immediately fire another request.

Based on the preceding discussion, we can observe that the application need to:

Divide the file into small parts.

Send only those parts which are changed across the network.

Use a technique like HTTP long polling to receive the file update.

Maintain a local metadata database to store file updates along with file metadata, which can help even when updates are made offline.

So, we can delegate these responsibilities to different components of the application in the following manner:

- **Internal file database**: It will store the file metadata such as name, path, last modification timestamp, version number, number of parts in which it is divided, and so on.
- **File divider**: It will divide the file into small parts so that we can intelligently transfer only those parts which are changed. It should also be able to reconstruct the whole file from those parts if needed.
- **File synchronizer**: It will serve two purposes:
    - Watch the file updates.
    - Monitor the local workspace and notify the File Service if there are any changes.
- **File service**: It will perform the following operations:
    - Receive the file updates either in the local workspace or by other users from the File Synchronizer and update the Internal File Database.
    - Submit the file changes to the cloud storage if changes are done locally and also invoke the appropriate service responsible for communicating to all the users that the file contents are changed so that they can also update the file and its metadata information.

To sum up, we will have four components of the application on the client-side, and the same is depicted in the following diagram:

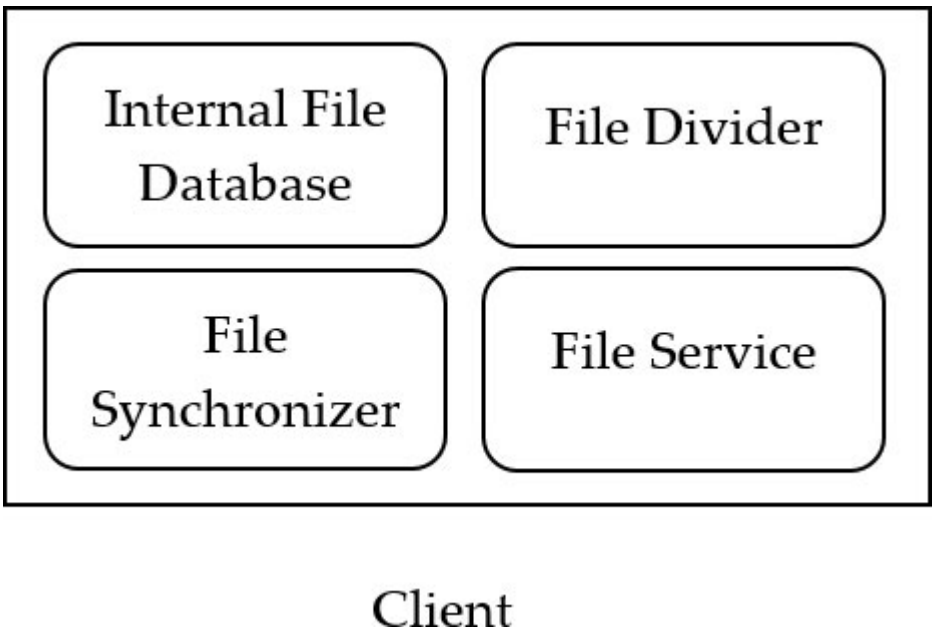


Figure 10.2: *Application components at client-side*

Now, moving forward, we will discuss how the application the NotifyService will work. It is responsible for two core operations:

- Notify the user about updates on the file.
- Make sure that updates are applied to the user's workspace.

The NotifyService will make sure that the workspace of all clients is in sync with each other. The client can directly interact with this service to communicate the file updates and also to retrieve the file updates, if any, done by the other users. The service will ensure that the updates it receives from the client are pushed to the cloud storage, and file metadata information is also updated in the File Metadata database. Prior to pushing the changes to the cloud storage, the service can perform a few checks as well with the help from the file metadata database, such as if a user has file edit permission, or if the file is no longer available, and so on.

Now, the NotifyService service must be intelligent enough to make sure that sharing file updates for other users should consume less bandwidth, resources, and time. Now, as we mentioned earlier, the file divider splits the file into small parts, so we can leverage this information to make sure that we send minimal data. We will send only the modified chunks across the network. Now, every user on receiving the notification to update their local copy shall update only the selected parts of the file only.

At this point, we must discuss the role played by the File Metadata database, which is a very crucial component of the design. The database, as the name suggests, is a go-to store for all the file-related information like a list of all the users the file is shared with, which user has what permission, last modification timestamp, file path, and so on. The database is updated by NotifyService, as mentioned in the previous paragraph. It is very important to decide whether this database should be SQL or NoSQL. Because a file can be edited by multiple users at the same time, we must make sure that changes done by the users are persisted and pushed to the cloud storage one by one. Overlapping different changes will lead to inconsistency. This requirement is inclined toward SQL databases as they follow ACID properties. So, we can use the SQL database for the File Metadata database.

A perineal question for every system design is, is the solution scalable? If we see carefully, there are still some gaps left. File I/O operation on each client and in the cloud storage is a time-consuming operation. So, does it make sense to wait for each user to update the file contents and also for the changes to persist in the cloud storage? Because we are designing a scalable system, a system where each file is shared among millions or maybe billions of users and each user pulling/pushing the changes, then we must need a different strategy. We can consider using the messaging queues to scale our system.

We will have two global queues in the following manner:

- **Request queue**: Whenever a modification is done, the client will publish the request to the queue. The NotifyService will pick the request and process it accordingly.
- **Response queue**: The NotifyService will publish the message to queue for all the interested clients so that they can update their local workspace as well.

Note: Messaging queues provide asynchronous communication between services. It stores the messages until they are processed and deleted. Each message is processed only once. It provides an efficient, loosely coupled communication between components of a distributed application. It is highly available, scalable, and reliable. A few examples are Amazon SQS, Kafka, and RabbitMQ.

Another important component of the application is Cloud storage, where files are actually stored. The clients are responsible to push/pull changes from this store. We can use Amazon S3 as the cloud storage for our application.

Now, let us consider a couple of scenarios:

- Client c1 created a new file and shared it with client c2.
- C1 and c2 edited the file in the same manner in their workspaces.

Now, in this scenario, does it make sense to update sense to update the data for both c1 and c2? To avoid such a situation, we can add some additional processing on the client-side. Whenever data is entering the client's workspace, we will compute the hash of it. Now, we will compute or maintain the hash of all parts of the file internally as well, and we will compare it with the incoming data's hash to determine if the file has actually changed or not. This way, we will save the unnecessary updates on the file.

The next important part is how to scale the File Metadata database. As we discussed earlier, a file Metadata database is going to be a SQL database, and SQL databases are difficult to scale horizontally. We can adopt a few schemes for partitioning the database:

- **Consistent hashing**: We will use the File object to compute the hash value, and the hash value will point to the database partition in which we will store the data. Consistent hashing will ensure that the partitions are loaded evenly.
- **Partitioning based on file name**: We will use the file name to determine the database partition. For example, if the number of partitions is 100, then we will compute the sum of the asci value of all filename characters and perform its modulus with 100 to find the partition number.

Now, consider a file that is shared with millions of users. If we fetch this file metadata from the database, then it will be very slow. We can use cache for speeding up the access. The cache will store the metadata information of those files that are actively edited and used by several users at the same time. We can use Redis for this purpose which is a key-value data store.

Since the file is broken into several parts, we can use file id + file part number as the unique key and data points to the metadata information.

Furthermore, if needed, we can use the **Least Recently Used** (**LRU**) scheme to evict the entry from the cache to discard the entry that is least recently used.

We can redefine the high-level design of the system in the following way:

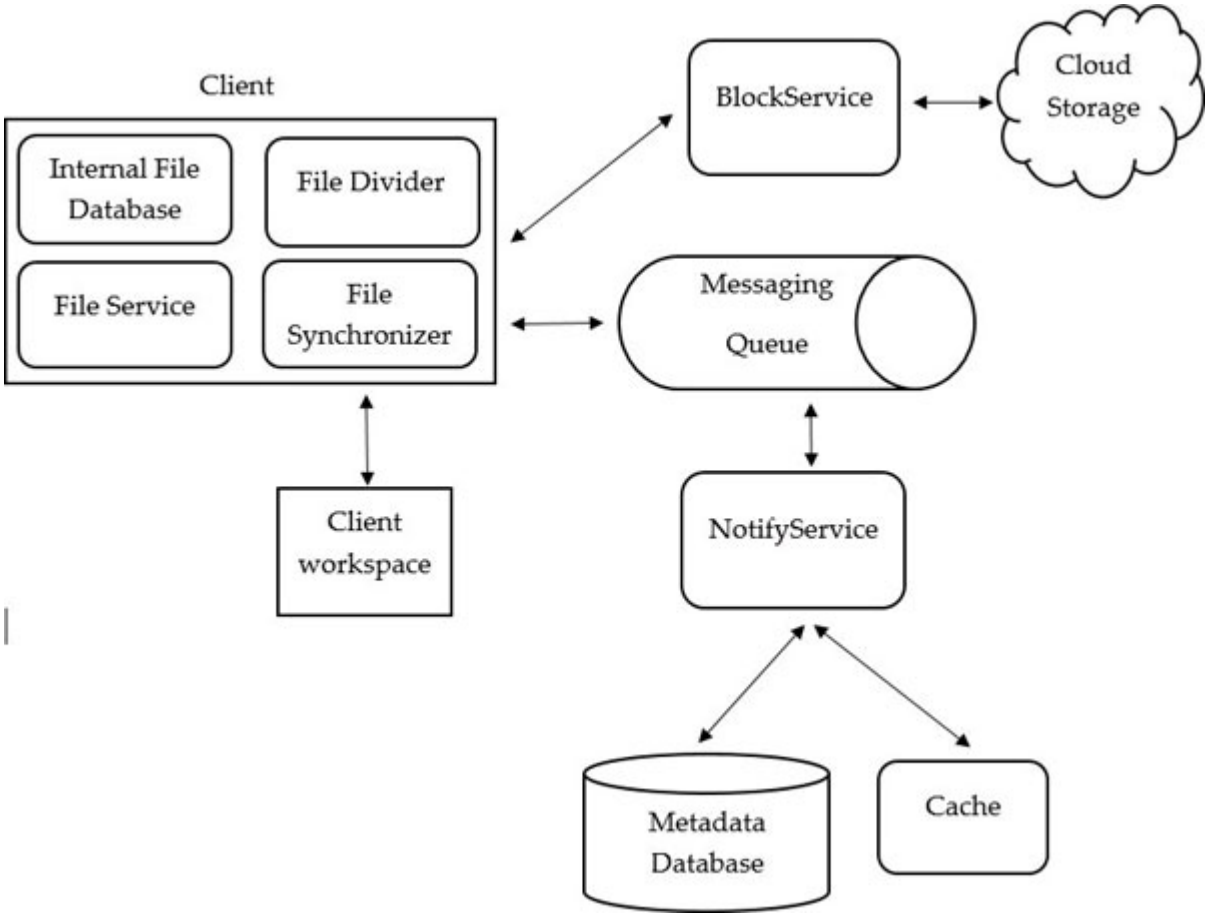


Figure 10.3: *Redefining the high-level design*

As depicted in the preceding diagram, the client-side application will monitor the client's workspace. Whenever there is a change, the block service is delegated the task to upload the changes to the cloud storage. A message is put to the queue for the notify service, which updates the database. The NotifyService also puts the message for all the clients who have subscribed for the file, and the client picks the message from the queue and updates their local workspaces. We can further scale the application in the following ways:

- Create redundancy for the services and introduce different servers in which these services will be running.
- Introduce a load balancer between the client and BlockService.

Conclusion

In this chapter, we discussed how to create a scalable file hosting service. We started with defining system requirements and then listed out the essential components of the system. Initially, a trivial high-level high design is drawn, and then we worked on scaling the individual components and finally ended up with a new high-level design of the system. Readers must note that the approach for this system design problem statement is similar to previous chapters, and the same methodology can be applied to any other system design problem as well. The upcoming chapter discusses another system design problem.

Points to remember

HTTP long polling is a mechanism in which the client sends a request to the server with the understanding that the server may not respond immediately, and the server responds only when data is available, or a timeout occurs.

Messaging queues are asynchronous communication channels between two different components of a distributed application.

ACID properties in the context of the database world stand for—Atomicity, Consistency, Isolation, and Durability.

SQL databases generally possess ACID properties.

LRU is a popular cache eviction policy to make room for new data when all the positions in the cache are filled. In this algorithm, the entry which has not been used recently is thrown out of the cache.

Multiple choice questions

1. **Which of the following states the primary purpose of messaging queues?**

    a. It is used to evenly distribute the incoming traffic across a number of servers.

b. It is used to store the data.
c. It is used to store the HTTP connection objects.
d. It is used to provide asynchronous communication between services.

2. **Which of the following states the primary purpose of cache?**
    a. It is used to evenly distribute the incoming traffic across a number of servers.
    b. It is used to store the data.
    c. It is used to store the HTTP connection objects.
    d. It is used to place the frequently accessed data as it provides low latency than databases.

Answers

1. d
2. d

Questions

1. Write pseudocode to implement the LRU cache?
2. Explain the ACID properties in the context of databases?
3. What is HTTP long polling?

Key terms

- HTTP long polling is a mechanism in which the server holds the client's request until it has data to send.
- Messaging queues are software components that provide an asynchronous communication channel between services of a distributed application.

CHAPTER 11

Design a Business Communication Tool

Introduction

In this chapter, we will discuss how to create a business communication tool. Communication tools like Messenger are a must for every organization as people often need to contact other colleagues for some discussion. We will follow the same principles as mentioned in the previous chapters to design it. First, we will list the requirements, then a high-level design is drawn, and post that, we will refine the design and all components individually so that the application works at scale.

Structure

In this chapter, we will cover the following topics:

- Defining system requirements
- Designing a scalable system

Objectives

After studying this unit, you will be able to understand how to define system requirements for a business communication tool and how to put in use various technologies to create a scalable business communication tool.

Defining system requirements

We aim to create a communication tool where employees can essentially send text messages to each other. Employees can use this application on their mobiles or desktop application. Let us start by defining the functional requirements of the system:

- The user should be able to chat with another user.
- Conversation history should be persistent.
- The application should be able to show whether a particular user is online/offline.

Some non-functional requirements:

- The application must be consistent. Conversation history between two users must be the same across all devices.
- The application should be highly available.
- There should be minimal latency.
- The application can be multi-tenant.

Note: Multi-tenant architecture is used to serve the single instance of an application to multiple users. Each customer is a tenant, and they are given the flexibility to customize UI (user interface) or some business rules. The architecture is scalable and less expensive to customers. However, there is some tradeoff like stricter authentication and chances of noisy tenants (the ones who monopolize the bandwidth).

Back-of-the-envelope calculations

Let us do some back-of-the-envelope calculations to see the storage and other requirements. Consider:

The number of daily active users: 50 million.

On average, each user sends 50 messages.

So, total messages per day = 25,00,000,000.

Consider the message size to be 500 bytes. So, to store all messages, we need 1.25 TB.

In 1 year, we need 12.5 TB of storage.

The calculations are approximate. Users might exchange pictures or files as well, which will be heavy in terms of size. Also, with every message, we have to keep track of timestamp, sender, and receiver id. So, total storage might be more than what we just calculated.

Designing a scalable system

Let us move on to the high-level design of the application. In order to design a high-level diagram, let us see how a typical workflow of the system looks as follows:

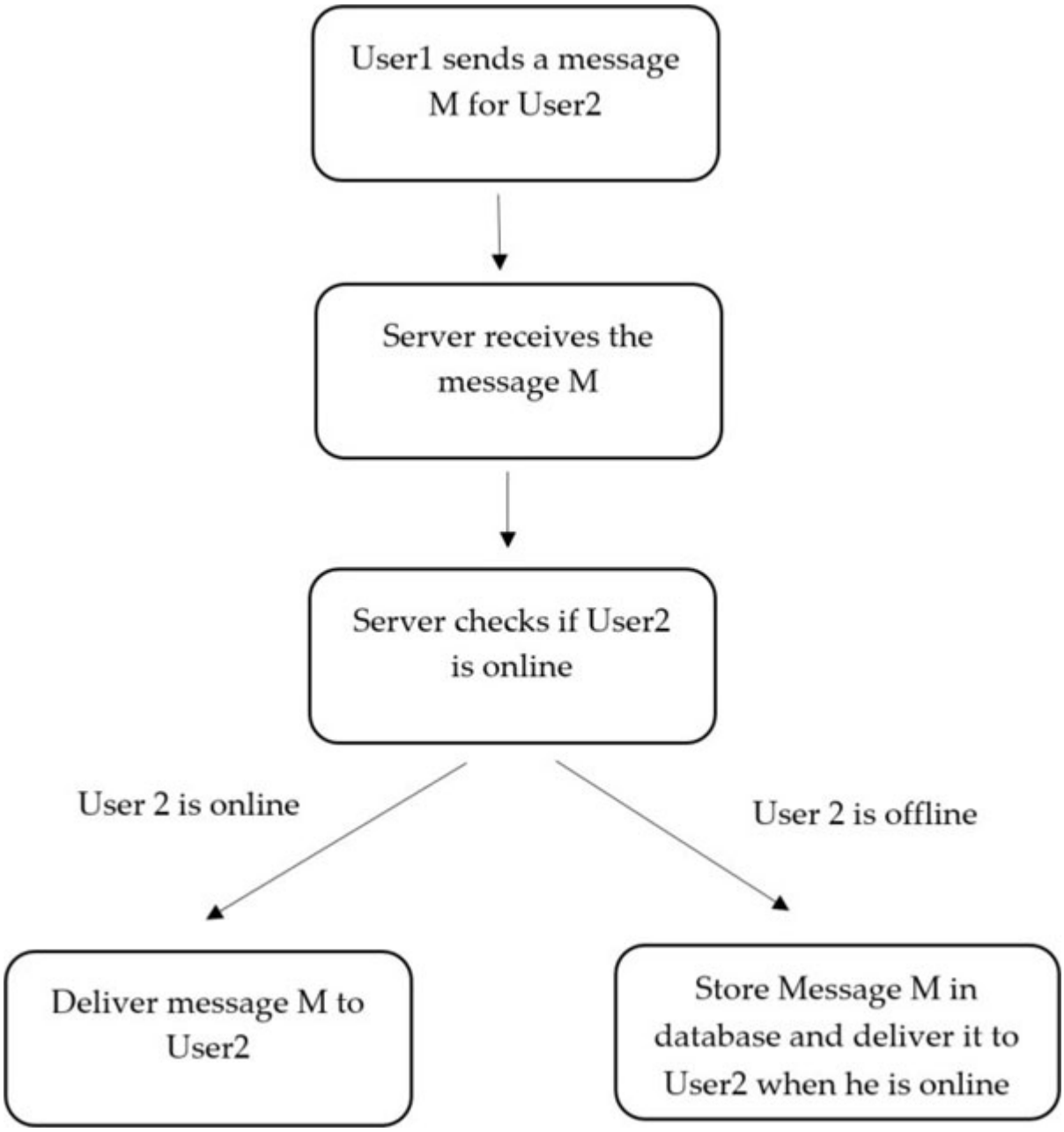


Figure 11.1: *A typical workflow of the chat application*

Based on the preceding workflow, we may have a trivial system arrangement in the following manner:

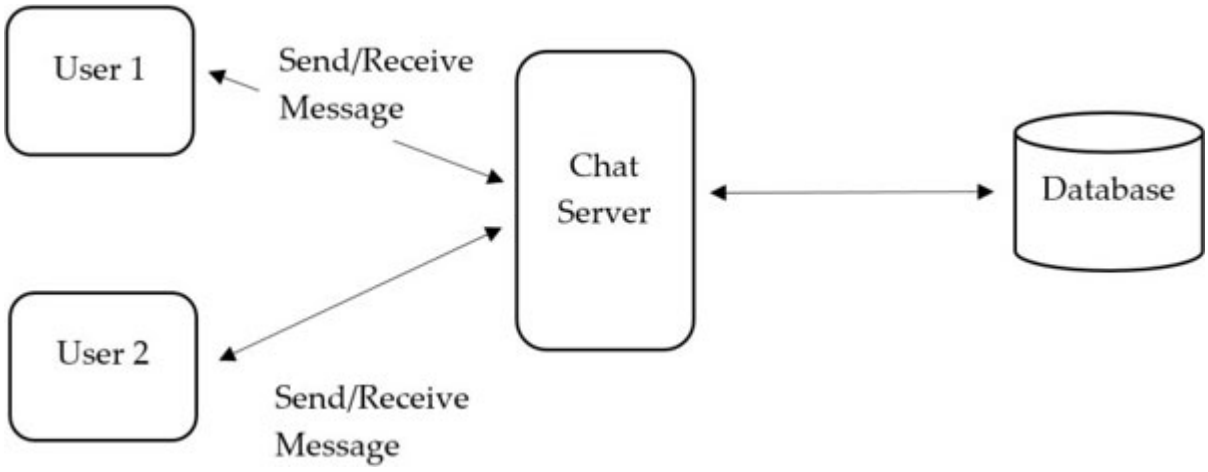


Figure 11.2: *A trivial high-level design of the chat application*

In the preceding design, the chat server receives/sends the message from/to a user. It keeps track of which user is online/offline and also stores and fetches the message from the database. However, there are still many unanswered questions with this design. How does the message from User1 reach User2? We can use two approaches to address this:

Users can check with the server at regular time intervals for any new messages. This approach has the disadvantage that most of the time, the user will get an empty response if there are no new messages. Moreover, regular checking will consume a lot of network resources and bandwidth as well.

Another approach could be to keep an active connection with the server and let the server send the response to the user whenever a new message arrives. This approach prevents the client from checking with the server repeatedly; thus, saving bandwidth and increasing performance. The server must keep track of all active connections so as to send data back to the client. It can store this in a key, value datastore where the key can be client id and value can be connection object. So, when a new message arrives for a user, the server just has to perform a lookup operation to retrieve the connection object. Furthermore, we can use HTTP long polling and WebSockets for keeping active connections between client-server.

Note: HTTP is based on the request-response model; that is, a client initiates a request, sends it to the server, the server computes the response, and sends it to the client. In a chat application, the server

needs to initiate the request, so HTTP may not be fit for the design. There are a few alternatives to this as follows:

- **WebSockets: They support bi-directional connection. So, both the client and server can initiate the request.**
- **HTTP long-polling: In this, the client sends a request on an HTTP connection, but the server only sends a response when it has data to send. When the client receives the response, it makes another request on the same HTTP connection.**

Now, suppose User U1 sends a message to User U2, but U2 is offline. In that case, what can we do? We can deal with this situation in the following two ways:

The server can store the messages in a database and wait till U2 reconnects. When U2 comes online, the server it is connected to will first check in the database if there are any pending messages from any other user.

The server can notify the sender U1 that U2 is offline and prompts him/her to resend the message. We can ease the user's convenience by storing the message so that the sender does not have to retype the whole message.

Let us discuss how a message delivery will happen (assuming that both U1 and U2 are online). Essentially, the following steps will occur when a U1 sends a message to U2:

U1 types and send a message for U2.

The message will reach the server S1 connected to U1.

S1 will trigger a database insert operation to store the message in the database. In parallel, S1 will locate the server S2, which is connected to U2, and pass the message.

S2 will send the message to U2.

U2 will send the acknowledgment to S2.

S2 will send the acknowledgment to S1.

S1 will send the acknowledgment to U1.

Let us consider a scenario in which U1 types and sends messages M1 and then M2 for U2. Now, suppose due to some network delay, M2 arrives before M1 at S2. In this case, U2 will first receive message M2 and then

M1. In order to resolve this, we can attach a sequence number with every message so as to identify which message was sent first, and accordingly, we can take care of sending messages to the receiver in the same order.

Storing messages: SQL or NoSQL?

Now, a very important question: which database to use to store chat messages?

To answer this, let us see what will happen if we use the SQL database. If we use the SQL database, we will initiate an insert query for every new text message and, similarly, read query for every text message read. If we think more, then it is easy to realize that people tend to write many small text messages in a chat, and they mostly read messages in a sequential manner. So, using a SQL database makes little sense for our use case. Then, what we should do? Use a NoSQL database to store messages? The answer is tricky as a NoSQL database will also end up having the same problem—initiating a database read/write operation for every message read/send, and again, it would be very bad for the system's performance. A better choice in our use case is to use a column-wide database like HBase. HBase is able to give a very fast lookup and also provides random access (read/write) to a large amount of structured data. The schema of a table in HBase typically looks as follows:

Row key	Column family			Column family			Column family		
	Col1	Col2	Col3	Col1	Col2	Col3	Col1	Col2	Col3
1									
2									
3									
4									

Table 11.1: *Typical schema of a column-wide database*

A table in HBase consists of rows, and each row is a collection of column families. Column family is a collection of several columns, and each column is a collection of key and value pairs. HBase will provide us various advantages, as follows:

- It can store multiple values against one key. So, we can store messages corresponding to a particular user in one row. It will be good for the system's performance when a user is accessing messages of a particular user or when we have to store several messages sent by a user.
- It will first collect all the updates in memory and then write it to the disk. This approach improves the system's performance when we have to store a large amount of small data.
- Value in the HBase database can be of variable size.

The next question we should take care of is—how to display user status, that is, whether a user is offline/online. Let us see how it can be done. Consider the following high-level design of the system:

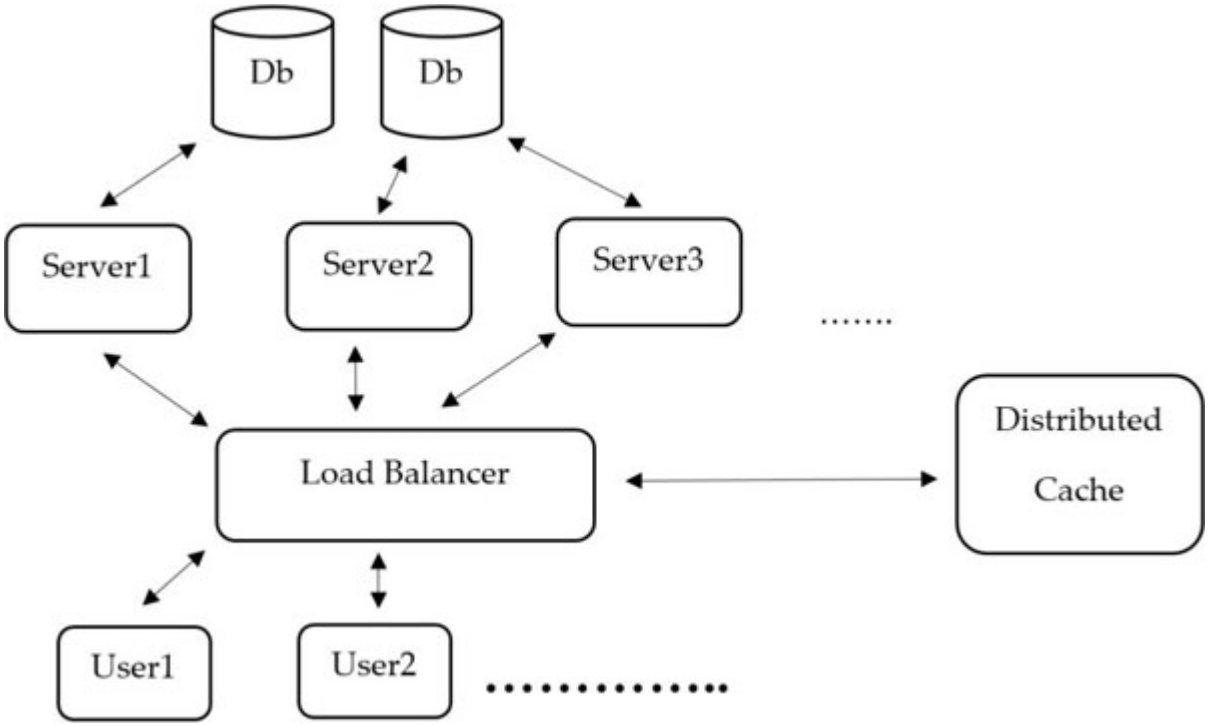


Figure 11.3: *Re-defining high-level design of the chat application*

In this high-level design, we have placed a load balancer between the user and chat servers. Whenever a user comes online, the load balancer connects them to any of the servers based on their current load. Then, let us say a bi-directional WebSocket connection is established between client and server so that even the server can send a request. Now, to display the online/offline status of the user, the user and server will constantly be heartbeat with each

other. This way, the server will know that the user is still online and connected to it, and the user will know that server is still alive and connected. We can leverage this information to display offline/online status. We will store the heartbeat information in our distributed cache memory. We can create our key for cache based on UserId and ServerId, and value can be the last timestamp of the received heartbeat. Now, there are many popular distributed caches, such as Redis, Memcache, and so on, which can be used. Some distributed cache offers plenty of advantages as follows:

- They follow a master–slave architecture.
- They can even write entire data to the disk.

Also, it is certainly feasible to put the data in distributed cache even when there are billions of users. Furthermore, reading and writing in a cache are very fast. So, reading user status and updating will be very quick. Now, let us see how to provide a user status to another user. We can adopt the following strategies to display a user's offline/online status:

Update status on demand: it implies, say, user a when wants to chat with user B; at that point, we will fetch the user B status from the cache only.

Aggressive update: say user A comes online, then at that point, we will fetch the status of all of its friends from the cache and also broadcast the status of User A to all its friends.

Say, User A sends a message to User B, and User B goes offline, and message delivery fails. In that case, we will fetch the user status from the database. The same can be done when User B was initially offline and comes online when user A is sending a message.

Additionally, we would need the status information in the database. As mentioned earlier, we can flush the entire cache content to the database. We can do this operation when the load on the system is very low, or we can run a periodic job to perform this operation.

Scaling data access

It is time to think about data partition schemes we would need to use for our use case because we will be keeping messages history since the start time, and data will grow exponentially every year. A viable solution would be to partition the data based on user id. Say we choose to create 100 shards.

Now, for every user, we will perform (userId % 100), and that will give us the shard number in which we will be storing the data of that particular user. We may also partition the data based on the message ID; however, the user generally searches all the messages of a particular user at a time. So, partitioning based on user id makes more sense.

It generally happens that the users scroll through the recent messages. So, we can maintain the messages in the user's current viewport in cache memory.

We discussed the variety of techniques in Chapter 9: "Design a Key-Value Data Store" to maintain data durability (replication groups). We can use the same strategy here for keeping the chat messages. However, most of the databases offer data durability, so we will not be discussing it here.

Conclusion

In this chapter, we designed a business communication tool using the same approach as mentioned in the previous chapters. We started with the system requirements, designed a basic high-level design, and then worked on scaling individual components. We used a lot of technologies such as distributed cache, load balancers, column-wide database, and so on in our application and also leveraged protocols such as long-polling and WebSockets as they fit perfectly for our use case. Readers are encouraged to learn these protocols and technologies usages as they are widely used. In the upcoming chapter, we will discuss another system design problem and try to solve it with the same approach.

Points to remember

- Websocket is a bi-directional communication protocol in which both client and server can initiate the request.
- A wide column database is a NoSQL database where each row is a collection of columns families, and each column family consists of various columns. It can be visualized as a two-dimensional key-value store.

Multiple choice question

1. **Which of the following is true for the WebSocket protocol?**
    a. It provides full-duplex communication.
    b. It provides a half-duplex communication
    c. It is a session layer protocol of the OSI model.
    d. It is a physical layer protocol of the OSI model.

Answer

1. **a**

Questions

1. What is a distributed key-value data store?
2. What is a column-wide database?
3. What are the different types of NoSQL databases?

Key terms

- WebSocket is a Layer 7 (OSI model) protocol providing a full-duplex communication channel.
- HTTP stands for Hypertext Transfer Protocol.

CHAPTER 12

Design a Card Authentication System

Introduction

In this chapter, we will design a card authentication system for an office space. The card authentication system is designed to authenticate a card so that unauthorized users are not allowed to enter the office premises or a part of the office. We will start by defining the requirements and then work toward seeing how to create a scalable system for the same.

Structure

In this chapter, we will cover the following topics:

- Defining system requirements
- Creating scalable system design

Objectives

After studying this unit, you will be able to understand how to define the system's requirements and a scalable card authentication system. You will also learn how to put in use different technologies to design a scalable system.

Defining system requirements

A card authentication system is used to authenticate the user so that an unauthorized user is not allowed to enter the building or a part of the building. A system will be installed at the gates of the building, outside a room, or any part of the office that needs to be secured, and the user will present their card near this system. This system internally calls the card authentication system, which will allow/disallow the user. Now, let us define the scope of our system. A company can have many offices, and

every office will have *n* number of card readers installed. Also, some rooms are confidential in nature, and only a class of employees are allowed. So, based on this, let us define the following functional requirement of the system:

- An admin can add/remove/modify the card permission.
- An admin can view the logs from Log Database for audit purposes.
- A user, when he places his card on the card reader machine, is allowed to pass through the gate if he has permission.

Other non-functional requirements could be as follows:

- The system should be highly available.
- The system must be reliable. We must not allow unauthorized entry.
- The system should be able to authenticate in a minimum time. Latency must be minimum.
- The system must store data in a secured fashion.

Back-of-the-envelope calculations

Let us perform some back-of-the-envelope calculations.

Number of employees = 100,00,00

Memory needed per user = 100 kB

Space consumed for all users = 100 GB

Number of card readers per office building = 50

Number of office premises = 100

So, considering that even if 10% of employees are on leave, the number of Query per second would be 900,000.

Creating scalable system design

Now, it is time to create a scalable system design. First, let us identify the important actors of the system. There will be an admin user who can add/modify/remove the card permission of a user. There will be a normal user who will use the card reader to enter the room or building. We can have a dedicated service for the admin and another service for the user

responsible solely for authenticating the user. Permission details will be stored in the database. The same is depicted as follows:

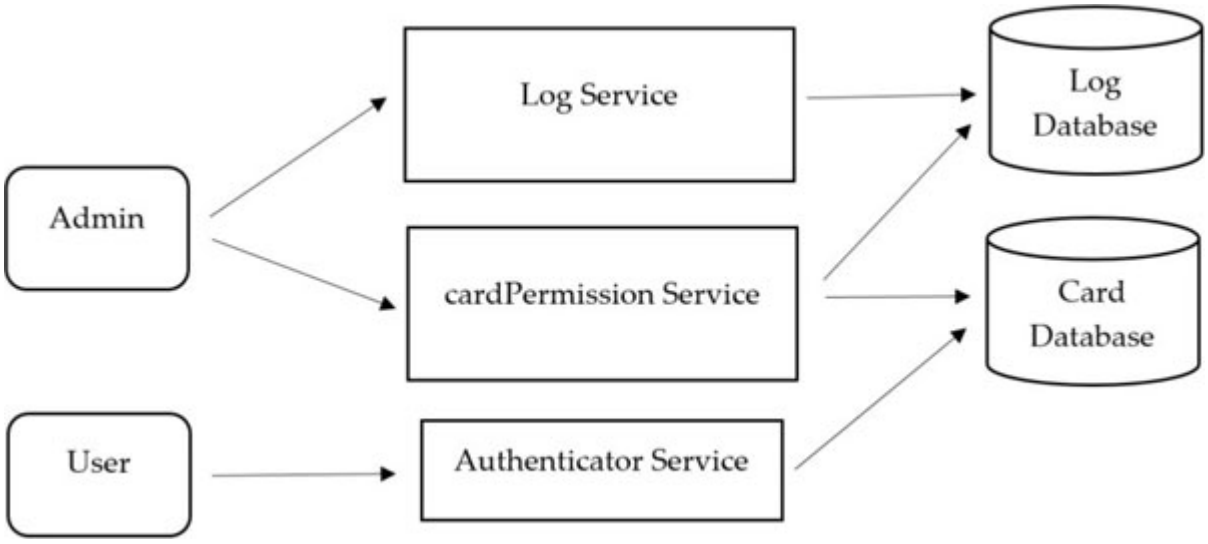


Figure 12.1: *A trivial high-level design*

As shown in *figure 12.1*, the admin will interact with the `cardPermission` service to edit the card details, and the same will be reflected in the database. Users will interact with the authenticator service, which will pull data from the database and compute if a user is authorized to enter the gate. The admin user can also view the log reports.

Scaling the design components

We have identified the core components of the system. Now, it is time to work on scaling these components.

- **Minimizing the database query**: Our system is going to be both read-heavy as well as write-heavy system. There will be many read queries on the database to authenticate the user and as many write queries for inserting the logs in the logs database. There are a couple of optimizations we can do for the database read operation, like using cache memory. We can place the cache between the authenticator service and the database. The cache will be used to store the permission details of the card.
- **Authenticator service**: It is the most important part of the system, and it is essential to harden this server for security purposes. We must keep

the servers running the authenticator service in a secured data center. Now, every card will have basic details like userId embedded in it, and every card reader will have a unique id. Whenever a user puts his card on the card reader machine, a request will be sent to the authenticator service that contains the user id and the card reader id. The card reader id will be used to identify the area and to check whether the user with the user id is permitted to enter the area or not.

- **Card database organization**: We need a database for storing the permission details. In our use case, we will have a pre-defined schema for all the tables, and additionally, we would need joins of different tables for computing the result.

So, the SQL database makes more sense for our use case. We will have the following tables:

- **User table**: It will be used to store the userId and the group it belongs to. Group can be a full-time employee, part-time employee, housekeeping, HR employee, Accounts Employee, and so on.

userId: Integer
Group: string

Table 12.1: *User table*

- **Group table**: It will be used to store the group and the list of departments that a particular group can access. Departments can be Accounts, Finance, Laboratory, Inventory, Security, and so on.

Group: String
Departments: list <String>

Table 12.2: *Group table*

- **Card table**: It will be used to store the card reader id and the department it belongs to.

cardReaderId: Integer
Department: String

Table 12.3: *Card table*

The authenticator service should respond with minimal latency. So, we must make sure that the database query should be fast to meet our needs.

We can use the database partitioning technique as follows:

- **Partition of the database based on user groups**: We can create a partition based on the user groups. So, a user belonging to the Accounts Employee group will be stored in one database, and similarly, the user belonging to the HR Employee group will be stored in another database. The same strategy will be used for every user group. A disadvantage with this approach is that it may lead to uneven partitions; that is, some partitions may have more data than the other.
- **Partition of the database based on user Id**: We can partition the data based on userId. Let us say we have 10 database partitions. So, for every userId, we can compute the userId % 10 to find out the database partition in which the entry will be stored.
- **Partition based on hash**: We can feed the userId or group to a hash function that will return the partition number to store the data.
- **Round-robin partitioning**: We can use the round-robin technique to partition the data. In this technique, the *i*th entry will be inserted in partition number (*i*% *n*), where *n* is the number of the partition.
- **Log database organization**: Log database is used by the admin to view reports like who all entered the particular department between 10 p.m. and 11 p.m. A log will be inserted for every interaction between the card and card reader system. Since, at any point in time, a large number of people will be going in and out, we must keep doing log insertion in an asynchronous manner so that there is no delay in the authentication of the card. Moreover, log insertion involves a database write operation that involves disk access, and hence, it is an expensive operation. Now, there are a few questions we must answer:
    - What database we can use for Logs: SQL or NoSQL?
    - The ideal choice would be a NoSQL database as data will be massive, and over a period of time, we need to horizontal partition the data. Since NoSQL is horizontally scalable, it makes sense to use them.
    - How to ensure asynchronous insertion of log data?
    - We can use Messaging queue in our system. Every log will be pushed to the queue first, and then it will be picked by another

service for storing in the log database.

The system needs to be highly available. This makes redundancy extremely indispensable for our system. We will have multiple instances of authenticator service running across multiple servers for the same.

We can place the load balancer between the user and authenticator servers for even distribution of the load.

Now, based on the preceding strategies, we can re-define the high-level design of the system in the following way:

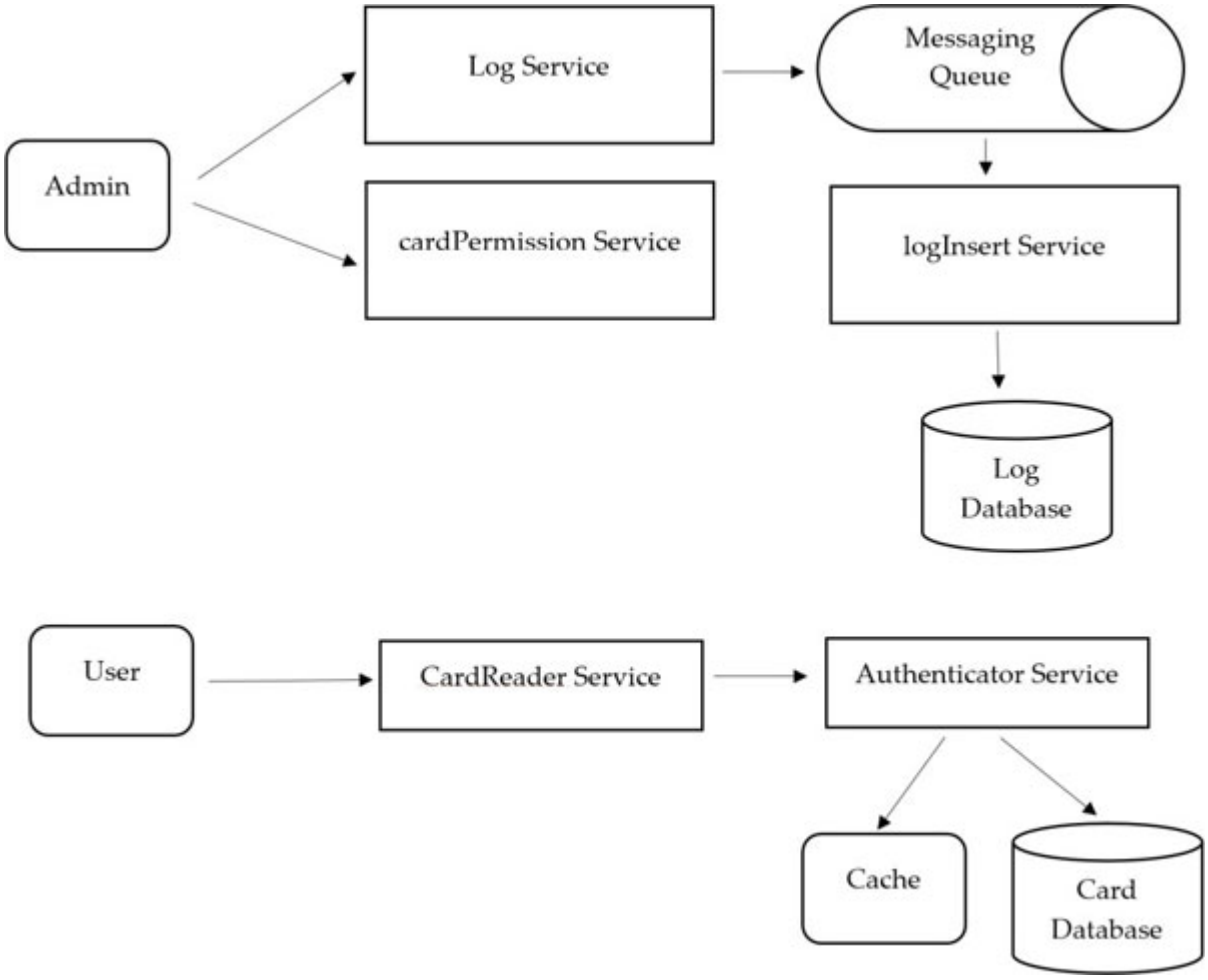


Figure 12.2: *Re-defining high-level design*

The preceding high-level design is much more refined. However, there are still a few things we can do to make it more scalable.

Load balancer

We can keep a load balancer between the card reader and authenticator service so as to ensure the minimum turnaround time for each request by sending the request to a server that can serve the request as early as possible.

Cache

A distributed cache server can be used because computing the access of a user is read-heavy. For those users who are frequently passing through the gates, we can keep the data in cache.

Security

The database storing the user permission details and the services sending the request and receiving the response are all critical points in our application because we cannot allow any kind of compromise in our system. It is thus essential to observe that data stored in the database and data being exchanged between services must be encrypted. However, the chapters were written with the intent to explain the basic technologies used in building the system and the basic approach for every system design. So, we will not be discussing the security aspect of the design.

Conclusion

In this chapter, we designed a card authentication system using the same approach discussed in previous chapters. We used a lot of technologies, such as Messaging queues, cache, and so on, to create a scalable system. Readers must realize that we have followed the microservice-based architecture, and so we have defined some services as well. Ideally, every service must be assigned a single responsibility. Readers are encouraged to apply the concepts and technologies learned in this chapter to design any other real-world systems as well. In the upcoming chapter, we will be discussing another system design problem.

Points to remember

- Redundancy is essential to make sure that there is no single point of failure.

- Messaging queues provide asynchronous communication between services of the application.

Multiple choice questions

1. **Which of the following is best suited if we want to access frequently hit data?**
    a. Cache
    b. Database
    c. Messaging queue
    d. Load balancer
2. **Which of the following data structure is best suited for cache?**
    a. Associative array
    b. List
    c. Stack
    d. Queue

Answers

1. **a**
2. **a**

Questions

1. What is a Service-Oriented Architecture?
2. How can you design a cache memory?
3. What are the differences between SQL and NoSQL databases?

Key terms

- Asynchronous communication refers to the communication in which the sender does not wait for the receiver to receive and respond.

CHAPTER 13

Design an Image Hosting Application

Introduction

This chapter aims to design a simple image hosting application that provides the ability to upload/download/view an image. The chapter discusses various principles and technologies that are in use to create a scalable application. The concept served in this chapter will be helpful to the readers to approach any system design problem.

Structure

In this chapter, we will cover the following topics:

- Defining system requirements
- Creating a scalable system

Objectives

After studying this unit, you will be able to understand how to design an image hosting application that works perfectly fine for a large number of users. The readers will understand the different technologies that play a crucial role in system design. The intention of this chapter for the readers is to relate the functioning of various scalable applications that are already in place and also be able to tackle any new system design challenge.

Defining system requirements

Consider a simple image hosting application where the user uploads an image. The image, once uploaded, must stay in the system. So, the system is reliable as well. The image, once uploaded, can be viewed/downloaded. One must note that the system has to provide low latency for image upload/download/view operation. Readers must note that for the sake of

simplicity, we will discuss strictly about image write/view/download operation only. We will not discuss how the user is going to log in and how we are assigning a unique login id to him, and other related things to the user.

Now, based on these discussions, let us define the system functional requirements:

- The user can upload/download/view a particular photo.
- The user can upload as many images as he/she can.
- The user can view images.

Non-functional requirements:

- Latency should be minimum while uploading/downloading the image.
- The system must be reliable. So, the image, once uploaded, must stay in the system.
- The system should be highly available.

Let us try to figure out in a rough manner how much space we might need. Consider that 15 photos are uploaded every second, and the average size of each photo is 100 kB.

So, 1,500 kB space is consumed every second.

Space consumed every day = 1500 × 24 × 60 × 60 KB = 1,296,00,000 KB, that is, approximately 130 GB.

In 10 years, the space consumed will be 130 × 365 × 10 = approximately 474 TB.

Creating a scalable system

Now, considering these calculations are done with various assumptions, the number of requests may shoot up easily. We may start with a simple arrangement (as shown in *figure 13.1*); however, it is very easy to exhaust such a system soon with an increasing number of requests.

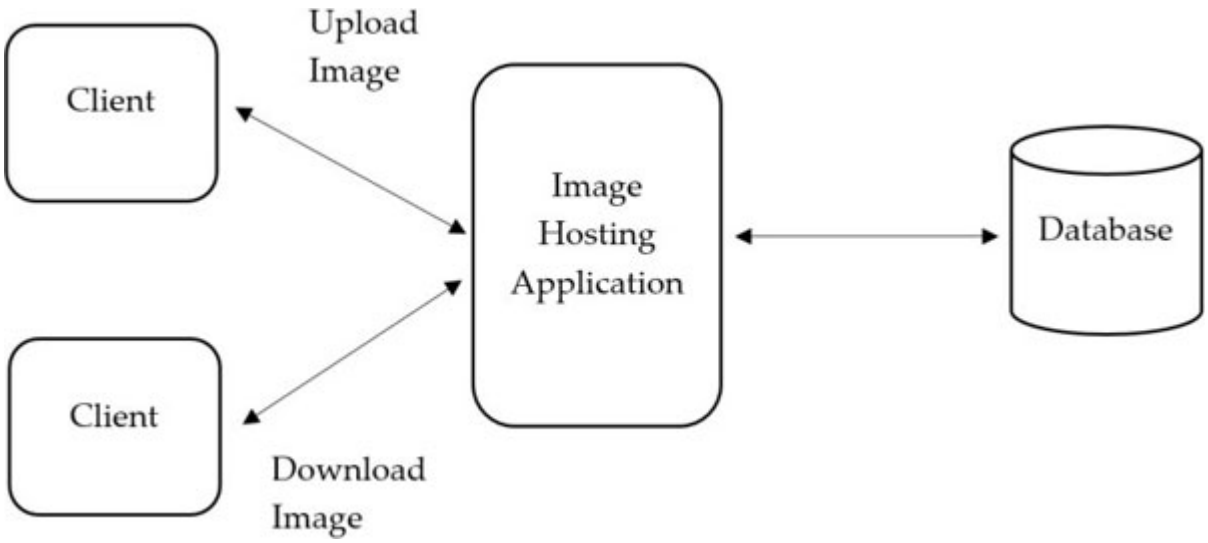


Figure 13.1: *A trivial high-level design*

In the preceding arrangement, we have several problems as follows:

- A single server is present to serve client requests.
- A single server will host the service, which will be responsible for uploading/downloading the image.
- When the traffic is too high, system latency will take a bow. Consider a scenario when there are multiple write requests followed by a read request. Since write request boils down to a database read operation (which is fairly expensive), the requests will be piled up, giving a huge blow to the system's performance.

Let us try to figure out how we can scale our system to work for a large number of users. Readers must note one point that it is always good to design a system as modular as possible by having loosely coupled modules. Each module is responsible for performing one function, and each piece interacts with the other with the help of APIs. When we delegate each module a single responsibility, then we have the flexibility of scaling it independently. Let us follow the same strategy and define two main services in our system: Image read service and image write service. Now, we may have a modified arrangement as shown in the following figure:

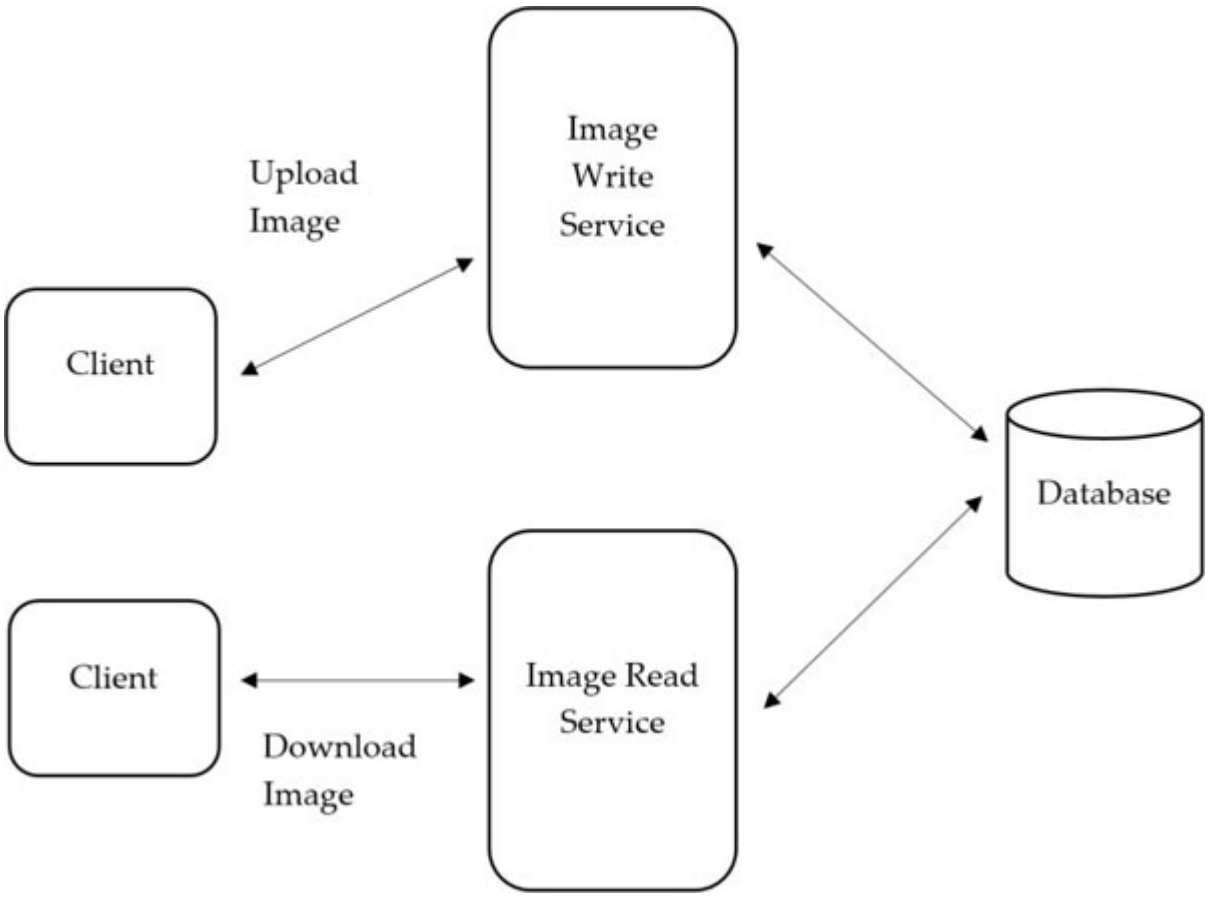


Figure 13.2: *Re-defining high-level design with loosely coupled services*

Creating two separate services for read and write operation provides various benefits like both of them can be scaled independently. Second, we know which service to debug if a situation comes like read is slow or write is slow, or Weblink returns an incorrect image. Now, let us assume that read service goes down. In that case, all the read requests will wait till the service is up and running again. To handle such situations, first, we must realize the fact that in the preceding arrangement, there is a single point of failure. So, if the server goes down, then the application will stop working. We must introduce redundancy to handle this situation. Now, redundancy in our case would be to create multiple instances of the same service that runs in different server machines so that even if one goes down, then there are other machines to take care of the client requests. Similarly, the database storing the images must have more than one copy of data in a different database, possibly in a different geographical location as well. In order to distribute the load among different services in a balanced manner, we must use a load balancer. So, our high-level design can possibly be shown as:

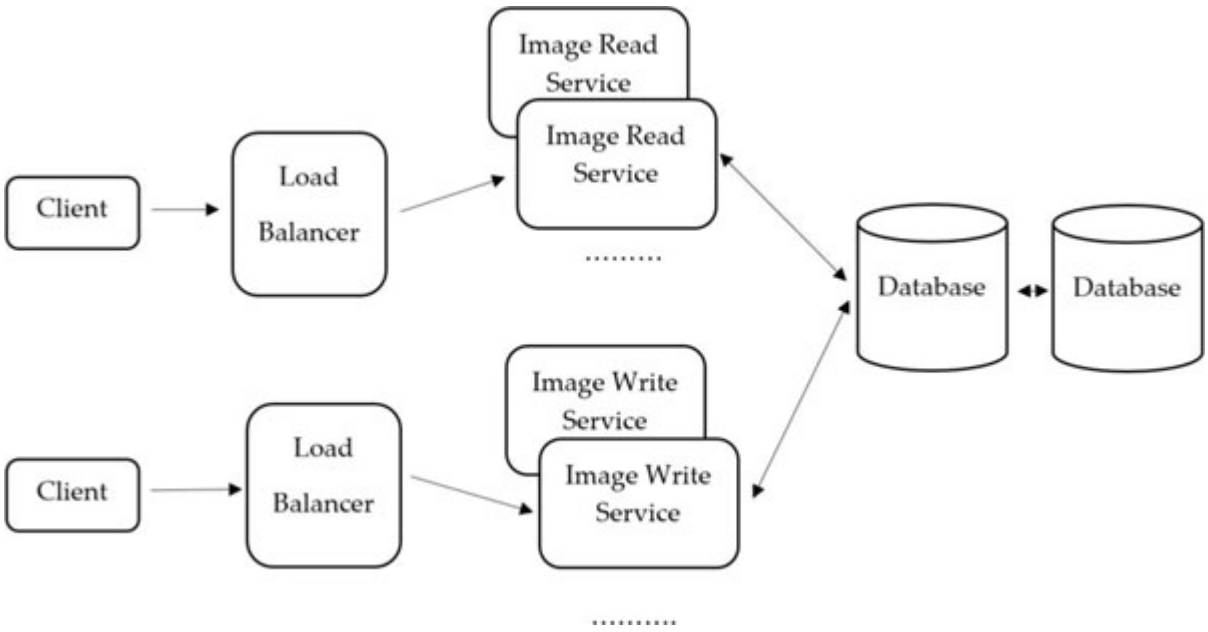


Figure 13.3: *Re-defining high-level design with redundancy*

With the preceding arrangement, we have multiple instances running. There is a load balancer to evenly distribute traffic among different servers. Further, database replication is done to ensure we keep multiple copies of the data.

Let us figure out how we can store our images. Images can be stored in an object store platform like Amazon S3, which can easily handle a massive amount of unstructured data.

Note: Object store treats data as an object that possesses various attributes like image data and a unique identifier/key. Each object can be retrieved using the unique identifier assigned to it. Object stores are very similar to key-value stores; however, there is a difference between the two. The value is key-value store is small in size, whereas, in the object store, that value can be huge, of the order of GB.

Readers must observe that image upload is an expensive operation. So, to make our architecture scalable, we will make it asynchronous. We can use Amazon SQS or any other messaging queues to achieve this. Since image size can be large, we will split the image into smaller chunks, assign them a sequence number and send them to messaging queue. One of the services at the other end of the queue will reassemble the chunks to construct the

whole image and then upload it to the object-store. With this change, our design can be modified to:

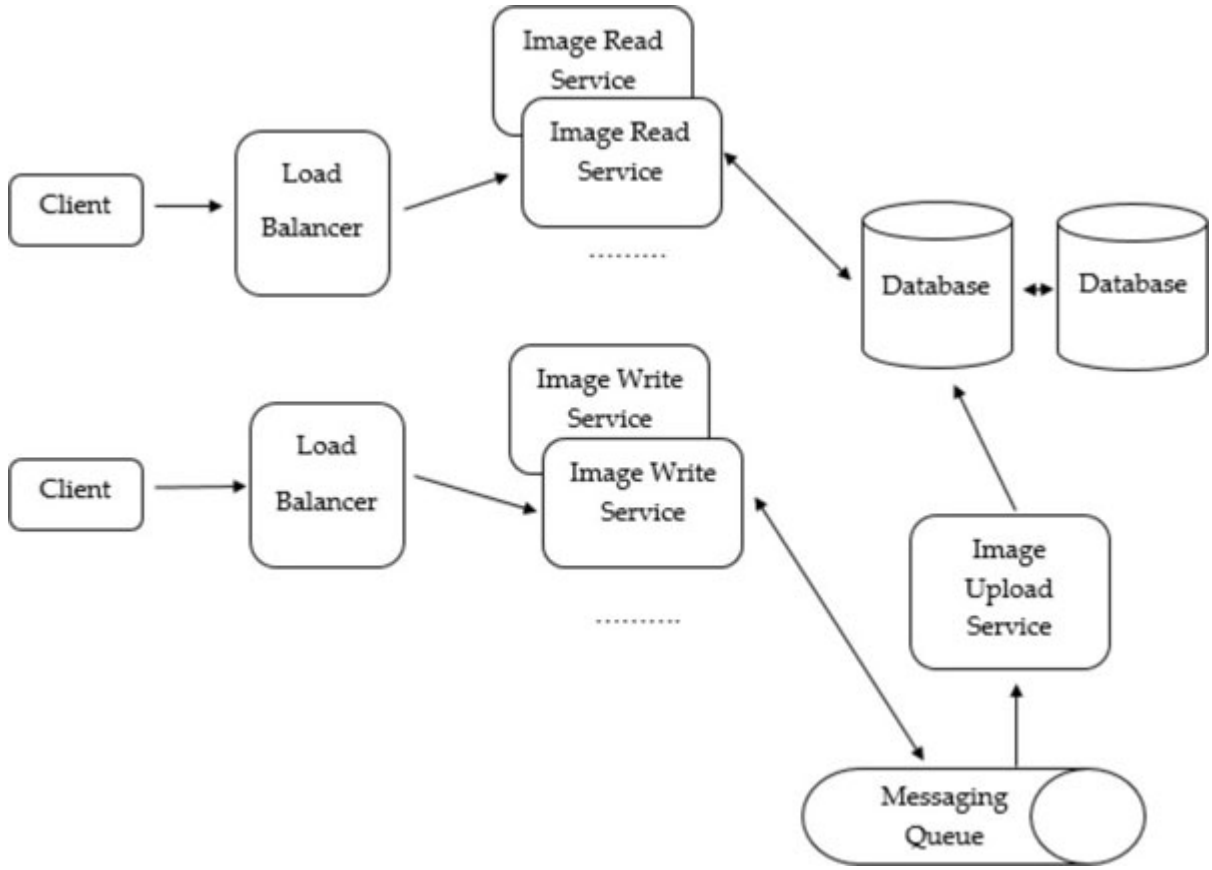


Figure 13.4: *Re-defining high-level design with asynchronous image upload operation*

With the preceding architecture, the image upload and write service can be scaled independently to cater to a large number of users.

Now, we can have a separate image table that may contain the attributes as follows:

- Photo unique identifier to object store
- Creation date and time
- User ID
- Tag

Using the photo unique identifier, we can retrieve it from the object-store. We are going to store the table corresponding to the preceding schema in a distributed no-SQL database because no-SQL databases are horizontally

scalable, and our image hosting application is going to deal with a massive amount of data.

Now, it is imperative to understand that since we are dealing with a massive amount of photos, our database read/write operation will be slow. In order to circumvent this problem, we have to do database sharding. In this scheme, we will create a horizontal partition of the data and store them across different shards. Let us discuss different approaches to store and retrieve data from the shard:

- We can partition data based on the user ID. In this manner, a particular shard will have all the photos of a particular user. Let us say we have 9 shards. So, we can do user id% 10, and this number will point us to the shard where we can store photos of the user. In order to assign a unique id to every photo, every shard can maintain its own counter. So, whenever there is an insert photo operation, we can append the shard id with this current counter number to generate a unique photo id.
- We can use a random number generator to generate a unique number. Then we can perform the modulus 10 of this number to get the shard in which we are going to insert that photo. We can improve this approach by having a separate key generator. The key generator will be responsible for generating a unique key that will be unique throughout the system. The only downside of this approach is that the key generator will be a single point of failure. However, this system can be made robust by adding more than one key generator. Different key-generators will work on different ranges of numbers. We can even place a load balancer in front of them to evenly distribute the workload.

Now, we will discuss how to display a feed for a particular user. There could be various strategies for the same, which are discussed as follows:

- We can put an additional field tag with every photo entry in the database. We may have tags such as nature, animals, city, and so on. Now, every user can specify tags for which he/she is interested to see pictures. So, for a particular user, we can fetch the tags he/she is interested in from the user database, and then we will query the photo

database using these tags to retrieve some 50 or 100 pictures. These photos will then be displayed in the user's feed.

- Another approach could be to sort photos based on their creation time so that we always show the latest pictures in the feed of the users. In order to sort photos, we can append a photo creation timestamp to the photo id, and we can index the database based on this photo id. The indexing will help us to quickly retrieve the latest photos.

The next question is how to send feed content to the users. We can adopt two approaches for this:

Whenever a user is online and goes to his feed to pull new data, then only we can perform the operation to fetch feed contents.

We can use a long polling technique in which the client feed will be updated whenever a new photo is added. In this case, the user does not have to pull new data. This approach has a serious downside. What if a tag is used by millions or billions of users? In that case, the operation will be very expensive.

So, a prudent approach would be to do a mix of both techniques.

It is imperative to make a note that the users can be geographically distributed, and serving photos at a large distance could be a bandwidth-consuming operation. Moreover, database read operation for the images is also an expensive operation. To optimize this, we can use **Content Delivery Network** (**CDN**) servers. CDN, as we know, is a group of distributed servers that provides high performance by placing the servers close to the users. Also, we can use a cache like Memcache to place pictures falling in mostly followed tags so that we can quickly return the pictures from the cache rather than querying the database.

Conclusion

In this chapter, we discussed how to create a scalable image hosting service. We have used several technologies and methodologies we learned in previous chapters, such as database sharding, cache, and content delivery network for this application as well. There are many other aspects of the system, such as how the user will log in, how images can be shared between a particular set of users, and many other things which are not discussed in

this chapter. It is up to the discussion panel to decide which things are important to be considered for the discussion. Readers are encouraged to pursue the learning of different concepts and technologies mentioned in this chapter so as to feel confident in tackling any system design challenge.

Points to remember

- The single responsibility principle states that a particular module or function should be responsible for a single functionality of the program.
- Content Delivery Network is a distributed network of servers that aims to speed up data access by bringing servers closer to the user.
- A load balancer is used to distribute the network evenly among a group of servers.

Multiple choice question

1. **Which of the following is true for cache?**
    a. A cache is a high-speed memory to speed up data access.
    b. A cache is used to evenly distribute traffic among a group of servers.
    c. A cache is used to secure data access.
    d. A cache is used to authenticate a user.

Answer

1. **a**

Questions

1. What is a Content Delivery Network?
2. How can you scale the database horizontally?

Key terms

- CDN stands for Content Delivery Network.

Index

A

abstraction
  about 85
  classes, using 85
  header files, using 85
Advance Messaging Queue Protocol (AMQP) 64
algorithm
  approach 131-134
  identifying 131
application components
  file divider 157
  file service 158
  file synchronizer 157
  internal file database 157
application layer
  about 37
  protocols 37
  services 37
asynchronism 16, 17
Atomicity, Consistency, Isolation, and Durability (ACID) 23
availability 146
Azure Kubernetes Service (AKS) 67

B

back-of-the-envelope calculations 128, 129
base class 76
behavioral design pattern 95
BlockService 156
blue-green deployment 67
bridge design pattern 99
business communication tool 169
  back-of-the-envelope calculations 166, 167
  data access, scaling 172, 173
  functional requirement 166
  high-level design 168
  messages, storing 170-172
  non-functional requirement 166
  scalable system, designing 167
  system requirement, defining 166

C

cache
  about 7, 8, 138
  distributed cache 10
  global cache 9, 10
  local cache 8
cache cluster 10
canary release 67
CAP theorem 20, 21
CAP theorem, properties
  availability 21
  consistency 21
  partition tolerance 21
card authentication system
  back-of-the-envelope calculations 176
  cache 181
  design components, scaling 177, 178
  functional requirement 176
  load balancer 181
  non-functional requirement 176
  scalable system design, creating 177
  security 181, 182
  system requirement, defining 176
chain of responsibility 95, 96
CI/CD process
  for microservices 65
CI/CD process, strategies
  blue-green deployment 67
  canary release 67
  rolling updates 66
class 74, 75
class diagram 116
cloud storage 160
Common Application Service Element (CASE) 36
compile-time polymorphism 83, 84
composition 122
connection load balancing 51
consistency 146-148
Content Delivery Network (CDN) 17-19, 192
Continuous Delivery (CD) 66
Continuous Deployment 66
Continuous Integration (CI) 66
creational design patterns
  about 90
  behavioral design pattern 95
  bridge design pattern 99
  chain of responsibility 95, 96
  decorator design pattern 101
  flyweight design pattern 102, 103
  mediator design pattern 96
  object pool design pattern 90, 91

observer design pattern 98
prototype design pattern 92
proxy design pattern 104, 106
singleton design pattern 93, 94
structural design pattern 99

D

data access layer 58
data access scaling
cache 7, 8
principles 7
proxy servers 11, 12
database
scaling 146
database cleanup 138, 139
Database Management System (DBMS) 22
databases
about 21, 22
graph database 24
non-relational database 22, 23
relational database 22, 23
spatial database 23, 24
database scaling methods
about 48
connection, establishing 50
connection load balancing 51
multi-master replica 49, 50
read replicas 48, 49
Datagram Delivery Protocol (DDP) 34
data indexing
about 12, 13
asynchronism 16, 17
load balancers 13
data-link layer
about 33
logical link control (LLC) 33
media access control (MAC) 33
services 33
data partition 134-138
data store layer 58
DBMS functions
data administration 22
data definition 22
data manipulation 22
data retrieval 22
decorator design pattern 101
derived class 76
design patterns
about 90

types 90
uses 90
distributed cache 10
distributed scalable systems
about 2, 3
bottlenecks, in designing 3, 4
distributed scalable systems, parameters
availability 4
concurrent users 5
cost 6
database access 4
latency 5
manageability 5
reliability 5
distributed system 2
durability 147, 148

E

encapsulation 85, 86
Enterprise Service Bus (ESB) 64

F

file hosting service
back-of-the-envelope calculations 145
high-level design, defining 149
high-level design, re-defining 148
scalable system, creating 155, 157
system requirement, defining 154, 155
file Metadata database 160
flyweight design pattern 102, 103
full-duplex 32
functional requirement
defining 127
function overloading 83
function overriding 84
FURPS+ model constraint
design constraint 112
implementation constraint 112
interface constraint 112
physical constraint 112
FURPS+ requirement
about 111
functionality 111
performance 112
reliability 112
supportability 112
usability 112

G

global cache 9, 10
graph database 24

H

half-duplex 32
hash table 135
hierarchical inheritance 79, 80
high-level design
  about 146
  re-defining 148, 149, 161
horizontal scaling 46, 47
HTTP long-polling 169
hybrid inheritance 81
Hypertext Transfer Protocol (HTTP) 37, 38

I

image hosting application
  functional requirement, defining 186
  non-functional requirement, defining 186
  scalable system, creating 187-192
  system requirement, defining 186
inheritance 76
Internet Control Message Protocol (ICMP) 34
Internet Protocol (IP) 38

J

Java Messaging Service (JMS) 64

K

key-value store
  characteristics 144
  functional requirement 144
  non-functional requirement 144

L

Least Recently Used (LRU) 161
load balancers
  about 13, 14
  algorithms 15
  types 14, 15
local cache 8
logical link control (LLC) 33

M

media access control (MAC) 33
mediator design pattern 96
MetaService 156
microservices
  about 62
  advantages 62
  disadvantages 63
  versus monolithic systems 65
  versus SOA 64
Microsoft Messaging Queue (MSMQ) 64
monolithic systems
  about 56-58
  versus microservices 65
multi-level inheritance 77
multi-master replica 49, 50
multiple inheritance 78, 79
multi-tenant architecture 166

N

network 30, 31
network layer
  about 34
  services 34
non-functional requirement
  defining 127
non-relational database 22, 23
NoSQL
  about 170
  versus SQL 130, 131
NotifyService
  about 156
  core operations 158
  working 158, 159

O

object 75, 76
object-oriented analysis and design
  class diagram 116-122
  performing 112
  system requirement, defining 113-116
object-oriented design methodology
  about 110, 111
  FURPS+ requirement 111
object-oriented programming (OOP)
  about 72, 73
  abstraction 85

  class 74, 75
  encapsulation 85, 86
  features 74
  hierarchical inheritance 79, 80
  inheritance 76
  multi-level inheritance 77
  multiple inheritance 78, 79
  object 75, 76
  single inheritance 76
object pool 90, 91
object store 189
observer design pattern 98
Open System Interconnection (OSI) 31
OSI model
  about 31
  application layer 37
  data-link layer 33
  network layer 34
  physical layer 32
  presentation layer 36
  session layer 35
  transport layer 34

P

parent class 76
performance 6
physical layer
  about 32
  services 32
polymorphism 83
  types 83
port-to-port protocol 35
presentation layer
  about 36
  functions 36
  protocols 37
prototype design pattern 92
proxy design pattern
  about 104-106
  purpose 105
proxy servers 11, 12

R

read replicas 48, 49
redundancy 24, 25
relational database 22, 23
Representational State Transfer (REST) 40, 64
Request queue 159

Response queue 159
REST architecture 40
RESTful Web application
  elements 40
RESTful Web service
  cacheable 41
  client-server 41
  code on demand 41
  layered system 41
  stateless 41
  uniform interface 41
reverse proxies
  advantages 12
runtime polymorphism 84

S

scalability 6
scalable system
  availability 146
  consistency 146-148
  creating 146
  database, scaling 146
  durability 147, 148
  high-level design 146
scalable system design
  database partitioning technique 179, 180
scalable system design, SQL database
  card table 179
  group table 178
  user table 178
service-oriented architecture (SOA)
  about 58-60
  approach, implementing 61
  characteristics 59
  core values 59
  versus microservices 64
session layer
  about 35
  Common Application Service Element (CASE) 36
  functions 36
  Specific Application Service Element (SASE) 37
Simple Object Access Protocol (SOAP)
  about 59
  characteristics 61
  working 61
simplex 32
single inheritance 76
singleton design pattern 93, 94
spatial database 23, 24

Specific Application Service Element (SASE) 37
SQL
  about 170
  versus NoSQL 130, 131
structural design pattern 99
subclass 76
super class 76
system design
  overview 126
  requirement, obtaining 126
  scaling 129, 130
system requirement
  defining 144

T

translation layer 36
Transmission Control Protocol (TCP)
  about 15, 35, 38
  close connection 39
  connection, establishing 39
  data packet, sending 39
  features 39
  versus User Datagram Protocol (UDP) 40
transport layer
  about 34, 35
  functions 35

U

Uniform Resource Identifier (URI) 41
User Datagram Protocol (UDP)
  about 35, 39
  versus Transmission Control Protocol (TCP) 40
user interface layer 58
user workflows
  defining 155

V

vertical scaling 47, 48

W

Web Service Description Language (WSDL) 59
WebSockets 169